

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных средств

К защите допустить:
Заведующий кафедрой ЭВС
_____ И.С. Азаров

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к дипломному проекту
на тему

**IP-ЯДРО НЕЙРОННОЙ СЕТИ ПРЯМОГО РАСПРОСТРАНЕНИЯ ДЛЯ
РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР**

БГУИР ДП 1-40 02 02 01 014 ПЗ

Студент	Е.А. Кривальцевич
Руководитель	М.И. Вашкевич
Консультанты: <i>от кафедры ЭВС</i>	М.И. Вашкевич
<i>по экономическому разделу</i>	И.В. Смирнов
Нормоконтролёр	Д.С. Лихачёв
Рецензент	

Минск 2025

Решением рабочей комиссии допущен(а) к
защите дипломного проекта.

Председатель рабочей комиссии

_____ (_____)

Подпись

Инициалы и фамилия

« ____ » _____ 2025 г.

РЕФЕРАТ

IP-ЯДРО НЕЙРОННОЙ СЕТИ ПРЯМОГО РАСПРОСТРАНЕНИЯ ДЛЯ РАСПОЗНОВАНИЯ РУКОПИСНЫХ ЦИФР : дипломный проект / Е. А. Кривальцевич. – Минск : БГУИР, 2025, – п.з. – 79 с., чертежей (плакатов) – 6 л. формата А1.

Основной задачей данного дипломного проекта является аппаратная реализация нейронной сети, использующей слой двумерного разделимого преобразования. В нем будет рассмотрен подход увеличения точности распознавания рукописных цифр при незначительном увеличении количества параметров нейронной сети.

Предложенное решение по увеличению точности распознавания направлено на оптимизацию аппаратных затрат нейронной сети.

В целом предложенное решение по совершенствованию полносвязных нейронных сетей позволит значительно повысить эффективность.

Ключевые слова: нейронная сеть, двумерное разделимое преобразование, MNIST, ПЛИС.

Министерство образования Республики Беларусь
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Кафедра

Электронных вычислительных средств

УТВЕРЖДАЮ
Зав. кафедрой ЭВС
_____ И.С. Азаров
«06» марта 2025 г.

ЗАДАНИЕ
на дипломный проект

Обучающемуся

Кривальцевичу Егору Александровичу
(фамилия, собственное имя, отчество (если таковое имеется))

Курс 4

Учебная группа 150701

Специальность 1-40 02 02 «Электронные вычислительные средства»

Тема дипломного проекта IP-ядро нейронной сети прямого распространения для распознавания рукописных цифр

Утверждена приказом ректора

«06» марта 2025 г. № 584-с

Исходные данные к дипломному проекту

Назначение разработки: система предназначена для распознавания рукописных цифр.

Технические характеристики:

- размер изображения 28x28;
- цвет изображения: в оттенках серого;
- функция активации: tanh, softmax;
- формат представления данных: фиксированная запятая;
- структура сети: многослойная прямого распространения;
- тактовая частота работы IP-ядра не менее 80 МГц
- аппаратная платформа – Xilinx Zynq-7000, xc7z010clg400-2

Перечень подлежащих разработке вопросов или краткое содержание расчетно-пояснительной записки

1 Введение. 2 Обзор аналогичных разработок. 3 Анализ ТЗ. 4 Разработка нейронной сети для распознавания рукописных цифр. 5 Разработка структуры IP-блока нейронной сети. 5.1 Представление исходных данных IP-блока нейронной сети. 5.2 Программная реализация модели IP-блока нейронной сети в арифметике с фиксированной запятой на языке Python. 5.3 Разработка электрической функциональной схемы. 6 Аппаратная реализация IP-блока нейронной сети для распознавания рукописных цифр. 7 Технико-экономическое обоснование дипломного проекта. 8 Анализ результатов проектирования. 9 Заключение. 10 Список используемых источников.

Перечень графического материала (с точным указанием обязательных чертежей и графиков)

- 1 Схема электрическая структурная IP-блока нейронной сети – 1 лист формата A2.
- 2 Схема электрическая функциональная IP-блока нейронной сети – 1 лист формата A1.
- 3 Схема электрическая функциональная процессорных элементов IP-блока нейронной сети – 1 лист формата A2.
- 4 Схема алгоритма работы IP-блока нейронной сети – 1 лист формата A1.
- 5 Постановка задачи – 1 лист формата A1 (плакат).
- 6 Результаты экспериментов – 1 лист формата A1 (плакат).
- 7 Результаты проектирования – 1 лист формата A1 (плакат).

Консультанты по дипломному проекту (с указанием разделов, по которым они консультируют)

Старший преподаватель кафедры экономики Смирнов И.В., «Экономическое обоснование разработки нейронной сети для распознавания рукописных цифр»

Примерный календарный график выполнения дипломного проекта

Наименование этапов дипломного проекта	Объем этапа в %	Срок выполнения этапа
I этап	40	24.03.25
II этап	20	11.04.25
III этап	20	05.05.25
Нормоконтроль		12.05.25 – 21.05.25
Рабочая комиссия		22.05.25 – 30.05.25
Рецензирование		02.06.25 – 12.06.25
Защита		13.06.25 – 30.06.25 (в соответствии с графиком заседаний ГЭК)

Дата выдачи задания

« » марта 2025 года

Срок сдачи студентом законченного дипломного проекта

«13» июня 2025 года

Руководитель дипломного проекта

(подпись)

(инициалы, фамилия)

Подпись обучающегося

(подпись)

Дата « 6 » марта 2025 г

СОДЕРЖАНИЕ

Введение	8
1 Обзор существующих нейронных сетей для распознавания рукописных цифр	9
1.1 Нейронные сети	9
1.2 Существующие нейронные сети для распознавания рукописных цифр	9
1.3 Полносвязные нейронные сети	10
1.4 Сверточные нейронные сети	15
1.5 Рекуррентные нейронные сети	17
1.6 Трансформеры нейронные сети	19
1.7 Гибридные архитектуры	20
2 Анализ технического задания	22
2.1 Постановка задачи	22
2.2 Анализ требований к нейронной сети	23
2.3 Анализ требований к аппаратной реализации	24
2.4 Выбор и обоснование метода решения задачи	24
3 Разработка нейронной сети для распознавания рукописных цифр	26
3.1 Функциональная спецификация системы	26
3.2 Разбиение системы на модули	26
3.3 Принцип работы LST преобразования	27
3.4 Обучение нейронной сети	30
4 Разработка структуры IP-блока нейронной сети для распознавания рукописных цифр	33
4.1 Описание структурной схемы	33
4.2 Программная реализация эталонной модели нейронной сети на языке Python	36
4.3 Разработка операционной части нейронной сети	37
4.4 Проектирование функциональной схемы нейронной сети	39
4.5 Построение алгоритма работы нейронной сети	40
4.6 Проектирование управляющей части	40
4.7 Построение графа переходов управляющего автомата	43
5 Аппаратная реализация IP-блока нейронной сети для распознавания рукописных цифр	45
5.1 Описание пакета Xilinx Vivado	45
5.2 Описание функциональных блоков	46
5.3 Описание интерфейса нейронной сети	46
5.4 Описание процесса создания блочной диаграммы	47
5.5 Результаты синтеза и симуляции	53
5.6 Устройство основных функциональных блоков	58
6 Анализ результатов тестирования IP-блока нейронной сети для распознавания рукописных цифр	63
6.1 Описание среды тестирования	63
6.2 Тестирование разработанного IP-блока нейронной сети	63
6.3 Экспериментальное исследование IP-блока нейронной сети	64
6.4 Сравнение с аналогичными архитектурами	66
7 Техничко-экономическое обоснование разработки IP-блока нейронной сети для распознавания рукописных цифр	68
7.1 Характеристика программного средства, разрабатываемого для собственных нужд	68

7.2 Расчет инвестиций в разработку программного средства для собственных нужд	68
7.3 Расчет экономического эффекта от использования программного средства для собственных нужд	69
7.4 Расчет показателей экономической эффективности разработки и использования программного средства в организации	69
Заключение	71
Список использованных источников	72
Приложение А (Обязательное) Отчет о проверке на заимствование	74
Приложение Б (Обязательное) Python описание обучения сети	75
Приложение В (Обязательное) Python описание эталонной сети	79
Приложение Г (Обязательное) Verilog описание устройства	85
Приложение Д (Обязательное) Python скрипты тестирования	104

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ, СИМВОЛОВ И ТЕРМИНОВ

LST – Learned 2D Separable Transform, Обучаемое двумерное разделимое преобразование.

CPU – центральный процессор.

GPU – графический процессор.

FPGA – Field-Programmable Gate Array.

ASIC – Application-Specific Integrated Circuit.

ПЛИС – Программируемая логическая интегральная микросхема.

IP - Intellectual Property.

MLP - многослойный перцептрон.

FCNN – Fully Connected Neural Network, полносвязная нейронная сеть.

CNN –

ВВЕДЕНИЕ

В последние годы значительный интерес вызывает аппаратная реализация нейронных сетей, поскольку программные решения, работающие на центральных процессорах (CPU) и графических процессорах (GPU), не всегда обеспечивают требуемую скорость обработки и энергоэффективность. В связи с этим использование специализированных аппаратных ускорителей на базе FPGA (Field-Programmable Gate Array) или ASIC (Application-Specific Integrated Circuit) становится актуальной задачей. Аппаратная реализация нейронной сети в виде IP-ядра позволяет значительно повысить производительность и снизить задержки обработки данных, что особенно важно для встроенных систем.

При реализации нейронных сетей на базе CPU и GPU как правило используются стандартизированные типы данных. При реализации на базе FPGA появляется возможность использовать для представления параметров нейронных сетей типов данных, обеспечивающих различную точность. Причем выбор точности представления напрямую будет влиять на аппаратные затраты[1].

Целью данного дипломного проекта является разработка IP-ядра нейронной сети прямого распространения для распознавания рукописных цифр. В данном проекте используется обученное двумерное разделяемое преобразование (LST2D), которое рассматривается как новый тип слоя нейронной сети. Он следует концепции распределения веса. Проектирование IP-ядра нейронной сети прямого распространения производится в несколько этапов, отраженных в структуре пояснительной записки данного дипломного проекта.

Первоначально проводится анализ существующих методов и архитектур нейронных сетей, применяемых для задач распознавания рукописных цифр. Рассматриваются различные подходы, включая классические многослойные перцептроны, свёрточные и рекуррентные нейронные сети. Итоги данного этапа изложены в первом разделе пояснительной записки.

На следующем этапе выполняется анализ технического задания, в результате которого была выбрана архитектура нейронной сети, наиболее подходящая для аппаратной реализации. Основные критерии выбора включают компактность модели и вычислительную эффективность. Результаты этого анализа представлены во втором разделе пояснительной записки.

В третьем и четвертом разделах пояснительной записки подробно описываются процесс проектирования IP-ядра и обучения нейронной сети, особенности аппаратной реализации и взаимодействие с внешними устройствами.

Для проверки работоспособности IP-ядра проектируется его программная модель. Реализация и тестирование работы модели на тестовом наборе данных MNIST выполняются в среде Python и Vivado. Эти этапы освещены в пятом разделе пояснительной записки.

Следующим этапом является тестирование разработанного IP-ядра. Также производится сравнение с существующими решениями. Результаты данного анализа представлены в седьмом разделе пояснительной записки.

Завершающим этапом технико-экономическое обоснование разработки IP-ядра, включающее анализ затрат на реализацию и оценку преимуществ аппаратного ускорения по сравнению с программными методами. Результаты данного этапа приведены в шестом разделе пояснительной записки.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет 82%. Отчет о проверке на заимствования представлен в приложении А.

1 ОБЗОР СУЩЕСТВУЮЩИХ НЕЙРОННЫХ СЕТЕЙ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

1.1 Нейронные сети

Нейронная сеть представляет из себя совокупность нейронов, соединенных друг с другом определенным образом. Нейрон представляет из себя элемент, который вычисляет выходной сигнал из совокупности входных сигналов. Структура нейрона представлена на рисунке 1.1.

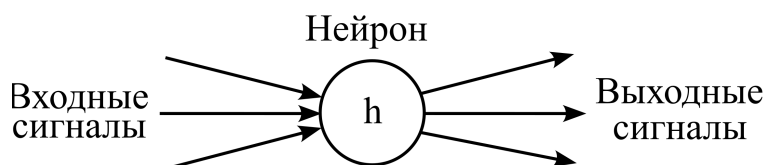


Рисунок 1.1 – Общее представление нейрона

Нейроны выполняют следующую последовательность действий:

- приём входных сигналов;
- комбинирование входных сигналов;
- вычисление выходных сигналов;
- передача выходных сигналов следующим нейронам.

Между собой нейроны могут быть соединены абсолютно по-разному, это определяется структурой конкретной сети. Искусственные нейронные сети можно рассматривать как направленный граф со взвешенными связями, в котором искусственные нейроны являются узлами. По архитектуре связей они могут быть сгруппированы в два класса: сети прямого распространения, в которых графы не имеют петель, и рекуррентные сети (RNN), или сети с обратными связями.

1.2 Существующие нейронные сети для распознавания рукописных цифр

Распознавание рукописных цифр является одной из классических задач машинного обучения, для решения которой применяется широкий спектр нейронных сетей. В зависимости от сложности задачи, требований к точности, скорости обработки и аппаратных ограничений используются различные архитектуры, включая полносвязные сети (FCNN), сверточные нейронные сети (CNN) и более сложные гибридные модели.

Одним из первых методов распознавания рукописных цифр стали многослойные перцептроны (MLP), относящиеся к классу полносвязных нейронных сетей. Такие сети состоят из входного слоя, нескольких скрытых слоев и выходного слоя, где каждый нейрон соединен со всеми нейронами предыдущего и последующего слоев. Они способны успешно решать задачу распознавания, но требуют большого количества параметров и вычислительных ресурсов, что делает их менее эффективными.

Для обработки изображений, включая рукописные цифры, более эффективными оказались сверточные нейронные сети. Эти сети включают сверточные слои, которые извлекают характерные признаки из изображения. CNN значительно превосходят полносвязные сети в точности и скорости работы, поскольку

используют пространственные зависимости в данных и требуют меньше параметров.

Хотя рекуррентные нейронные сети и их усовершенствованные версии, такие как Long Short-Term Memory Network (LSTM) и Gated Recurrent Unit Network (GRU), чаще применяются для обработки последовательных данных, они могут использоваться и для распознавания рукописных цифр. В частности, они эффективны в случаях, когда рукописные символы анализируются в контексте строки.

Современные архитектуры, такие как Vision Transformer (ViT) и его модификации, предлагают альтернативный подход к обработке изображений, основанный на механизме самовнимания. Хотя традиционно трансформеры использовались в обработке естественного языка (NLP), их успешное применение в компьютерном зрении позволило достичь новых высот в распознавании символов и цифр.

Для повышения точности и эффективности могут применяться гибридные подходы, комбинирующие преимущества разных типов нейронных сетей. Например, модели, совмещающие CNN и LSTM, успешно применяются для распознавания рукописного текста, где CNN используется для выделения признаков, а LSTM — для анализа последовательностей.

В последующих разделах будет представлен детальный анализ наиболее распространённых нейросетевых моделей, применяемых для данной задачи, а также рассмотрены их основные особенности. Кроме того, будет приведена общая информация о различных функциях активации, используемых в современных архитектурах нейронных сетей.

1.3 Полносвязные нейронные сети

Полносвязная нейронная сеть (Fully Connected Neural Network) — это базовая архитектура искусственных нейронных сетей, в которой каждый нейрон одного слоя соединен со всеми нейронами следующего слоя. Такая архитектура чаще всего применяется в качестве классификатора после сверточных или рекуррентных слоев. Количество нейронов на каждом слое может отличаться от соседних слоев.

1.3.1 Полносвязная нейронная сеть состоит из следующих основных компонентов:

- 1 Входной слой — принимает данные.
- 2 Скрытые слои — выполняют нелинейные преобразования входных данных. Обычно включают несколько таких слоев.
- 3 Выходной слой — содержит количество нейронов, соответствующее числу классов, и использует функцию активации, например softmax. Функция активации — это математическая функция, которая определяет, передаст ли нейрон сигнал дальше по сети.

Математически данную сеть можно описать формулой для вычисления выхода нейрона в полносвязном слое

$$h = f(Wx + b), \quad (1.1)$$

где x — входной вектор;
 W — матрица весов;

b – вектор смещений;
 $f(\cdot)$ – функция активации;
 h – выходной вектор нейронов.

Общая структура полносвязных нейронных сетей показана на рисунке 1.2.

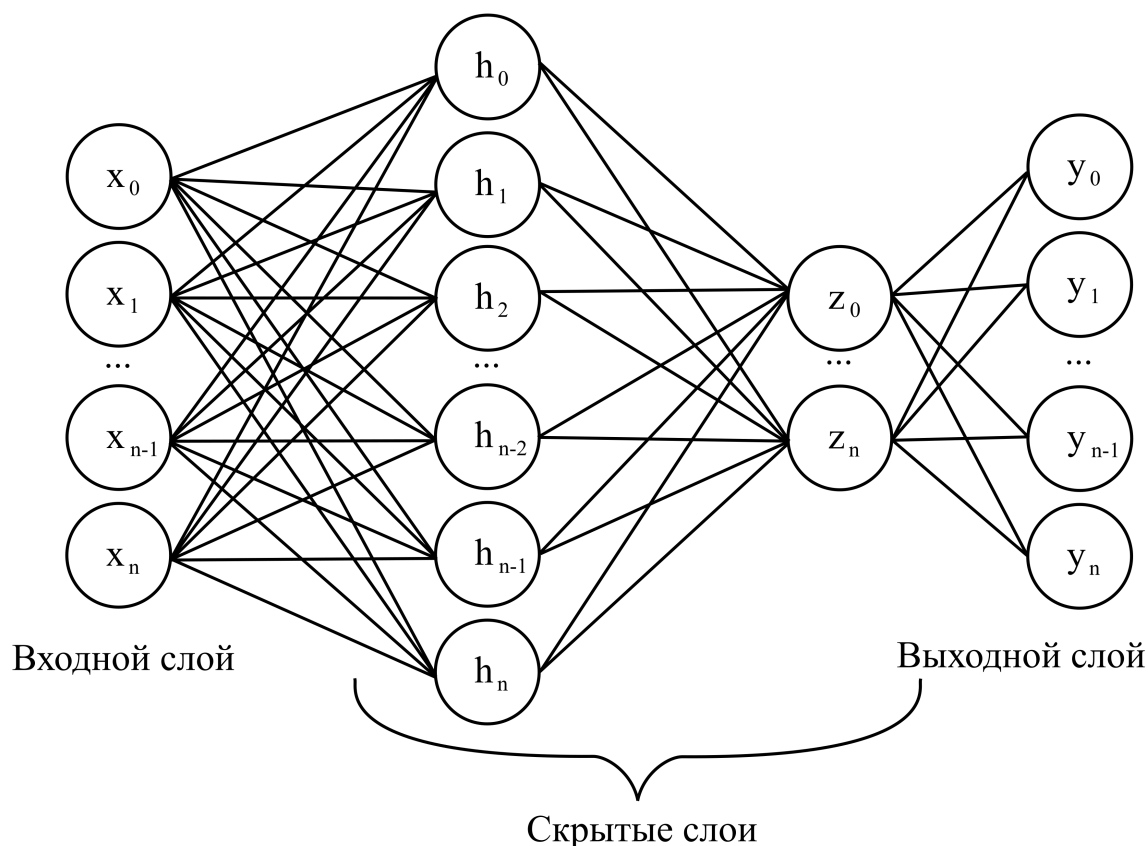


Рисунок 1.2 – Общая структура полносвязных нейронных сетей

Для многослойной архитектуры выход слоя l является входом для следующего слоя. В следствии чего, уравнения (1.1) можно преобразовать, с учетом многослойности, к виду

$$h^{(l+1)} = f(W^{(l)}h^{(l)} + b^{(l)}). \quad (1.2)$$

где $W^{(l)}$ – матрица весов текущего слоя;
 $b^{(l)}$ – вектор смещений текущего слоя;
 $h^{(l)}$ – выходной вектор нейронов текущего слоя;
 $f(\cdot)$ – функция активации;
 $h^{(l+1)}$ – выходной вектор нейронов следующего слоя.

1.3.2 ReLU (Rectified Linear Unit) — одна из наиболее популярных и широко используемых функций активации в нейронных сетях. Она определяется следующим образом

$$f(x) = \max(0, x). \quad (1.3)$$

где x – входной вектор;

$\max(\cdot)$ – операция выбора максимального значения из всех элементов в окне.

График функции ReLU представлен на рисунке 1.3.

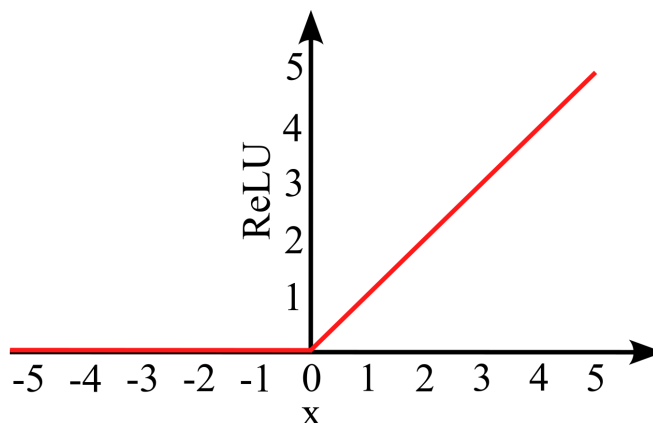


Рисунок 1.3 – График функции ReLU

Функция ReLU ускоряет обучение сети и устраняет проблему исчезающего градиента, так как имеет линейное поведение для положительных значений. Сама функция проста в реализации, но может быть чувствительна к большим значениям градиентов и в отрицательной области возвращает 0.

1.3.3 Сигмоида (Logistic Function) — это одна из классических функций активации, используемая в нейронных сетях. Математическое представление в виде формулы

$$f(x) = \frac{1}{1 + e^{-x}}. \quad (1.4)$$

где x – входной вектор.

Сигмоида подходит для моделирования вероятностей, так как имеет диапазон значений $(0, 1)$. Функция гладкая и дифференцируемая, что упрощает вычисление градиента, однако выходы не центрированы вокруг нуля, что замедляет обучение.

График функции сигмоида представлен на рисунке 1.4.

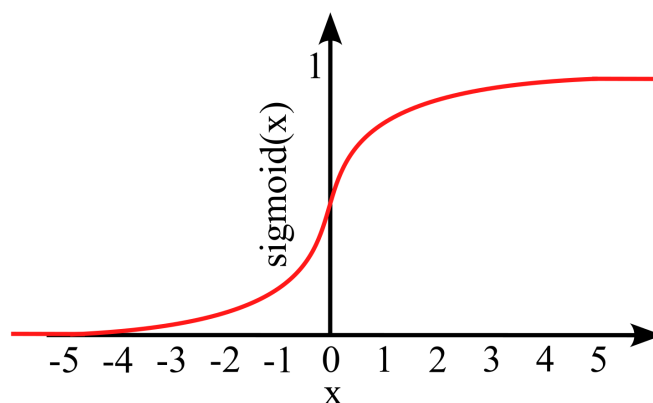


Рисунок 1.4 – График функции сигмоида

1.3.4 Функция Softmax применяется в многоклассовой классификации и преобразует входные значения в вероятностное распределение

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}, \quad (1.5)$$

где z_i — входное значение для i -го класса.

Гистограмма функции Softmax представлен на рисунке 1.5.

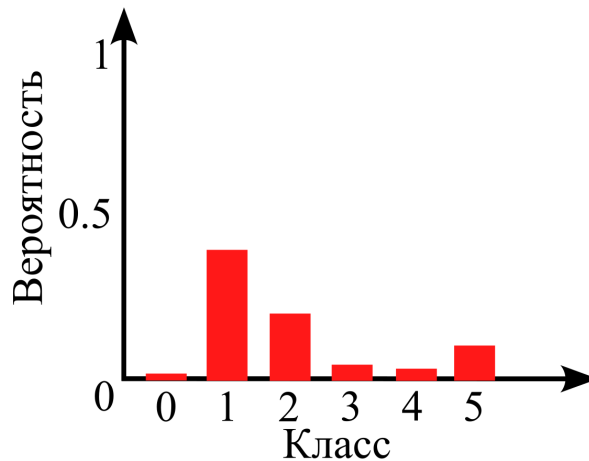


Рисунок 1.5 – Гистограмма функции Softmax

Softmax позволяет интерпретировать выходные значения сети как вероятности принадлежности к классам. Однако склонна к затуханию градиента при слишком больших входных значениях.

1.3.5 Гиперболический тангенс (Tanh) — это сигмоидная функция, но симметричная относительно начала координат. Он определяется следующим образом

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}. \quad (1.6)$$

где x — входной вектор.

Функция принимает значения в диапазоне $(-1, 1)$, что делает её более предпочтительной по сравнению с сигмоидой в скрытых слоях нейронных сетей. Основное преимущество функции $\tanh(x)$ заключается в том, что производная функции принимает большие значения в среднем диапазоне, что уменьшает проблему исчезающего градиента по сравнению с сигмоидой.

Кроме того, благодаря симметрии относительно нуля, выходы функции $\tanh(x)$ сбалансированы — среднее значение ближе к нулю, что может способствовать более быстрой сходимости модели во время обучения. В отличие от сигмоидальной функции, где все значения положительные, $\tanh$ помогает избежать смещения активаций в одну сторону. Однако при больших по модулю входных значениях функция также подвержена эффекту насыщения, где градиенты становятся близкими к нулю.

График функции гиперболический тангенс представлен на рисунке 1.6.

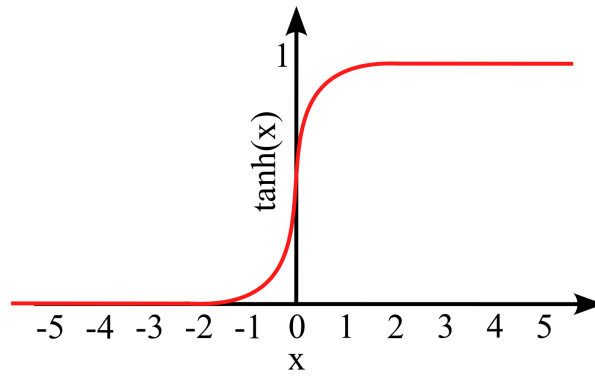


Рисунок 1.6 – График функции гиперболический тангенс

1.3.6 Обучение FCNN осуществляется с использованием метода обратного распространения ошибки (Backpropagation) и оптимизационного алгоритма, например, стохастического градиентного спуска (SGD)

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}, \quad (1.7)$$

$$b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}, \quad (1.8)$$

где η – скорость обучения;

L – функция потерь, например, кросс-энтропия для классификации.

Во время обратного распространения градиенты функции потерь вычисляются по отношению к весам и смещениям сети, после чего параметры обновляются в сторону, уменьшающую значение ошибки. Этот процесс повторяется итеративно для каждого батча данных, что позволяет модели постепенно улучшать свою способность к предсказанию.

Энтропия измеряет степень неопределенности в распределении вероятностей. Кросс-энтропия, в свою очередь, оценивает расхождение между двумя распределениями: истинным и предсказанным. Чем ближе $\hat{y}_i$ к y_i , тем меньше значение функции потерь.

Функцию кросс-энтропии описывается формулой

$$L = - \sum_i y_i \log(\hat{y}_i), \quad (1.9)$$

где y_i – истинное значение (обычно one-hot вектор);

$\hat{y}_i$ – вероятность, предсказанная моделью для класса i .

Кросс-энтропия особенно эффективна в задачах многоклассовой классификации, так как сильнее штрафует за уверенные, но неверные предсказания. Это способствует более быстрому обучению модели и лучшей сходимости.

1.3.7 Структура однослойной нейронной сети прямого распространения, состоящей из полносвязного слоя с выходной функцией активации softmax показана на рисунке 1.7.

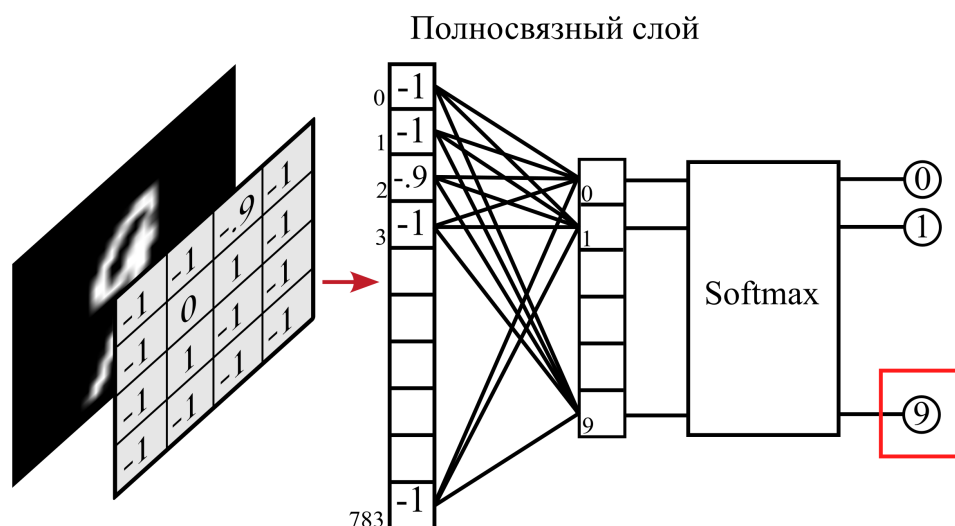


Рисунок 1.7 – Однослойной нейронной сети прямого распространения

Данная архитектура позволяет добиться точности 92,4%[2]. Нейронная сеть использует 8624 параметра, которые необходимо хранить в памяти в качестве весовых коэффициентов и смещений.

1.4 Сверточные нейронные сети

Сверточные нейронные сети (Convolutional Neural Networks) представляют собой архитектуру глубокого обучения, предназначенную для обработки данных обладающих сетчатой топологией, таких как изображения. Их основное преимущество заключается в способности эффективно извлекать пространственные и иерархические особенности входных данных с помощью операций свертки.

Принцип работы сверточных нейронных сетей показан на рисунке 1.8[3].

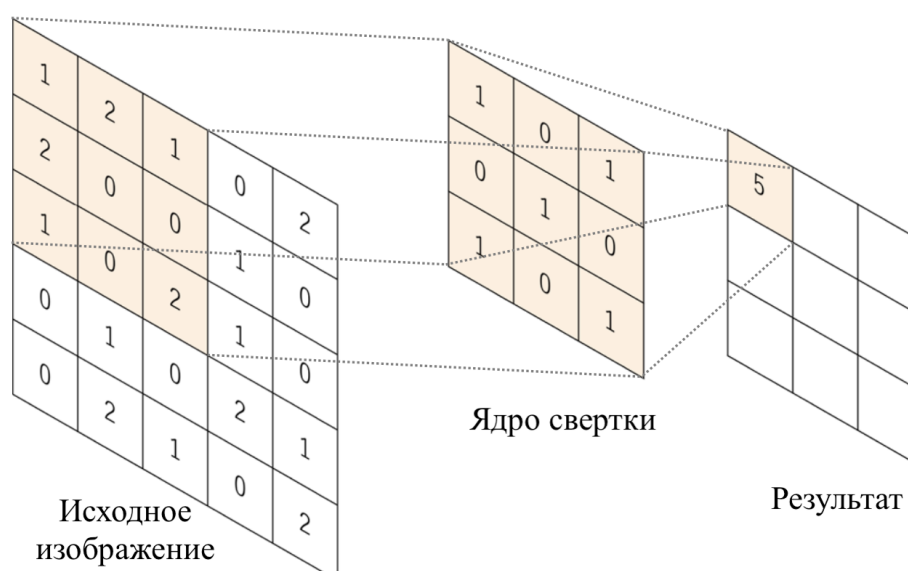


Рисунок 1.8 – Принцип работы сверточных нейронных сетей

1.4.1 Стандартная архитектура CNN включает в себя следующие основные слои:

1 Сверточные слои – выполняют операцию свертки, выделяя значимые признаки входных данных, такие как границы, текстуры и формы.

2 Функция активации – вводит нелинейность, позволяя сети аппроксимировать сложные зависимости.

3 Слои подвыборки (Pooling) – уменьшают размерность представления, что снижает количество параметров и вычислительную сложность.

4 Полносвязные слои – традиционные нейронные слои, выполняющие классификацию извлеченных признаков.

5 Функция потерь – используется для оценки ошибки модели.

1.4.2 Основная операция в сверточных нейронных сетях – это свертка (convolution), которая заключается в пошаговом применении фильтра (ядра свёртки) к локальным областям входного изображения. Формально операция свёртки между входным изображением X и ядром K определяется следующим образом

$$S(i, j) = \sum_m \sum_n X(i + m, j + n) \cdot K(m, n), \quad (1.10)$$

где $X(i, j)$ – входное изображение;

$K(m, n)$ – ядро свертки;

$S(i, j)$ – выходное изображение после свертки.

Сверточный слой применяется к входному изображению, используя несколько фильтров, каждый из которых извлекает определенные характеристики. Как правило, чем глубже слой, тем более абстрактные признаки он способен захватывать.

1.4.3 Для уменьшения размерности карт признаков применяется операция подвыборки. Один из наиболее распространенных методов – max pooling, который выбирает максимальное значение в окне $k \times k$

$$P(i, j) = \max_{(m,n) \in k \times k} S(i + m, j + n). \quad (1.11)$$

где $P(i, j)$ – значение на выходе слоя подвыборки в позиции (i, j) ;

$S(i + m, j + n)$ – значение входной карты признаков в позиции $(i + m, j + n)$;

$\max(\cdot)$ – операция выбора максимального значения из всех элементов в окне.

Этот метод помогает уменьшить объем вычислений и делает модель более устойчивой к сдвигам изображения.

1.4.4 Функции потерь измеряют отклонение предсказаний модели от истинных значений. Выбор функции зависит от типа задачи:

1 Cross-Entropy Loss — используется в задачах классификации, усиливает влияние ошибочных предсказаний.

2 Mean Squared Error (MSE) — применяется в регрессии, чувствительна к выбросам.

3 Mean Absolute Error (MAE) — альтернатива MSE, менее чувствительная к выбросам.

4 Huber Loss — сочетает MSE и MAE; более устойчив к выбросам.

5 Focal Loss — модифицированная кросс-энтропия для задач с дисбалансом классов.

1.4.5 Обучение CNN происходит с использованием алгоритма обратного распространения ошибки и градиентного спуска. Для обновления параметров используется правило

$$w^{(t+1)} = w^{(t)} - \eta \frac{\partial L}{\partial w}, \quad (1.12)$$

где η — коэффициент обучения;

$\frac{\partial L}{\partial w}$ — градиент функции потерь по параметру w .

1.4.6 Структура сверточной нейронной сети, показатели точности которой 90% показана на рисунке 1.9[4].

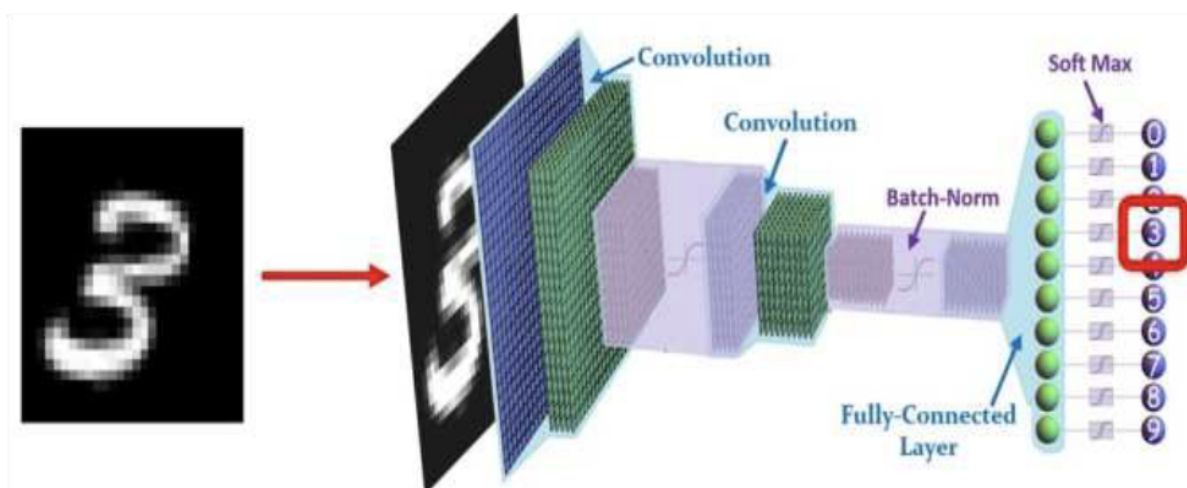


Рисунок 1.9 – Однослойной нейронной сети прямого распространения

Данная конфигурации состоит из:

- 1 Входной слой: $[28 \times 28]$.
- 2 Сверточный слой: 3 маски $[3 \times 3]$.
- 3 Слой пакетной нормализации.
- 4 Слой ReLU.
- 5 Полносвязанный слой.
- 6 Слой Softmax.
- 7 Слой классификации.

Нейронная сеть использует 23520 параметра, которые необходимо хранить в памяти в качестве весовых коэффициентов и смещений.

1.5 Рекуррентные нейронные сети

Рекуррентные нейронные сети (Recurrent Neural Networks) представляют собой особый класс нейронных сетей, предназначенный для обработки последовательных данных, в которых важен порядок элементов во времени. В отличие

от традиционных полносвязных или сверточных нейросетей, которые предполагают, что входы являются независимыми, RNN обладают внутренней памятью, позволяющей учитывать контекст предыдущих состояний при формировании отклика на текущий вход.

Принцип работы рекуррентной архитектуры нейронных сетей показан на рисунке 1.10. Как видно из рисунка, ключевым элементом архитектуры является блок Z^{-1} , реализующий механизм обратной связи, за счёт которого выход текущего шага зависит не только от текущего входа.

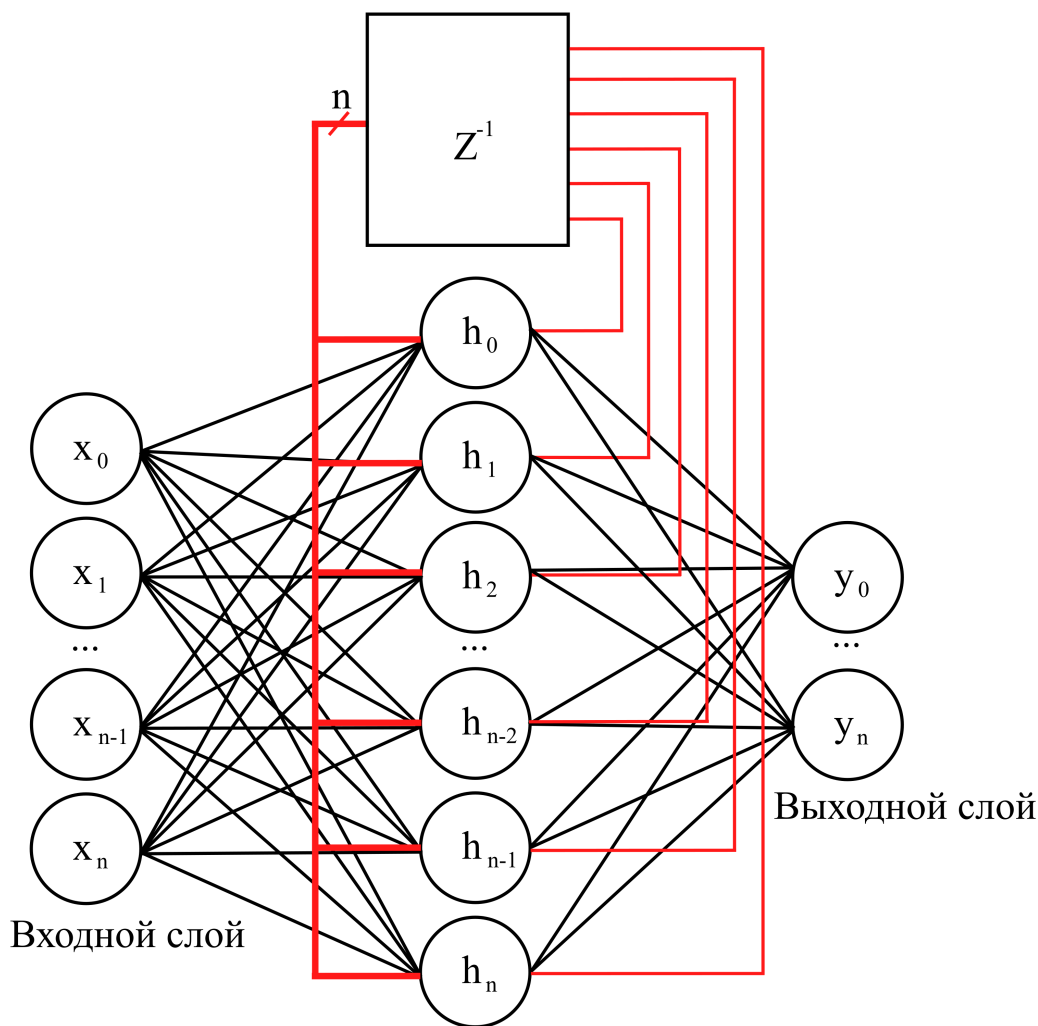


Рисунок 1.10 – Принцип работы рекуррентных нейронных сетей

1.5.1 Основное отличие RNN от других типов нейронных сетей заключается в наличии обратных связей, позволяющих передавать информацию от предыдущих состояний. Для каждого временного шага t вычисляется новое состояние h_t на основе входных данных x_t и предыдущего состояния h_{t-1}

$$h_t = f(W_x x_t + W_h h_{t-1} + b), \quad (1.13)$$

где h_t – скрытое состояние в момент времени t ;

x_t – входной вектор в момент времени t ;

W_x, W_h – обучаемые весовые матрицы;

b – вектор смещения;
 f – функция активации, например $\tanh$ или $ReLU$.
 Выход сети y_t определяется как

$$y_t = g(W_y h_t + c), \quad (1.14)$$

где W_y – матрица весов выходного слоя;
 c – вектор смещения;
 g – функция активации выходного слоя.

1.5.2 Рекуррентные сети могут применяться для распознавания рукописных цифр. Один из распространенных подходов — обработка строк рукописного текста, где каждый символ представлен как последовательность пикселей или векторных признаков. В таких задачах используются LSTM (Long Short-Term Memory) или GRU (Gated Recurrent Unit), способные учитывать пространственно-временные зависимости между элементами изображения.

Пример архитектуры LSTM представлен на рисунке 1.11[5].

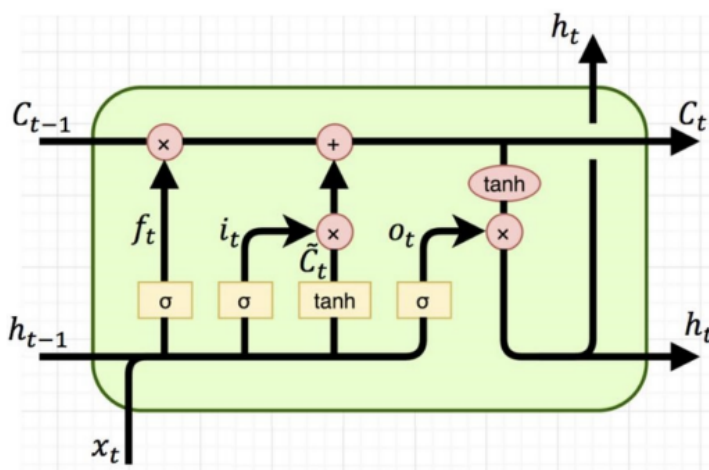


Рисунок 1.11 – Пример архитектуры LSTM

Данная архитектура позволяет решать задачу распознавания рукописного текста с точностью 94,7%.

1.6 Трансформеры нейронные сети

Трансформеры представляют собой архитектуру нейронных сетей, которые используют механизмы самовнимания (self-attention) для обработки входной информации параллельно, что позволяет достигать высокой эффективности и точности.

1.6.1 Архитектура трансформера состоит из энкодера и декодера, каждый из которых содержит несколько слоев. Основные компоненты трансформера:

1 Механизм самовнимания позволяет модели учитывать важность различных частей входных данных.

2 Многоголовочное внимание (multi-head attention) улучшает способность модели учитывать разные аспекты входной информации.

3 Обратная связь ускоряет обучение и предотвращает исчезновение градиента.

4 Механизм нормализации стабилизирует обучение.

5 Полносвязные слои.

Механизм самовнимания вычисляется следующим образом

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (1.15)$$

где Q , K и V — матрицы, полученные линейными преобразованиями данных;
 d_k — размерность ключей.

1.6.2 Хотя трансформеры изначально были разработаны для задач обработки естественного языка, они также успешно применяются для распознавания рукописных цифр. Основной подход заключается в преобразование изображений в последовательности пикселей и обработка их с помощью Vision Transformer (ViT).

Трансформеры обеспечивают высокую точность – 90,30% и используют большое количество параметров – 163729 при обработке изображений, особенно при наличии больших объемов обучающих данных, что делает их перспективным направлением для распознавания рукописных цифр[6].

1.7 Гибридные архитектуры

Гибридные архитектуры нейронных сетей представляют собой комбинацию различных типов нейронных сетей, объединяя их преимущества для улучшения производительности в задачах распознавания изображений, в том числе рукописных цифр. Такие модели могут включать элементы сверточных, рекуррентных, трансформерных и полносвязных сетей.

1.7.1 Одним из наиболее распространенных гибридных подходов является использование сверточных нейронных сетей для извлечения признаков и рекуррентных нейронных сетей для обработки последовательностей.

CNN выполняет обработку изображений, извлекая пространственные признаки, которые затем передаются в RNN.

RNN, такие как LSTM или GRU, анализируют временные зависимости между последовательными фрагментами извлеченных признаков, что особенно полезно при обработке последовательностей символов или цифр.

Математически процесс можно описать следующим образом

$$F = \text{CNN}(X), \quad (1.16)$$

$$h_t = \text{RNN}(F_t, h_{t-1}), \quad (1.17)$$

где X – входное изображение;

F – извлеченные признаки;

h_t – скрытое состояние RNN.

1.7.2 Другой популярный гибридный подход – объединение CNN и транс-

формеров. CNN используются для локального извлечения признаков, а механизм внимания трансформера помогает учитывать глобальный контекст изображения. Формально, этот процесс можно записать следующим образом

$$F = \text{CNN}(X), \quad (1.18)$$

$$Z = \text{Transformer}(F), \quad (1.19)$$

где Z – закодированное представление входного изображения.

1.7.3 Гибридные модели активно применяются для задач распознавания рукописных цифр, поскольку они позволяют достичь высокой точности за счет сочетания мощных методов обработки изображений и анализа последовательностей.

CNN-RNN используются для распознавания рукописных цифр в последовательностях, таких как почтовые индексы.

CNN-Transformer подходят для высокоточного классифицирования цифр в условиях зашумленных или искаженных изображений. Сеть, архитектура которой представлена на рисунке 1.12, позволяет добиться точности 99,89% [7].

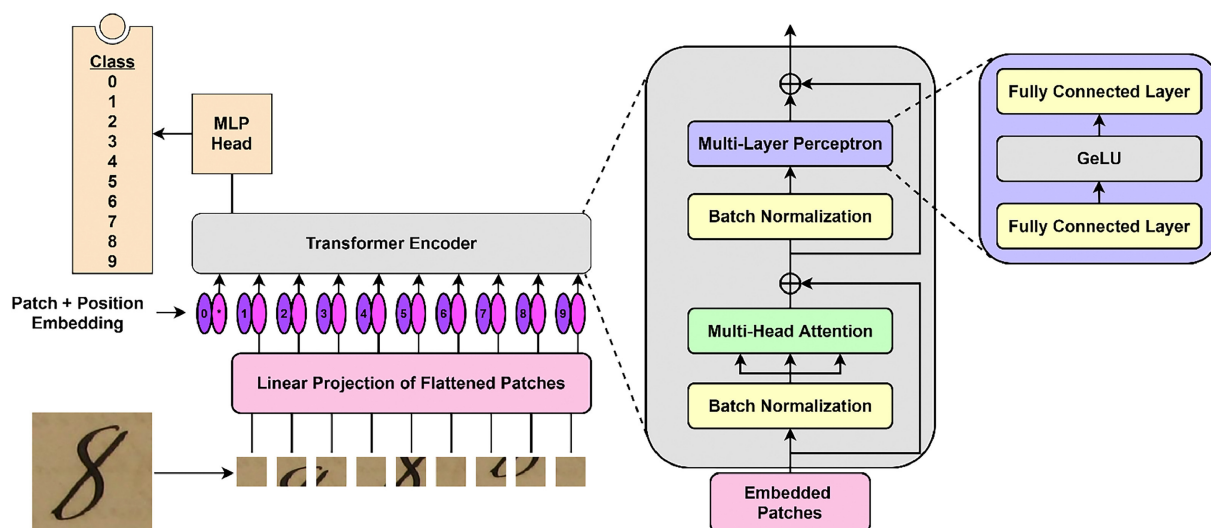


Рисунок 1.12 – Архитектура CNN-Transformer

Благодаря комбинированию разных методов гибридные архитектуры обеспечивают улучшенные результаты в задачах компьютерного зрения.

Входное изображение разбивается на небольшие фрагменты (patches). Это аналогично разбиению изображения на фиксированные блоки, которые затем используются как отдельные элементы входной последовательности.

Каждый патч преобразуется в вектор фиксированной размерности с помощью линейного слоя. Также добавляется позиционное кодирование (Patch + Position Embedding), чтобы сохранить информацию о порядке патчей.

Вектора патчей подаются в трансформерный энкодер.

Затем он поступает в MLP Head, который выводит вероятности принадлежности к каждому из классов.

2 АНАЛИЗ ТЕХНИЧЕСКОГО ЗАДАНИЯ

2.1 Постановка задачи

В данной работе основное внимание уделяется задаче распознавания изображений рукописных цифр с использованием нейронных сетей, оптимизированных для аппаратной реализации на ПЛИС (FPGA).

Целью дипломной работы является проектирование и реализация компактной нейронной сети прямого распространения, способной обрабатывать изображения размером 28×28 пикселей, представляющих рукописные цифры из стандартного набора данных MNIST. Основной задачей является не только достижение высокой точности классификации, но и обеспечение совместимости с аппаратной реализацией, особенно в условиях ограниченных вычислительных и энергетических ресурсов.

В качестве основы для нейросетевой архитектуры используется метод Learned 2D Separable Transform, предложенный в работе [8]. Преимущество данного подхода заключается в использовании разделяемых по строкам и столбцам операций, что позволяет существенно снизить количество параметров модели по сравнению с традиционными полносвязными слоями. LST-архитектура даёт возможность эффективно использовать параллелизм, характерный для FPGA, и обеспечивает высокую скорость обработки без значительных потерь в точности.

Для аппаратной реализации модели выбрана микросхема XC7Z010CLG400-2, входящая в состав популярной отладочной платы Zybo Z7. Это решение основано на архитектуре Xilinx Zynq-7000, сочетающей возможности ПЛИС и встроенного процессора ARM, что делает его удобным для интеграции программно-аппаратных систем.

Вид сверху отладочной платы Zybo Z7 на базе Xilinx Zynq-7000, с указанием основных интерфейсов и компонентов: USB, HDMI, DDR, питание, PMOD-разъемы и т.д. представлен на рисунке 2.1 [9].

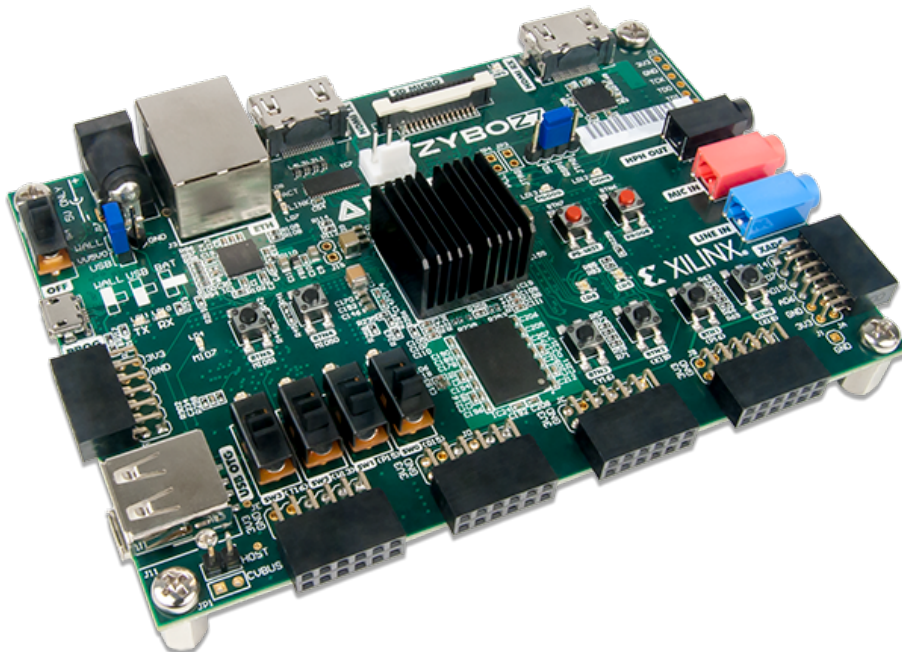


Рисунок 2.1 – ПЛИС Zybo Z7

Внутренняя структура Zybo Z7 представлена на рисунке 2.2.

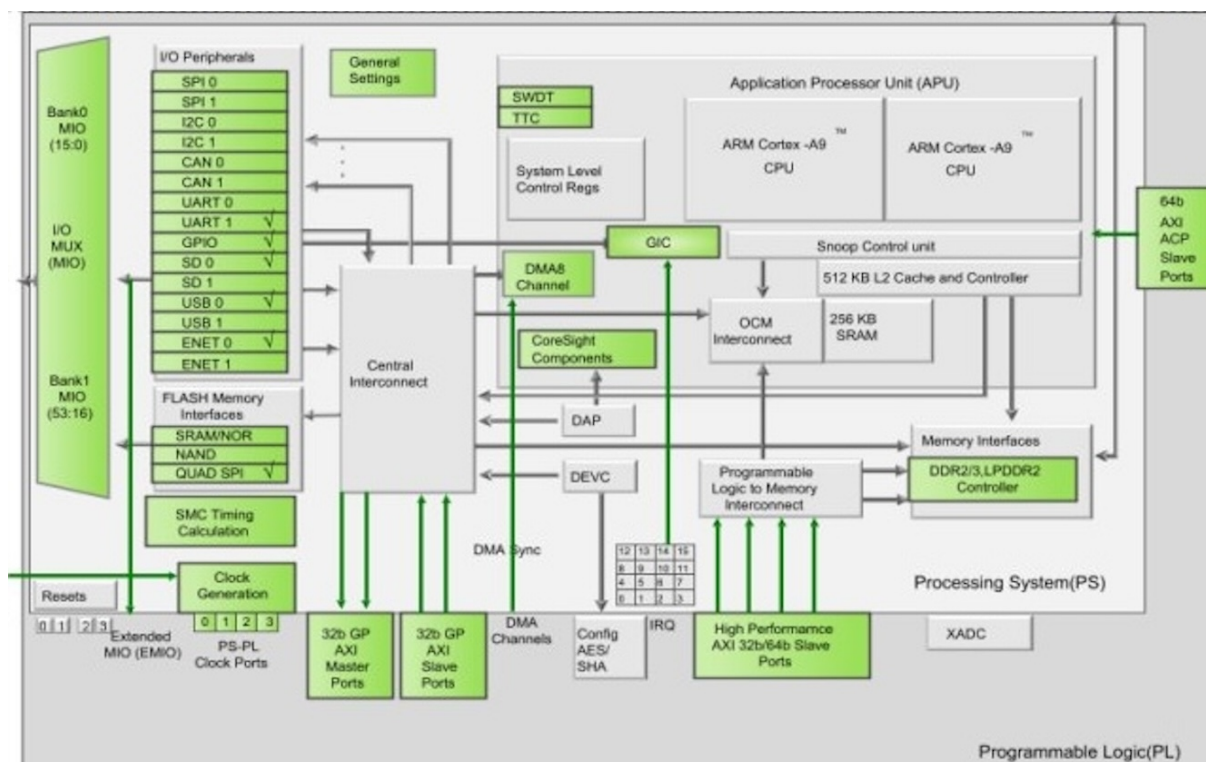


Рисунок 2.2 – Структура ПЛИС Zybo Z7

На схеме продемонстрированы основные блоки SoC Zynq-7000, включая: процессорную систему (PS), программируемую логику (PL), AXI-интерфейсы, контроллеры памяти и периферийные модули. В рамках настоящего проекта будет использоваться взаимодействие между PL и PS через AXI-шину

Для взаимодействия с моделью и удобства проведения тестирования будет использоваться программная платформа PYNQ, позволяющая запускать и отлаживать алгоритмы с помощью Jupyter Notebook и Python API. Таким образом, будет реализован полный цикл: от проектирования и обучения нейросети – до её верификации и аппаратной имплементации.

Постановка задачи и предъявленные требования представлены в графическом материале ГУИР 431289.001 ПЛ.

2.2 Анализ требований к нейронной сети

Выбор архитектуры нейронной сети требует учета как точности модели, так и её аппаратной эффективности. В условиях ограниченных ресурсов FPGA особое внимание уделяется упрощению структуры и снижению объема вычислений. Основные требования к нейросети включают:

1 Модель должна обеспечивать точность не ниже 97% на тестовой выборке MNIST, что соответствует уровню, достигаемому современными методами машинного обучения. Для достижения данной цели необходимо правильно подобрать архитектуру, функцию потерь, функцию активации и метод оптимизации.

2 Для реализации на аппаратном уровне необходимо минимизировать количество параметров модели, сохраняя при этом высокую точность. Это осо-

бенно важно для устройств с ограниченными ресурсами, таких как встраиваемые системы.

3 Память в ПЛИС системах имеет жёсткие ограничения. Модель должна быть оптимизирована таким образом, чтобы не только сама структура занимала как можно меньше памяти, но и чтобы объем оперативных данных, хранимых во время выполнения активации, был минимален.

4 Выходной слой сети должен использовать softmax для получения вероятностного распределения по классам. В скрытых слоях могут использоваться функции активации, оптимальные для аппаратной реализации, например tanh.

2.3 Анализ требований к аппаратной реализации

Эффективность работы нейронной сети на аппаратном уровне зависит от грамотного распределения вычислительных ресурсов, схемотехнической оптимизации и способности к параллелизму.

Основные аппаратные требования и ограничения при реализации модели на FPGA включают:

1 В архитектуре FPGA присутствуют специализированные блоки: логические ячейки (LUT), регистры, блоки умножения (DSP), встроенная память (BRAM). Для успешной реализации модели необходимо уйти от операций с плавающей запятой, заменяя их на фиксированную арифметику, а также эффективно задействовать ресурсы DSP и BRAM. В FPGA важно использовать внутренний параллелизм.

2 Так как встраиваемые системы часто работают от автономных источников питания, архитектура должна потреблять как можно меньше энергии. Этого можно достичь путём сокращения количества операций, понижения разрядности чисел, а также выбора энергоэффективных функций активации и структур слоёв.

3 Архитектура должна быть масштабируемой. Кроме того, должна быть возможность модификации модели для адаптации под новые задачи.

4 Реализация должна быть совместима с использованием Python-интерфейсов и поддерживать управление через PYNQ API. Это необходимо для быстрой верификации модели, автоматизации тестирования и сбора метрик производительности.

Основной задачей является выбор оптимальной архитектуры сети и методов вычислений, которые обеспечат баланс между производительностью и потреблением ресурсов.

2.4 Выбор и обоснование метода решения задачи

Выбор архитектуры нейросети — один из ключевых аспектов в реализации высокоэффективного классификатора. Для решения задачи распознавания рукописных цифр рассматриваются различные подходы, каждый из которых имеет свои преимущества и ограничения.

Полносвязные нейронные сети — обладают простой структурой и легко реализуются, однако требуют большого количества параметров. Это делает их менее подходящими для аппаратной реализации, так как объем необходимых вычислений и памяти растет пропорционально числу нейронов в слоях.

Сверточные нейронные сети — обеспечивают высокую точность за счет эффективного извлечения признаков из изображений. Они используют свертки, позволяющие минимизировать количество параметров по сравнению с полно-

связными сетями.

Рекуррентные нейронные сети – могут использоваться для последовательной обработки данных, однако их применение к изображениям ограничено. Такие сети лучше подходят для обработки временных рядов и речи, но могут быть адаптированы к изображениям.

Трансформеры – показывают высокую производительность в задачах обработки естественного языка и компьютерного зрения, но требуют значительных вычислительных ресурсов. Несмотря на их преимущества, высокая сложность и потребление памяти делают их менее подходящими для работы на FPGA.

Гибридные архитектуры – могут комбинировать различные методы, такие как CNN и рекуррентные слои, что позволяет достичь оптимального баланса между точностью и вычислительными затратами.

LST2D архитектуры — выступают в качестве альтернативы архитектурам полносвязных слоев и позволяют использовать минимальные вычислительные ресурсы.

На основании требований к аппаратной реализации и соотношения сложности структуры доступных архитектур, предлагается использовать обучаемое двумерное разделимое преобразование, которое можно рассматривать как новый тип вычислительного слоя для построения различных архитектур нейронных сетей[8].

Данная структура обеспечивает высокую точность классификации изображений благодаря эффективному представлению пространственных зависимостей и позволяет сократить вычислительные затраты за счет оптимизированной структуры слоев.

Двумерный входной массив сначала преобразуется по строкам с помощью первого линейного слоя, а затем – по столбцам с использованием второго слоя с отдельными весами. Это позволяет:

- сократить число параметров до $O(n^2)$ вместо $O(2n)$;
- повысить скорость выполнения;
- минимизировать потребление памяти.

Для реализации поставленных задач необходимо разработать структурную и функциональную схемы устройства. Описать алгоритм работы с учетом всех вышеперечисленных функций. По алгоритму описать модель на HDL языках.

3 РАЗРАБОТКА НЕЙРОННОЙ СЕТИ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

3.1 Функциональная спецификация системы

Разработка системы распознавания рукописных цифр, основанной на нейронной сети прямого распространения, требует составления функциональной спецификации. Такая спецификация определяет основные требования к системе, описывает её функциональные компоненты, а также интерфейсы взаимодействия с пользователем и внешним окружением.

Проектируемая система будет реализована в виде IP-блока, представляющего собой нейронную сеть прямого распространения. Предварительное обучение сети предполагается проводить вне ПЛИС, с использованием обучающей выборки изображений рукописных цифр. После обучения модель будет внедряться в IP-блок и использоваться для классификации входных изображений.

В рамках анализа пользовательских требований система должна давать ответы на следующие ключевые вопросы:

- 1 Какие средства необходимо предусмотреть для ввода данных?
- 2 Какие средства необходимо предусмотреть для вывода результатов?
- 3 Как будет осуществляться хранение весовых коэффициентов?
- 4 Какие средства необходимы для работы с пользователем?

Составим функциональную спецификацию для разрабатываемой системы:

1 Ввод данных о рукописных цифрах будет осуществляться посредством последовательной передачи значений пикселей входного изображения. Данные будут поступать в IP-блок по интерфейсу AXI4-Lite, обеспечивая взаимодействие с процессорной системой, размещённой на базе ПЛИС.

2 Результаты распознавания, генерируемые моделью, будут передаваться обратно в процессорную систему по тому же интерфейсу AXI4-Lite. Полученные данные могут быть далее обработаны, в частности – использованы для построения матрицы ошибок или других аналитических задач.

3 Весовые коэффициенты нейронной сети будут храниться во внутренней блочной памяти ПЛИС. Хранение будет организовано с поддержкой трёх отдельных весовых групп: весов первого слоя LST, весов второго слоя LST, а также весов выходного полносвязного слоя.

4 Система будет реализована в виде IP-блока, подключаемого к процессорной системе через интерфейс AXI4-Lite. Такое решение обеспечивает стандартизированное взаимодействие и позволяет гибко интегрировать блок в состав аппаратной платформы.

3.2 Разбиение системы на модули

Система распознавания рукописных цифр, реализованная на базе нейронной сети прямого распространения, может быть логически разделена на ряд функциональных модулей. Каждый из них выполняет строго определённую задачу в процессе преобразования входного изображения в соответствующую цифровую метку. Ниже приводится описание всех ключевых модулей системы:

1 Модуль памяти изображения предназначен для хранения входного изображения на всех этапах обработки. Он обеспечивает доступ к значениям пикселей изображения при необходимости многократного считывания.

2 Модуль обработки пикселя отвечает за формирование пары «значение

пикселя – соответствующий весовой коэффициент», передаваемой в модуль МАС. Этот модуль извлекает данные из памяти изображения и памяти весов, синхронизируя подачу входных значений на следующий вычислительный этап.

3 Модуль МАС является основным вычислительным блоком системы, реализующий базовую операцию нейронной сети – умножение входного значения на соответствующий вес с последующим накоплением результата. Модуль используется для формирования выходов линейных слоёв.

4 Модули памяти весов обеспечивают хранение предварительно обученных весовых коэффициентов нейронной сети. Для корректной работы системы предусмотрено разделение памяти на несколько независимых блоков, соответствующих разным слоям сети: первому слою LST, второму слою LST и выходному полносвязному слою.

5 Модуль функции активации выполняет нелинейное преобразование выходов линейных слоёв LST с помощью функции гиперболического тангенса. Это преобразование необходимо для придания нейронной сети способности моделировать сложные зависимости.

6 Модуль функции активации softmax осуществляет преобразование выходного вектора нейронной сети в вероятностное распределение по классам. Этот модуль необходим на финальном этапе обработки, чтобы получить интерпретируемые результаты классификации.

7 Модуль управления отвечает за координацию работы всех функциональных блоков системы. Он генерирует управляющие сигналы, определяет порядок операций и обеспечивает корректную последовательность выполнения всех этапов обработки изображения.

3.3 Принцип работы LST преобразования

Двумерное разделимое преобразование используется в обработке изображений для уменьшения вычислительной сложности. Основная идея заключается в том, что вместо хранения полного двумерного ядра свёртки размером $n \times n$ используется его представление в виде произведения двух одномерных векторов. Это позволяет сократить число параметров с n^2 до $2n$, что снижает затраты на вычисления и память.

$$W = V \times h^T, \quad (3.1)$$

где $W \in \mathbb{R}^{n \times n}$;
 $v, h^T \in \mathbb{R}^{n \times 1}$.

LST основан на идее совместного использования весов одного полносвязного слоя для обработки всех строк изображения. После этого второй общий полносвязный слой используется для обработки всех столбцов представления изображения, полученных из первого слоя. Использование слоев LST в архитектуре нейронной сети значительно сокращает количество параметров модели по сравнению с моделями, которые используют стекированные полносвязные слои.

LST можно использовать для построения архитектур глубоких нейронных сетей (DNN), заменяя традиционную нейронную сеть прямого распространения.

Предложенное обученное двумерное разделимое преобразование обрабатывает изображение построчно, а не преобразует его в вектор, как это обычно делается в полносвязных сетях.

С математической точки зрения LST2D принимает двумерное изображение

X размером $d_{in} \times d_{in}$ в качестве входных данных и создает двумерное выходное изображение Y размером $d_{out} \times d_{out}$

$$Y = LST_{d_{in} \times d_{out}}(X) = \tan(W_2 \tan(W_1 X^T)). \quad (3.2)$$

где W_1, W_2 — матрицы весов слоев FC1 и FC2 соответственно.

Принцип работы LST2D представлен на рисунке 3.1[8].

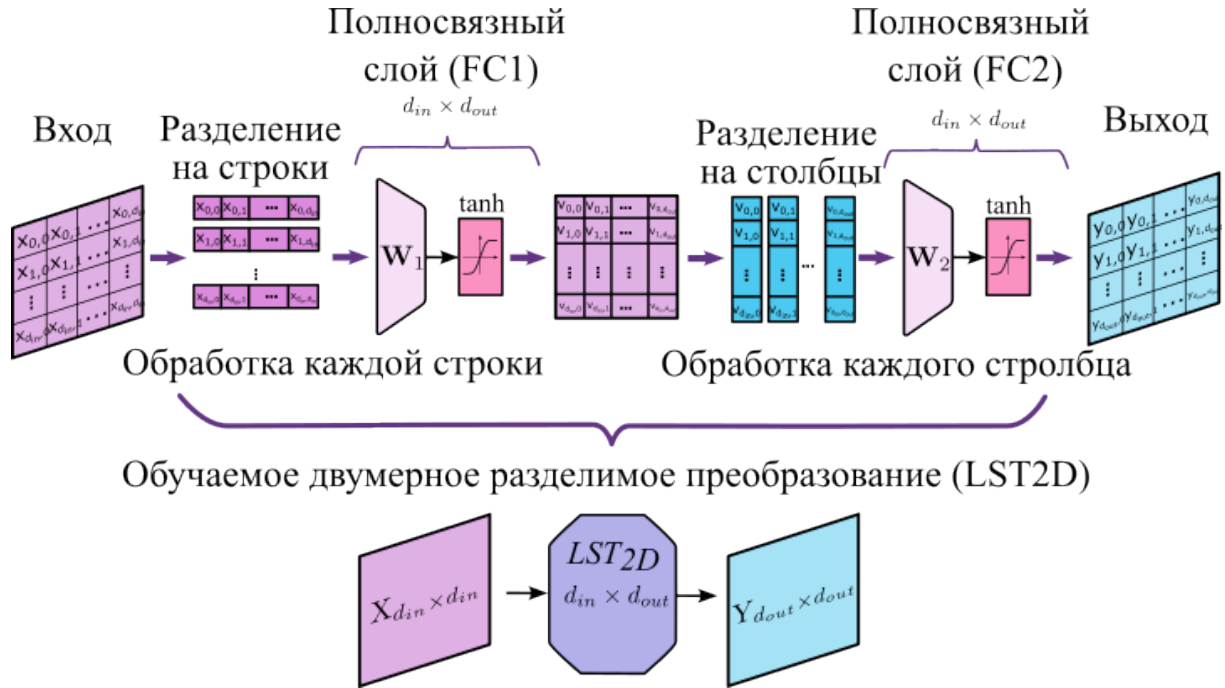


Рисунок 3.1 – Принцип работы LST2D

Количество обучаемых параметров зависит от размерности изображений

$$N = 2 \cdot (d_{in} + 1) \cdot d_{out} = 2 \cdot 29 \cdot 28 = 1624. \quad (3.3)$$

Здесь d_{in} и d_{out} — гиперпараметры преобразования.

Простейшая архитектура нейронной сети на основе LST2D для распознавания цифр MNIST требует одного блока LST2D, за которым следует полносвязный слой с функцией активации softmax, что показано на рисунке 3.2[8].

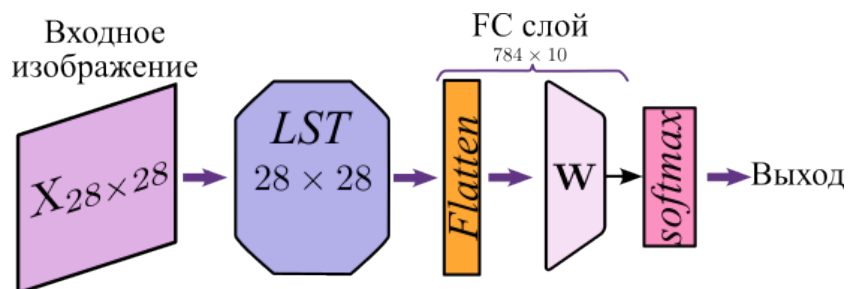


Рисунок 3.2 – Принцип работы LST-1

Алгоритм работы LST слоя показывает последовательность действий по преобразованию входного изображения размером 28×28 в выходное, такого же размера представлен на рисунке 3.3.

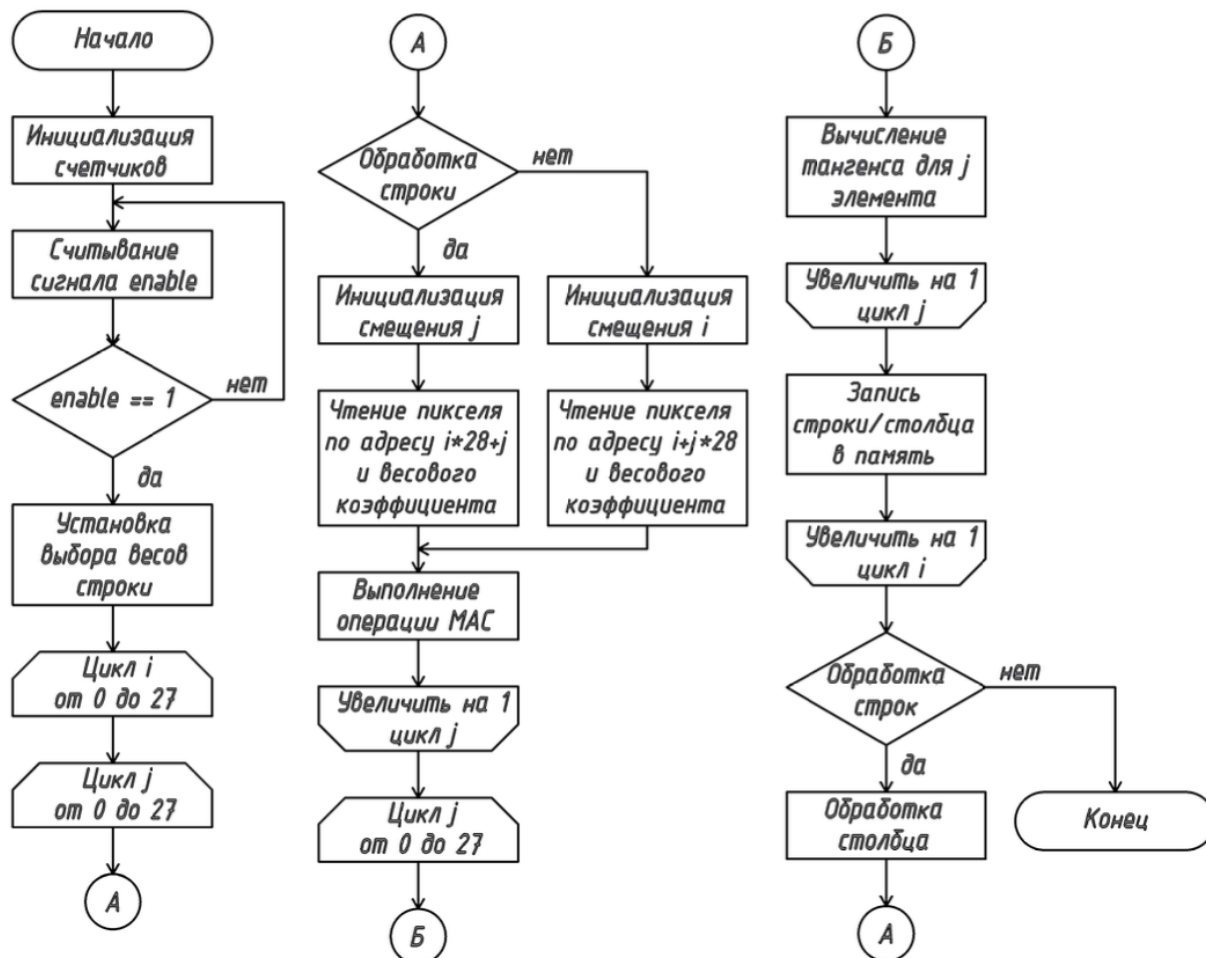


Рисунок 3.3 – Алгоритм работы LST2D слоя

Данную архитектуру можно масштабировать. Пример двухблочной архитектуры LST2D приведен на рисунке 3.4[8].

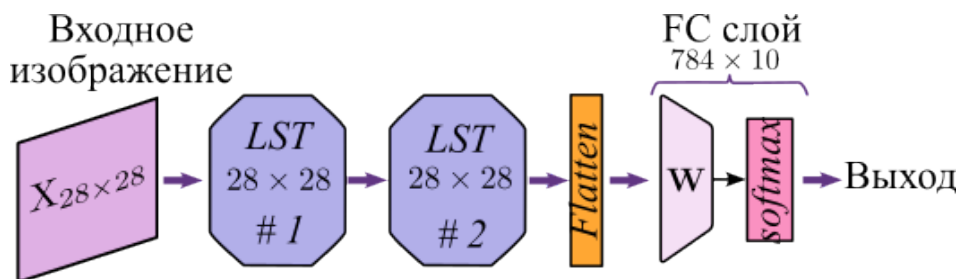


Рисунок 3.4 – Принцип работы LST-2

Пример обработки изображения нейронной сетью архитектуры LST-1 представлен на рисунке 3.5[8].

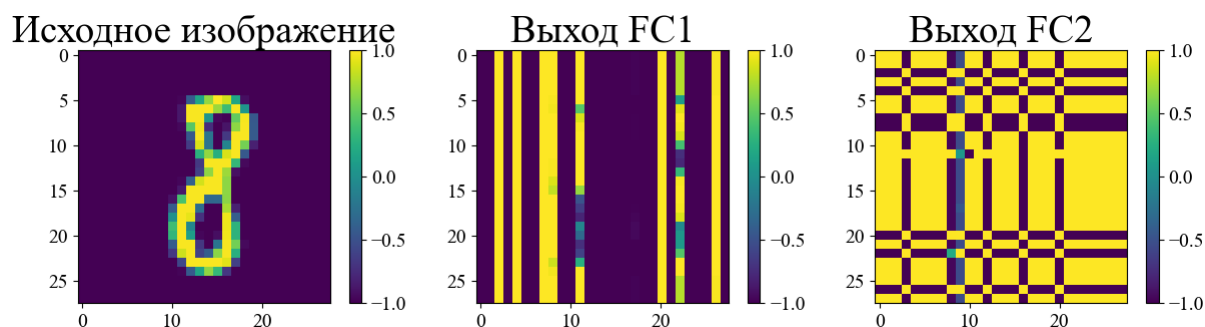


Рисунок 3.5 – Архитектура CNN-Transformer

3.4 Обучение нейронной сети

Обучение нейронной сети LST-1 на основе 60000 тренировочных изображений из базы данных MNIST. Для обучения используется фреймворк PyTorch.

В качестве исходных данных используется набор MNIST, содержащий 70 тысяч полутоновых изображений размера 28×28 пикселей, представляющих рукописные цифры от 0 до 9 [10]. Данный набор разделен на две части:

- Тренировочная выборка – 60 тысяч изображений.
- Тестовая выборка – 10 тысяч изображений.

Пример данных из базы MNIST представлен на рисунке 3.6.

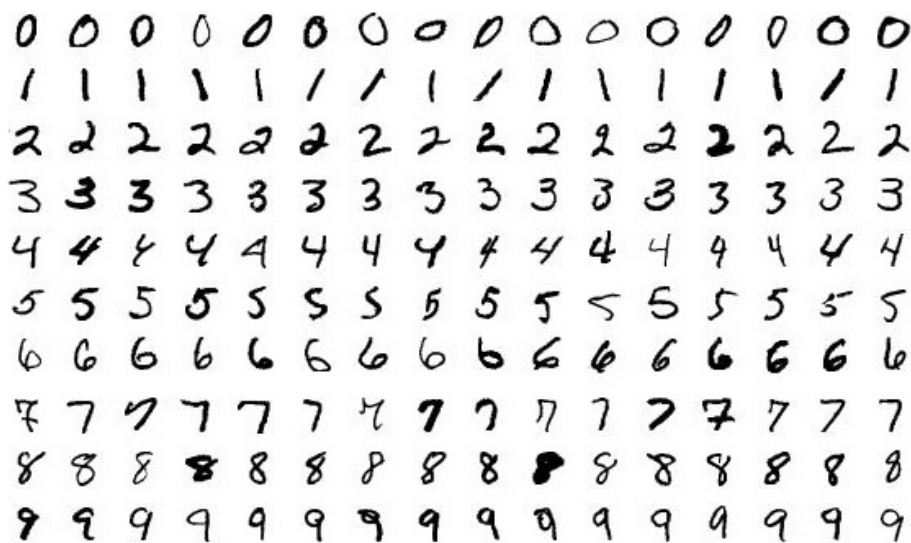


Рисунок 3.6 – Пример данных из MNIST

Из тренировочного набора выделяется 1000 изображений для валидации, оставшиеся 59000 используются для непосредственного обучения модели.

Перед подачей в нейросеть изображения проходят предварительную обработку, включающую нормализацию значений пикселей. Для упрощения процесса обучения выполняется нормализация данных таким образом, чтобы значение каждого пикселя было в диапазоне от -1 до 1 , а стандартное отклонение (σ) составляло 0.5 . Это позволяет сделать градиентный спуск более устойчивым и ускорить процесс сходимости модели. Пример такой обработки представлен на рисунке 3.7.

Нормализованные изображения

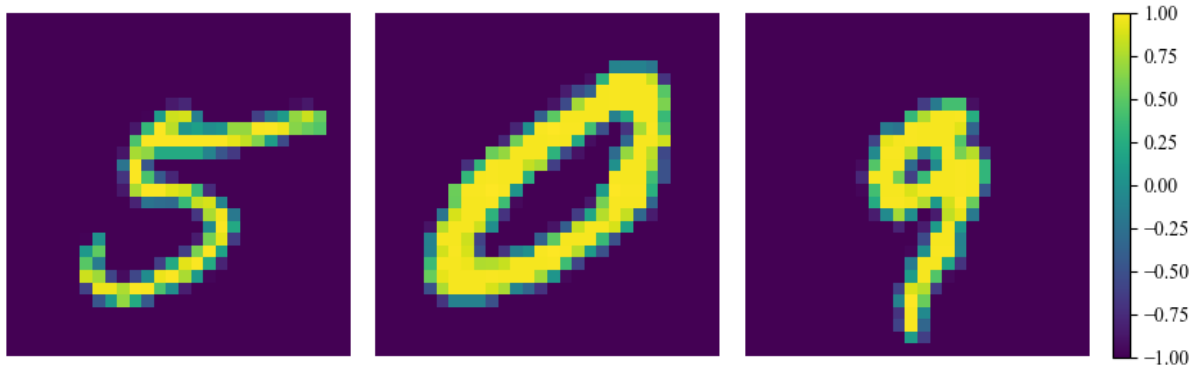


Рисунок 3.7 – Пример обработанных данных из MNIST

Нормализация изображений осуществляется в соответствии с описанными требованиями осуществляется предварительно, что показано в фрагменте листинга ниже:

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])
```

После нормализации изображения подаются на вход нейронной сети L2DST. Архитектура модели L2DST представляет собой последовательное соединение двух линейных слоёв с функцией активации гиперболический тангенс $\tanh$ и одного полносвязного слоя с функцией активации softmax , применяемой для выполнения классификации. Основной особенностью реализации является использование слоёв `LazyLinear`, позволяющих автоматически определить размер входного пространства на этапе первого запуска модели.

Ниже приведён фрагмент программы, описывающий структуру модели:

```
class L2DST(nn.Module):
    def __init__(self, ffn_num_hiddens, ffn_num_outputs):
        super().__init__()
        self.dense1 = nn.LazyLinear(ffn_num_hiddens)
        self.dense2 = nn.LazyLinear(ffn_num_outputs)

    def get_embeddings(self, X):
        out = torch.nn.functional.tanh(self.dense1(X))
        e1 = out
        out = torch.transpose(e1, -1, -2)
        out = torch.nn.functional.tanh(self.dense2(out))
        e2 = out
        out = torch.transpose(e2, -2, -1)

        return e1, e2
```

Обучение модели осуществляется с использованием стохастического оптимизатора Adam, известного своей эффективностью на широком спектре задач глубокого обучения. Начальное значение скорости обучения задано равным 1.5×10^{-3} , а также применяется регуляризация через параметр `weight_decay`, равный нулю.

Для обеспечения устойчивого процесса обучения применяется поэтапное снижение скорости обучения с помощью механизма StepLR. Каждые 10 эпох происходит уменьшение текущей скорости обучения на 3%, что позволяет модели более точно подстраиваться к минимальному значению функции потерь на поздних этапах обучения.

```
criterion = nn.NLLLoss()
optimizer = optim.Adam(model.parameters(), lr=15e-4, weight_decay=0e-5)
scheduler = lr_scheduler.StepLR(optimizer, step_size=10, gamma=0.97,
                                verbose=False)
```

На рисунке 3.8 показан график функции потерь за 370 эпох.

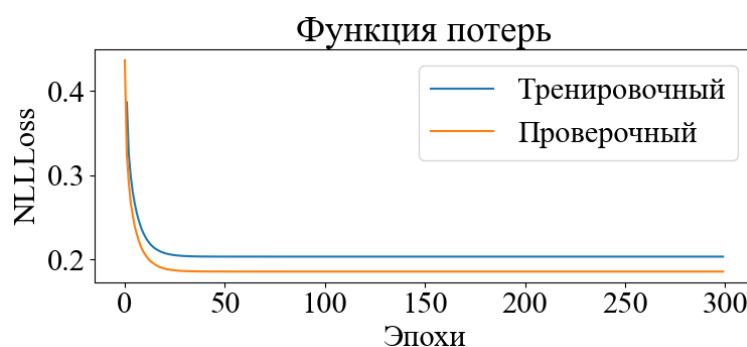


Рисунок 3.8 – Графики функции потерь на обучающем и тестовом наборе

После завершения каждой эпохи вычисляются средние значения ошибки на тренировочном и валидационном наборах данных.

Python описание процесса обучения нейронной сети прямого распространения приведено в приложении Б.

4 РАЗРАБОТКА СТРУКТУРЫ IP-БЛОКА НЕЙРОННОЙ СЕТИ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

4.1 Описание структурной схемы

Принцип работы данной нейронной сети основан на комбинации обучаемого двумерного разделимого преобразования и полносвязной нейронной сети. Такая архитектура позволяет эффективно обрабатывать изображения, снижая вычислительные затраты при аппаратной реализации.

Для окончательной классификации используется полносвязный слой с функцией активации softmax. Данный слой преобразует выходные значения в вероятностное распределение по классам, позволяя определить, к какой цифре относится входное изображение.

Вычислительное ядро разработанного устройства состоит из 28 МАС (Multiply-ACcumulate) ядер, распределенных по специализированным блокам. Эти блоки организованы таким образом, чтобы обеспечить эффективное выполнение операций умножения входного вектора изображения на матрицу весов слоя нейронной сети.

Десять МАС-ядер из общего массива используются на финальном тапе вычислений и непосредственно участвуют в формировании входных данных для функции активации softmax. Эти ядра получают входные данные из трех независимых блоков памяти, что позволяет оптимизировать процесс выборки весов и повысить скорость вычислений. Остальные 18 МАС-ядер распределены по блокам, где каждая вычислительная единица получает данные из двух запоминающих устройств.

IP-блок устройства предназначен для интеграции в систему на кристалле (СнК) и взаимодействует с процессорной системой через AXI-uP интерфейсы. СнК – это интегральная схема, которая объединяет в одном чипе все основные компоненты вычислительной системы: процессор, ОЗУ (RAM) и ПЗУ (ROM), графический процессор (GPU), контроллеры периферии. Общая структура представлена на рисунке 4.1.

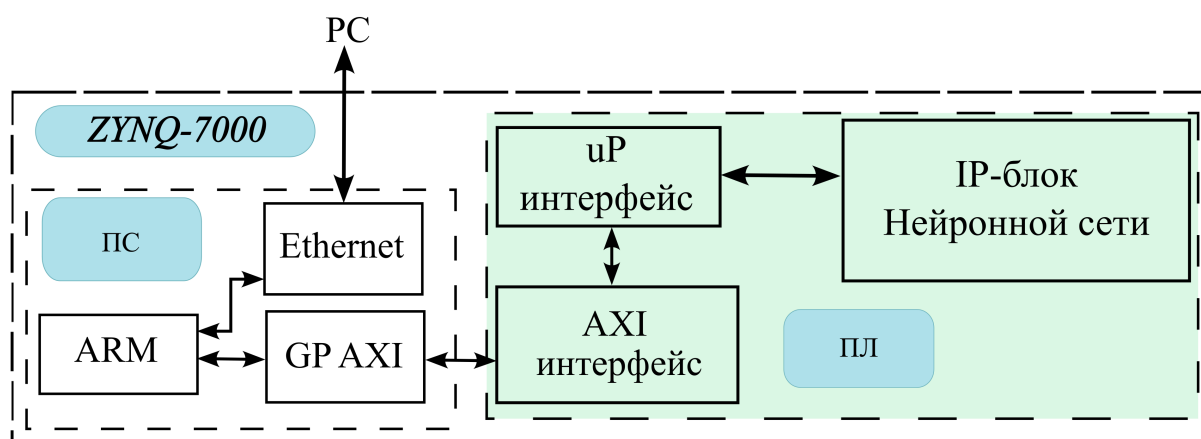


Рисунок 4.1 – Пример использования IP-блока нейронной сети

Последовательность обработки данных организуется следующим образом:

1 Исходное изображение размером 28×28 пикселей поступает на вход нейронной сети через переходник интерфейсов AXI-uP. Данные интерфейсы

используется для взаимодействия с процессорной системой и обеспечивают передачу изображения в IP-блок. Пиксели изображения принимаются последовательно и записываются во внутреннюю память IP-блока, предназначенную для хранения изображения на протяжении всего цикла обработки.

2 Каждый отдельный пиксель изображения поочередно передаётся на вход массива из 28 умножающих-накопительных ядер (МАС), которые одновременно выполняют операции умножения входного значения пикселя на соответствующий весовой коэффициент. Весовые коэффициенты извлекаются из специализированных блоков памяти весов, которые хранят параметры, полученные в результате предварительного обучения модели.

3 За согласованность и последовательность операций отвечает управляющий модуль. Он обеспечивает адресацию ячеек памяти изображения и весов, организует подачу данных в вычислительные блоки и отслеживает фазы обработки. Управляющее устройство также координирует этапы переключения между слоями, а также формирует сигналы запуска и завершения вычислений.

4 После завершения расчетов в первом скрытом слое (LST-1), результат – новое представление входного изображения — передаётся во второй уровень вычислений. Здесь используются 10 МАС-ядер, каждое из которых обрабатывает соответствующий вектор признаков, поступающий от первого слоя. В результате получается выходной вектор размерности 10, где каждый элемент отражает степень принадлежности изображения к соответствующему классу (от 0 до 9).

5 Полученный выходной вектор подаётся в блок активации softmax. Этот модуль преобразует значения в вероятностную форму, нормируя их так, чтобы сумма элементов вектора была равна единице. На выходе формируется распределение вероятностей принадлежности изображения к каждому из 10 классов.

6 Индекс элемента вектора softmax, имеющий наибольшее значение (то есть наиболее вероятный класс), определяется и передаётся через интерфейс AXI4-lite обратно в процессорную систему. Этот интерфейс реализует простую и эффективную регистровую модель доступа, обеспечивая передачу управляющих команд и данных между IP-блоком нейронной сети и процессором. Интерфейс uP (user processor interface) позволяет интегрировать модель в более широкую систему на кристалле, облегчая её взаимодействие с внешним программным обеспечением и обеспечивая гибкость конфигурации [12].

Основные сигналы uP-интерфейса:

1 ADDR – адрес регистра, к которому выполняется обращение.

2 DATA_IN – входные данные, записываемые в регистр.

3 DATA_OUT – выходные данные, считываемые из регистра.

4 RD – сигнал чтения из регистра.

5 WR – сигнал записи в регистр.

6 CLK – синхросигнал интерфейса.

7 RESET – глобальный сброс интерфейса.

8 READY – флаг готовности к передаче данных.

Временная диаграмма работы uP-интерфейса представлена на рисунке 4.2.

AXI4-Lite – это упрощённый вариант AXI4-интерфейса, предназначенный для управления и обмена небольшими порциями данных. В отличие от AXI4, он поддерживает только однослотовые транзакции без разделения на несколько пакетов [11].

Основные сигналы AXI4-Lite:

1 AWADDR – адрес записи.

- 2 AWVALID – флаг валидности адреса записи.
- 3 WDATA – записываемые данные.
- 4 WVALID – сигнал валидности данных.
- 5 BREADY – подтверждение завершения записи.
- 6 ARADDR – адрес чтения.
- 7 ARVALID – флаг валидности адреса чтения.
- 8 RREADY – готовность принять данные.
- 9 RDATA – считанные данные.
- 10 RVALID – сигнал валидности данных.
- 11 ACLK – системный тактовый сигнал.
- 12 ARESETN – асинхронный сброс, активный по нулю.

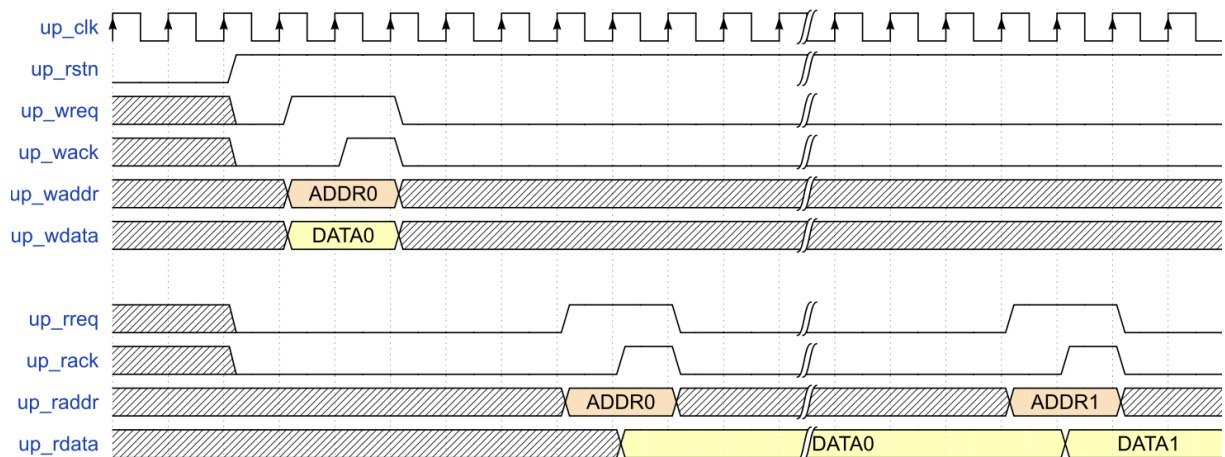


Рисунок 4.2 – Временная диаграмма транзакций чтения и записи интерфейса uP

Временная диаграмма работы AXI4-lite-интерфейса представлена на рисунке 4.3.

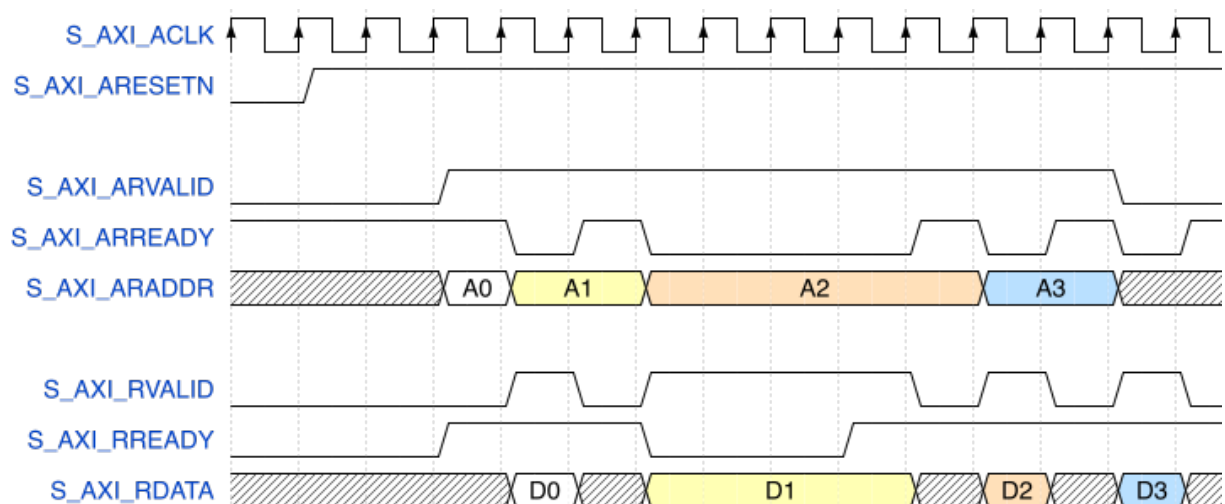


Рисунок 4.3 – Временная диаграмма транзакций чтения и записи интерфейса AXI4-lite

Оба интерфейса широко применяются в проектировании цифровых систем. uP-интерфейс удобен для работы с простыми IP-блоками, тогда как AXI4-Lite предоставляет стандартизированный метод взаимодействия с процессорными системами, такими как ARM-ядра на FPGA.

Электрическая структурная схема IP-блока нейронной сети прямого пространства приведена в графическом материале ГУИР 431289.002 Э1.

4.2 Программная реализация эталонной модели нейронной сети на языке Python

Программная реализация нейронной сети выполняется с использованием языка Python. В качестве эталонной модели реализуется нейронная сеть на основе библиотеки fixpoint, позволяющая провести сравнение точности вычислений при использовании чисел с фиксированной запятой и чисел с плавающей запятой.

Эталонная модель представляет собой нейросеть с двумя скрытыми слоями и активационной функцией tanh.

Для аппаратной реализации на FPGA и эталонной модели числа представляются в формате с фиксированной запятой (Fixed-Point). В отличие от чисел с плавающей запятой (Floating-Point), данный формат обеспечивает более эффективное использование аппаратных ресурсов, таких как блоки DSP и BRAM.

В данной работе используется фиксированное представление с разрядностью Q6.7, где:

- 6 бит отведены под целую часть числа.
- 7 бит используются для дробной части.

Таким образом, числа в этом формате могут представлять значения в диапазоне $[-32, 31.996]$ с шагом $2^{-7} = 0.0078125$.

При переводе значений из плавающей запятой в фиксированную используется метод округления к ближайшему минимальному числу (округление вниз, или Round Down). Этот метод позволяет минимизировать ошибки округления и избежать систематического смещения, которое может возникать при округлении к ближайшему числу.

Такой метод округления позволяет повысить точность вычислений и уменьшить накопление ошибок при последовательных операциях, что критично для работы нейросети на FPGA.

Входные изображения размером 28×28 проходят последовательную обработку:

- Первый слой выполняет линейное преобразование входных данных с весами и смещением, после чего применяется функция активации тангенс.

- Второй слой аналогично выполняет линейное преобразование, используя параметры весов и смещений, после чего применяется функция активации тангенс.

- Выходной слой выполняет линейное преобразование входных данных с весами и смещением, а затем классифицирует изображение, вычисляя вероятности классов с помощью softmax.

Для проверки корректности работы алгоритма в числах с фиксированной запятой разрабатывается программная эмуляция аппаратного вычислителя. Эмуляция реализуется с использованием классов:

- 1 MAC – блок умножения и аккумуляции с фиксированной точностью.

2 MEM – память для хранения весов, смещений и промежуточных значений.

3 TANH – реализация функции гиперболического тангенса в фиксированной точности.

Обработка входных данных в модели с фиксированной запятой включает следующие этапы:

1 Загрузка изображения из базы данных MNIST в память.

2 Преобразование входных данных согласно формату Q6.7.

3 Выполнение первого преобразования и применение функции тангенс.

4 Обработка данных во втором слое с аналогичной активацией.

5 Вычисление выходных значений нейросети и применение softmax.

Сравнение выходных значений между моделью с фиксированной точностью и моделью с плавающей точкой представлено на рисунке 4.4.

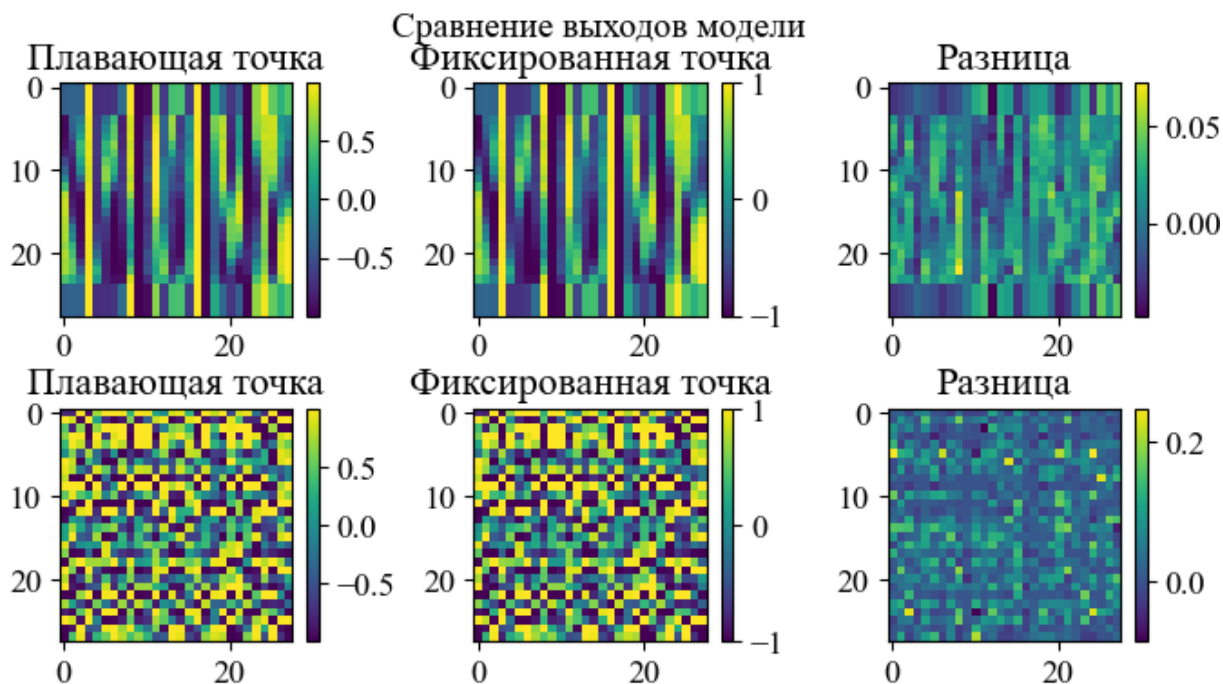


Рисунок 4.4 – Сравнение результатов вычислений

Графический анализ разности значений в обработке одного и того же изображения позволяет оценить влияние округлений и возможные потери точности.

Разработанная эталонная модель на Python позволяет проверить корректность вычислений перед аппаратной реализацией, а также провести тестирование точности чисел с фиксированной запятой.

Python описание нейронной сети с использованием библиотеки fixpoint приведено в приложении В.

4.3 Разработка операционной части нейронной сети

Нейронная сеть есть объединение управляющего и операционного автоматов, что показано на рисунке 4.5. Такое разбиение соответствует классическому подходу к построению цифровых вычислительных устройств, при котором логика работы системы делится на управление и выполнение.



Рисунок 4.5 – Представление нейронной сети в виде управляющего и операционного автоматов

Элементарное действие, выполняемое в операционном автомате, называется микрооперацией. Пусть множество микроопераций, выполняемых под воздействием сигналов управляющего автомата, обозначается как $Y = y_1, \dots, y_N$.

Для выполнения микрооперации y_i ($i = 1, \dots, N$) необходимо появление соответствующего управляющего сигнала, принимающего значение единицы. Если в операционном автомате одновременно выполняется несколько микроопераций, они образуют микрокоманду.

Множество микрокоманд в совокупности с логическими условиями $X = x_1, \dots, x_N$, определяющими последовательность их выполнения, составляют микропрограмму. Логические условия соответствуют осведомительным сигналам, которые формируются в операционном автомате.

IP-блок нейронной сети должен быть спроектирован как независимый компонент системы, обеспечивающий модульность и возможность интеграции в различные вычислительные среды. Для этого необходимо, чтобы блок обладал четко определенным и простым интерфейсом, что позволит эффективно взаимодействовать с другими компонентами системы.

Интерфейс IP-блока должен обеспечивать передачу входных данных, управление процессом вычислений и выдачу результатов. Важно, чтобы структура взаимодействия соответствовала требованиям к гибкости и масштабируемости системы.

На рисунке 4.6 представлен требуемый вид интерфейса IP-блока, демонстрирующий основные входные и выходные сигналы.

Входные сигналы включают:

- clk – тактовый сигнал, синхронизирующий работу IP-блока;
- rst_n – активный по низкому уровню сигнал сброса, приводящий систему в исходное состояние;
- start_i – управляющий сигнал запуска обработки входных данных;
- addr_i – адресный сигнал, указывающий на текущую ячейку памяти изображения;
- data_i – входные данные, представляющие собой значение пикселя изображения, поступающего в IP-блок.

Выходные сигналы включают:

- num_o – выходной сигнал, содержащий распознанное значение цифры (от 0 до 9), представляющее итог работы нейронной сети;
- rdy_o – сигнал готовности, свидетельствующий о завершении обработки и готовности результата к чтению со стороны процессорной системы.



Рисунок 4.6 – Интерфейс IP-блока нейронной сети

4.4 Проектирование функциональной схемы нейронной сети

При разработке IP-блока нейронной сети необходимо определить его функциональную структуру, обеспечивающую корректное выполнение вычислений, взаимодействие с внешними компонентами системы и эффективное управление процессом обработки данных.

Функциональная схема IP-блока должна описывать основные модули и их взаимодействие, включая блоки обработки входных данных, вычислительное ядро нейронной сети, управляющий модуль и интерфейсы для связи с внешней средой. Важно учитывать требования к производительности, ресурсоемкости и масштабируемости, чтобы обеспечить возможность интеграции IP-блока в различные аппаратные платформы.

Данная нейронная сеть состоит из нескольких основных блоков. Существует два типа блоков обработки пикселя, которые отличаются количеством блоков памяти, которые должны обрабатываться мультиплексором и поступать в качестве входных данных на умножитель.

После вычисления полносвязного слоя для конкретной строки или столбца данные поступают в регистр обработки, состоящий из 28 чисел разрядностью 13 бит. Затем данные из этого регистра последовательно поступают в регистр данных тангенса, из которого они отправляются в блок аппроксимации тангенса гиперболического.

Также используются два пяти разрядных и один десяти разрядный счетчики, которые используются для генерации адреса в память весов и в память изображения.

Блок функции активации softmax реализован на принципе сравнения всех десяти входных классов и выбора наибольшего значения, что будет соответствовать наиболее вероятному значению. В результате на выход поступает индекс максимального элемента.

Для управления работой и генерирования микроопераций на основе логических условий используется микропрограммный автомат.

Электрическая функциональная схема IP-блока нейронной сети прямого распространения приведена в графическом материале ГУИР 431289.003 Э1.

Электрическая функциональная схема блоков обработки пикселей приведена в графическом материале ГУИР 431289.004 Э1.

4.5 Построение алгоритма работы нейронной сети

На данном этапе проектирования создается алгоритм, определяющая последовательность выполнения операций внутри IP-блока нейронной сети. Она описывает порядок обработки входных данных, передачу информации между модулями и выполнение вычислений.

Алгоритм строится на основе дискретных состояний, в которых выполняются конкретные микрооперации. Каждое состояние характеризуется активными управляющими сигналами, обеспечивающими выполнение определенной части вычислений.

Основные этапы работы алгоритма включают:

1 Загрузку изображения в память – входные данные загружаются в буфер оперативной памяти, откуда они будут переданы в вычислительные блоки.

2 Выполнение операций LST-преобразования – производится начальная обработка изображения, включающая нормализацию и предварительное выделение признаков.

3 Передача данных в полносвязный слой – результаты преобразования передаются в полносвязный слой нейросети для дальнейшей обработки.

4 Выполнение финальной классификации – производится анализ данных нейросетью и определение класса входного изображения.

5 Отправка данных в систему обработки – результаты классификации передаются в систему для последующего использования.

Алгоритм работы нейронной сети по распознаванию рукописных цифр представлен в графическом материале ГУИР 431289.005 ПД.

4.6 Проектирование управляющей части

Проектирование управляющей части вычислительного устройства выполняется с применением метода структурирования конечных состояний, который широко используется при разработке цифровых систем управления. Используется канонический метод структурного синтеза микропрограммного автомата (МПА), основанный на синхронной логике. Данный метод обеспечивает формальное построение управляющей логики устройства на основе конечного множества состояний, переходов между ними и соответствующих управляющих сигналов.

МПА делится на два основных структурных компонента:

1 Память управляющих микрокоманд, в которой хранится заданная последовательность управляющих слов. Каждое управляющее слово определяет активность тех или иных сигналов на каждом такте работы устройства, в зависимости от текущего состояния.

2 Комбинационная схема управления, которая отвечает за генерацию адреса следующей микрокоманды. Адрес определяется на основе текущего состояния автомата и внешних условий. Это позволяет гибко управлять переходами между состояниями в зависимости от динамики процесса обработки.

На рисунке 4.7 представлена обобщённая структурная схема такого микропрограммного автомата.

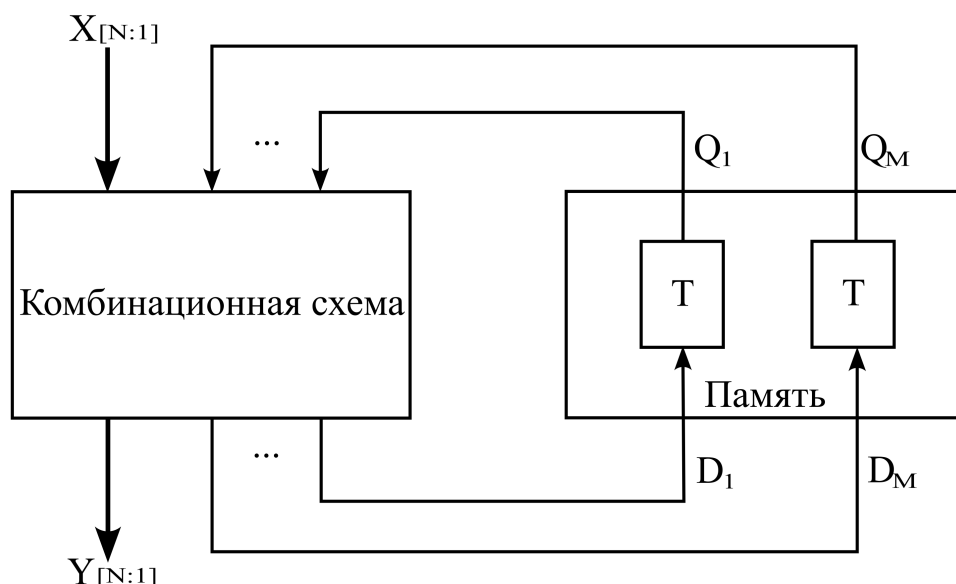


Рисунок 4.7 – Представление управляющей части нейронной сети в виде композиции памяти и комбинационной схемы

Память автомата состоит из заранее выбранных элементов памяти, обеспечивающих хранение информации о текущем состоянии устройства. В качестве элементов памяти используются триггеры, которые позволяют надежно фиксировать и изменять состояние системы.

Количество элементов памяти, необходимых для хранения состояний автомата, определяется по следующей формуле:

$$N = \log_2 M, \quad (4.1)$$

где M – число состояний микропрограммного автомата.

Однако в практических реализациях часто применяется кодирование состояний с использованием кода Грея. Код Грея представляет собой двоичную систему кодирования, в которой любые два последовательных числа отличаются изменением только одного бита [13]. Это позволяет уменьшить количество переключений триггеров при переходе между состояниями, снижая вероятность ошибок, возникающих из-за временных несовпадений сигналов.

Дополнительно, в разработке автоматов управления применяется стратегия, при которой старший бит кодового слова указывает на нахождение автомата в состоянии ожидания. Это состояние используется для инициализации системы, сброса счетчиков и минимизации нежелательных переходов. Применение старшего бита в качестве указателя на режим ожидания помогает уменьшить влияние гличей (glitch) на граф переходов. Гличи – это кратковременные нежелательные изменения сигнала, возникающие при переключении состояний из-за различий во времени распространения сигналов через логические элементы [14]. Они могут приводить к некорректным переходам в автомате, вызывая ошибки в работе системы.

В связи с этим, традиционная формула определения числа элементов памяти модифицируется следующим образом:

$$N = \log_2 M + 1 = \log_2 11 + 1 = 5, \quad (4.2)$$

где $M = 11$ – число состояний микропрограммного автомата.

Каждое состояние автомата описывается представлением из 5 бит, что позволяет повысить надежность работы системы, уменьшить количество переключений триггеров и снизить вероятность ошибок, связанных с глitchами.

В соответствии со списком микроопераций (Y) и логических условий (X), а также разработанной микропрограммой составим граф-схему алгоритма, которая представлена на рисунке 4.8.

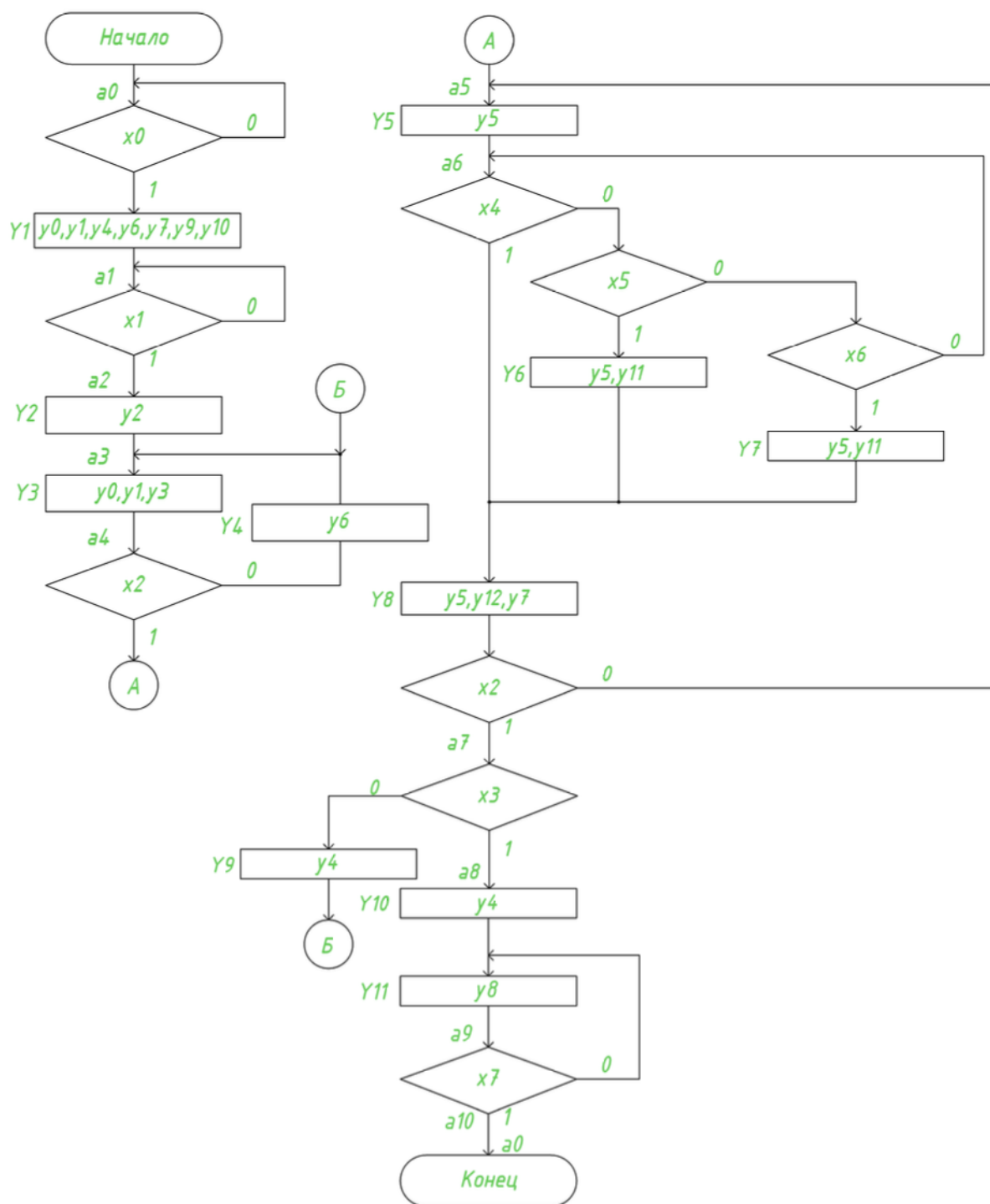


Рисунок 4.8 – Граф-схема алгоритма

В соответствии с разработанной граф-схемой алгоритма составлена таблица 1, в которой представлены все переходы и их условия.

Таблица 1 – Таблица переходов граф-схемы алгоритма

h	a_m	a_s	X_h		Y_t
1	a_0	a_0	$\overline{x_0}$	–	–
2		a_1	x_0	Y_1	$y_0, y_1, y_4, y_6, y_7, y_9, y_{10}$
3	a_1	a_1	$\overline{x_1}$	–	–
4		a_2	x_1	–	–
5	a_2	a_3	1	Y_2	y_2
6	a_3	a_4	1	Y_3	y_0, y_1, y_3
7	a_4	a_4	$\overline{x_2}$	Y_4	y_6
8		a_5	x_2	–	–
9	a_5	a_6	1	Y_5	y_5
10	a_6	a_6	$x_4 \mid \overline{x_4}x_5 \mid \overline{x_4}\overline{x_5}x_6$	$- \mid Y_6 \mid Y_7$	y_6, y_7
11		a_7	x_2	Y_8	y_8
12	a_7	a_3	$\overline{x_3}$	Y_9	y_4
13		a_8	x_3	Y_{10}	y_{10}
14	a_8	a_9	1	–	–
15	a_9	a_9	$\overline{x_7}$	Y_{11}	y_{11}
16		a_{10}	x_7	–	–
17	a_{10}	a_0	1	–	–

4.7 Построение графа переходов управляющего автомата

Граф переходов управляющего автомата позволяет представить граф-схему алгоритма в соответствии с условными обозначениями. Такая форма представления облегчает анализ работы системы и позволяет четко проследить взаимосвязь между различными этапами обработки данных. Граф переходов управляющего автомата представлен на рисунке 4.9.

Для каждого состояния определено mnemonic обозначение, которое обеспечивает удобство представления и понимания процесса работы автомата. Все состояния кодируются в виде пятибитных значений, сформированных в коде Грея.

Состояния граф-схемы алгоритма объединены в блоки:

1 INIT – инициализация системы, установка начальных параметров, необходимых для корректной работы всех последующих этапов.

2 LOAD – загрузка изображения.

3 LD_MEM – подготовка данных.

4 INIT_CALC – инициализация смещения.

5 CALC – обработка строки/столбца.

- 6 RDY – готовность результата обработки.
 7 TG_CALC – вычисление тангенса.
 8 UPD_RCO – смена слоя.
 9 INIT_OUT – инициализация выходных смещений.
 10 OUT_CALC – обработка выходного полносвязного слоя.
 11 RDY_OUT – готовность детектирования.

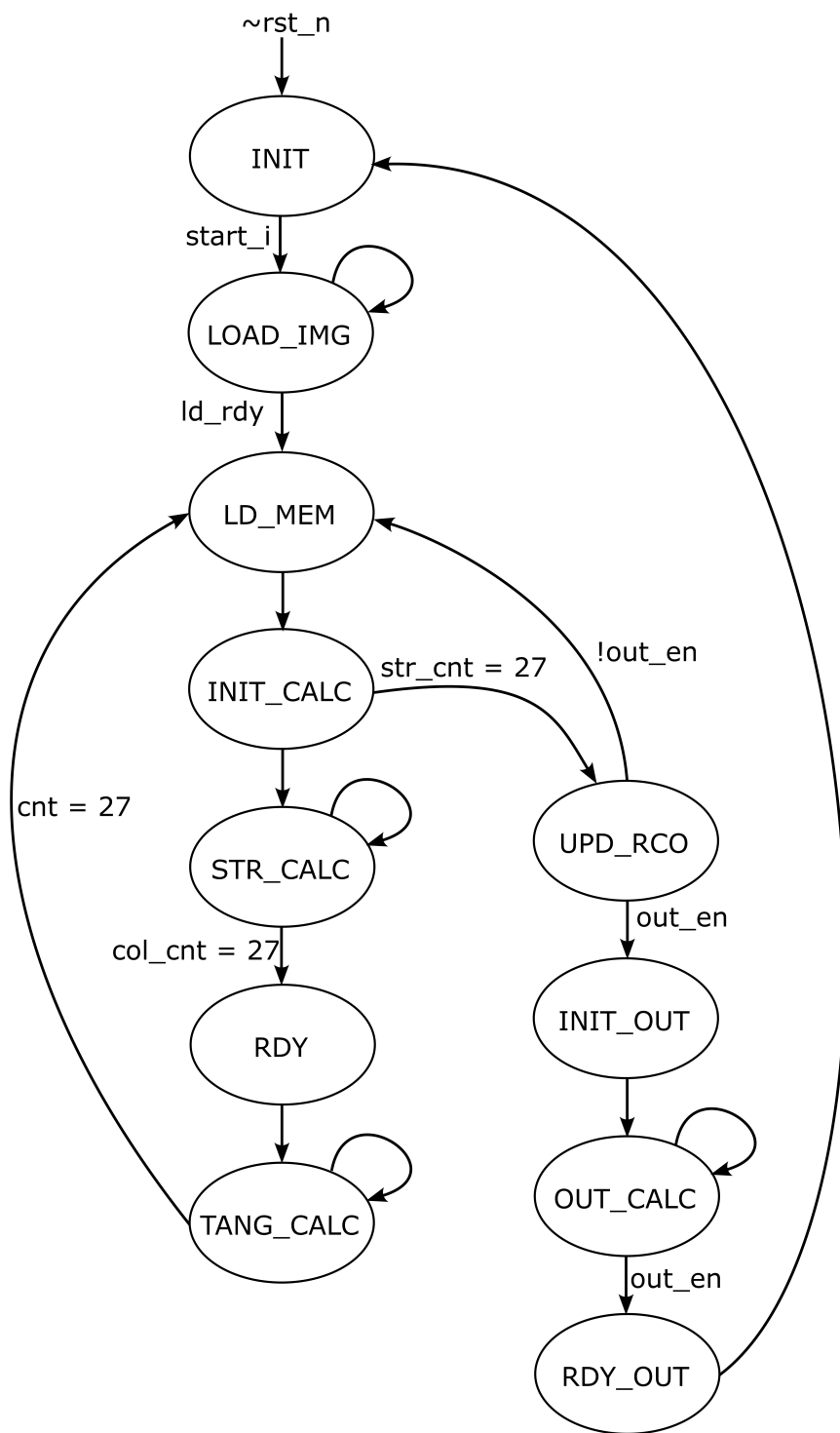


Рисунок 4.9 – Граф переходов

5 АППАРАТНАЯ РЕАЛИЗАЦИЯ IP-БЛОКА НЕЙРОННОЙ СЕТИ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

5.1 Описание пакета Xilinx Vivado

Xilinx Vivado – это современный программный пакет для проектирования цифровых схем на базе программируемых логических интегральных схем, разработанный компанией Xilinx. Данный программный комплекс является преемником Xilinx ISE и значительно превосходит его по возможностям и производительности.

Vivado предназначен для автоматизированного проектирования цифровых схем, начиная с этапа описания логики на языках описания аппаратуры и заканчивая загрузкой сгенерированного битового потока в устройство[15].

Одним из ключевых преимуществ Vivado является использование более производительных алгоритмов синтеза и оптимизации, что позволяет существенно ускорить процесс разработки. В отличие от традиционных средств проектирования, в Vivado реализована поддержка параллельных вычислений, что делает его особенно эффективным при работе с большими проектами.

В состав пакета Vivado входят несколько инструментов, среди которых[15]:

1 Vivado IDE – интегрированная среда разработки, предоставляющая удобный графический интерфейс для управления проектами, анализа схемотехнической логики и моделирования.

2 Vivado Synthesis – инструмент синтеза логики, который преобразует код HDL в оптимизированное представление на уровне вентилях.

3 Vivado Implementation – модуль размещения, трассировки и оптимизации логики на кристалле FPGA.

4 Vivado Simulator – встроенный инструмент для моделирования цифровых схем на уровне поведенческой, функциональной и временной верификации.

5 Vivado IP Integrator – инструмент для работы с IP-ядрами и объединения их в сложные системы на кристалле.

6 Vivado High-Level Synthesis (HLS) – средство высокоуровневого синтеза, позволяющее описывать аппаратные алгоритмы на языках C/C++ и автоматически преобразовывать их в оптимизированный RTL-код.

Основные возможности Vivado:

1 Поддержка языков описания аппаратуры VHDL и Verilog. Vivado позволяет разрабатывать схемы на двух основных языках HDL, а также поддерживает SystemVerilog для верификации цифровых проектов. Это дает широкие возможности для разработки и тестирования аппаратуры.

2 Автоматизированный синтез и размещение логики на FPGA. Vivado включает мощный механизм синтеза, который автоматически преобразует HDL-код в оптимизированную сетевую схему, выполняет ее размещение на кристалле FPGA и прокладывает соединения между логическими блоками.

3 Встроенные инструменты для моделирования и отладки.

4 Генерация IP-ядер и использование встроенных библиотек компонентов.

Для реализации нейронной сети Vivado используется на нескольких ключевых этапах разработки. Во-первых, он применяется для генерации аппаратной логики, необходимой для выполнения расчетов внутри FPGA. Во-вторых, с его помощью производится компиляция проекта и синтез аппаратной схемы. В-третьих, Vivado используется для загрузки сгенерированного битового потока в ПЛИС и отладки работы модели.

5.2 Описание функциональных блоков

Аппаратная реализация нейронной сети включает несколько ключевых функциональных блоков:

- 1 Блок обработки пикселя.
- 2 Блок вычисления тангенса.
- 3 Блок вычисления softmax.
- 4 Блок памяти изображения.
- 5 Блок управляющего автомата.
- 6 Блок MAC.

Каждый из этих блоков реализуется в виде аппаратного модуля на HDL.

В блоке обработки пикселя осуществляется установка параметра смещения для подготовки к вычислению слоя, а также выбирается соответствующий данному этапу весовой коэффициент из памяти.

В блоке вычисления тангенса осуществляется аппроксимированный расчет функции активации тангенс гиперболический для LST-1 слоя.

Прямая реализация вычисления тангенса будет занимать большое количество вычислительных ресурсов. Поэтому для удобства реализации используется аппроксимация в соответствии с формулой[16]:

$$\tanh(x) \approx F(x) = \begin{cases} \text{sign}(x), & \text{если } |x| > 2, \\ (1 + \frac{x}{4}) \cdot x, & \text{если } -2 < x < 0, \\ (1 - \frac{x}{4}) \cdot x, & \text{если } 0 < x < 2. \end{cases} \quad (5.1)$$

В блоке вычисления softmax происходит сравнение входных данных и определение наиболее вероятного класса.

В блоке памяти изображения храниться результат обработки на каждом этапе. От приемки входных данных, до выхода LST-1 слоя.

Блок управляющего автомата осуществляет управление и синхронизацию между всеми блоками устройства и обрабатывает входные управляющие сигналы.

Блок MAC строится на основе блока DSP48. Данный блок используется для вычисления операции умножения с накоплением. В данном блоке по сигналу инициализации устанавливается смещение, необходимо для текущего этапа вычислений, а затем выполнение операции MAC на каждом такте синхронизации.

Реализация данных блоков на языке SystemVerilog представлена в приложении Г.

5.3 Описание интерфейса нейронной сети

Как было показано ранее на рисунке 4.5 нейронная сеть состоит из пяти обязательных входных и двух выходных сигналов. Взаимодействие процессорной системы и программируемой логики осуществляется через интерфейс AXI4-lite.

Необходимым условием, выдвигаемым к IP-блоку будет поддержка интерфейса AXI4-lite. Для упрощения формирования IP-блока будем использовать связку AXI-uP. uP интерфейс напрямую взаимодействует с модулем верхнего уровня. Далее данные передаются в AXI-slave модуль, который и подключается к процессорной системе.

5.4 Описание процесса создания блочной диаграммы

Рассмотрим основные задачи связанные с созданием блочной диаграммы (Block diagram), подключением готовых IP-ядер и созданием собственных, а также созданием топ-модуля на языке Verilog. Создадим блочную диаграмму, что показано на рисунке 5.1.

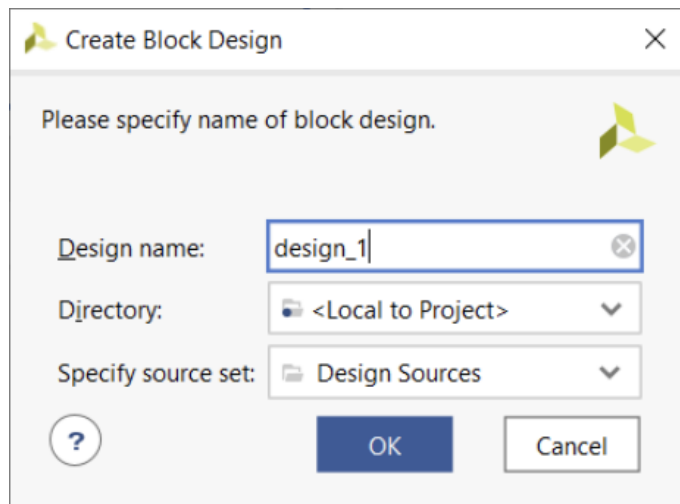


Рисунок 5.1 – Создание блочной диаграммы

Приступим к добавлению выводов процессорной системы в ПЛИС и сопутствующих IP-ядер, для этого нужно добавить ZYNQ7. После САПР Vivado предложит запустить мастер автоматического подключения «Block Automation», что показано на рисунке 5.2.

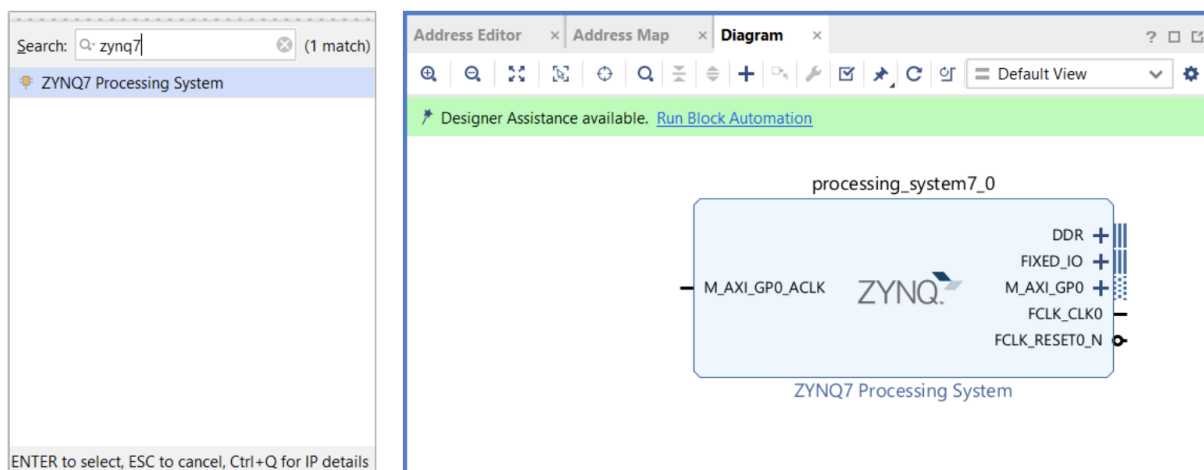


Рисунок 5.2 – Окно выбора IP-ядра и блочной диаграммы создаваемого проекта с «Мастером подключений» типа «Block Automation»

Мастер автоматического подключения «Block Automation» ZYNQ7 обладает опциями, показанными на рисунке 5.3.

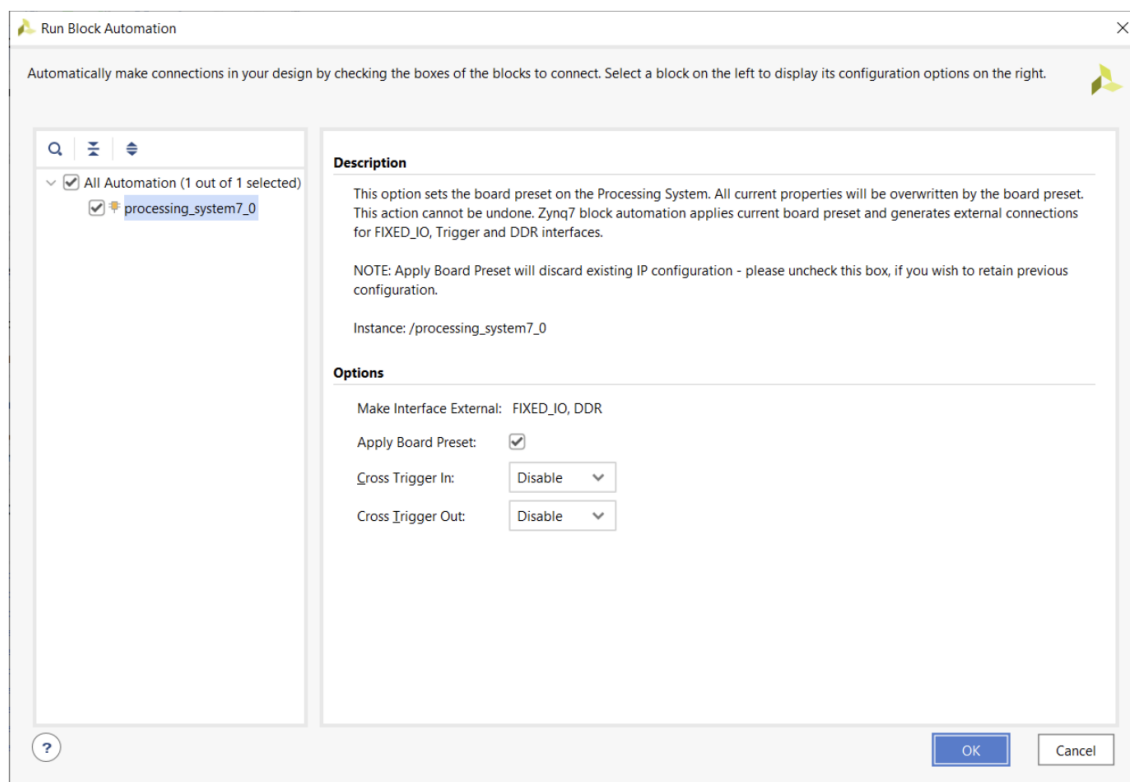


Рисунок 5.3 – Опции мастера автоматической настройки ZYNQ7

Перейдем к созданию собственного IP-ядра, что показано на рисунке 5.4

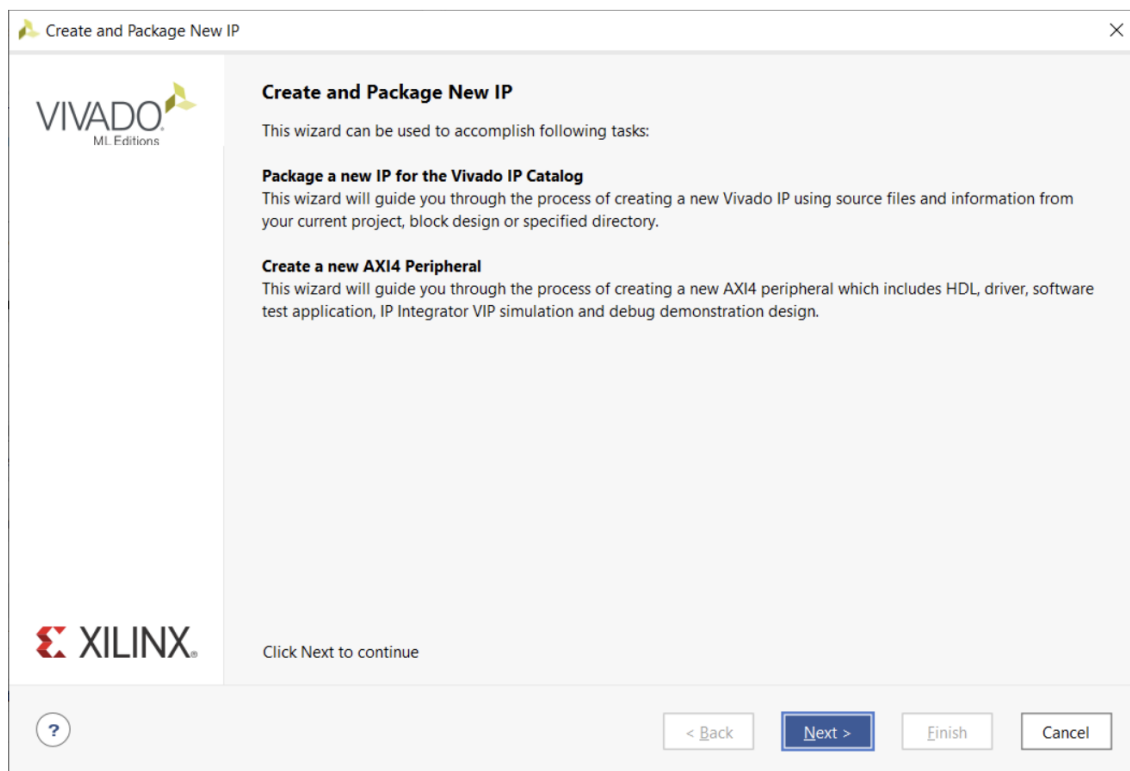


Рисунок 5.4 – Создание IP-ядра в САПР Vivado

Создадим новое IP-ядро и выберем тип интерфейса AXI4(рисунок 5.5).

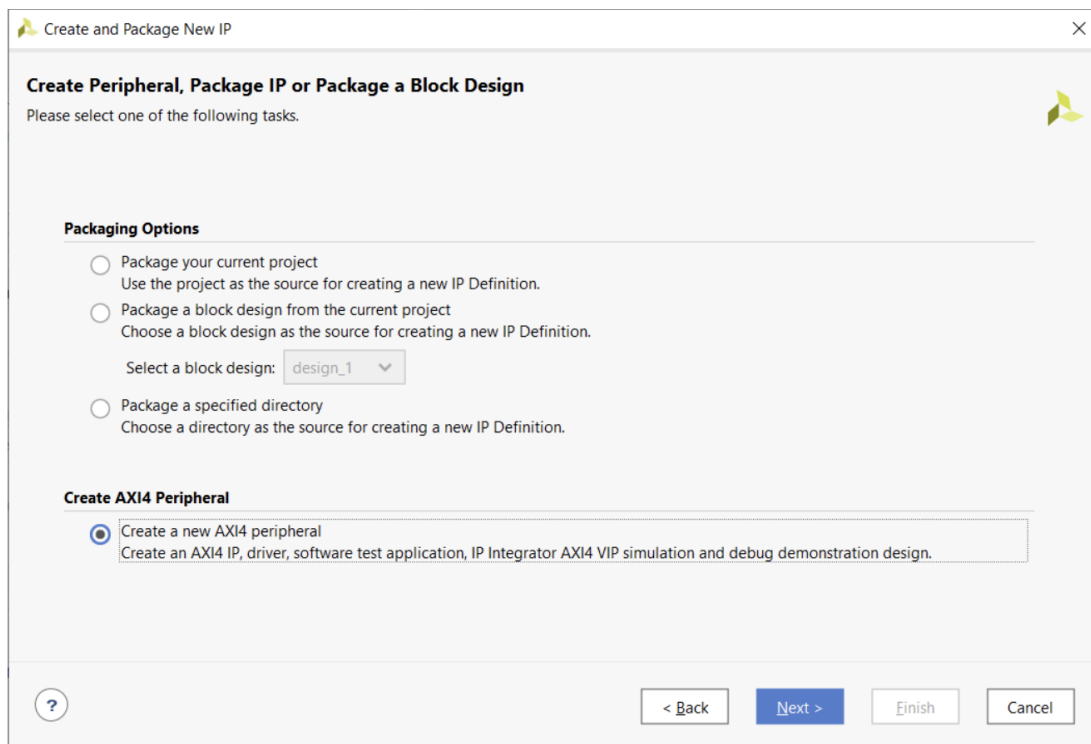


Рисунок 5.5 – Выбор типа IP-ядра

Зададим имя и описание нового IP-ядра(рисунок 5.6).

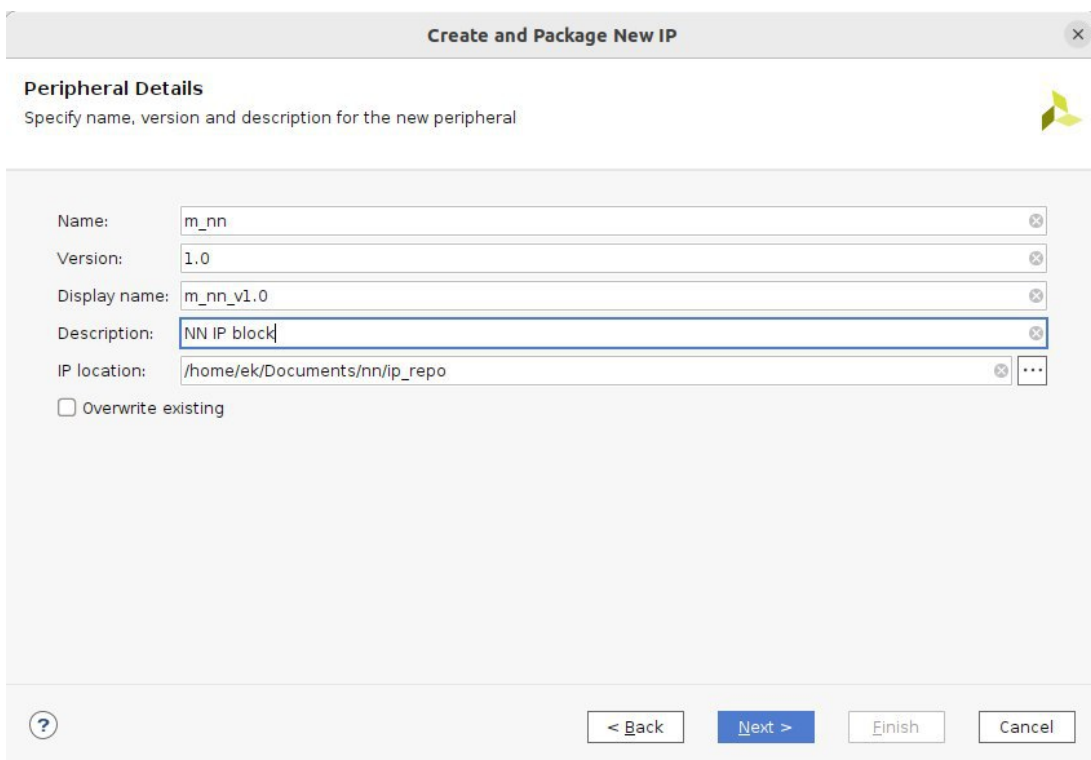


Рисунок 5.6 – Описание IP-ядра

Укажем типы интерфейсов будущего IP-ядра, а также зададим имя AXI4-Lite интерфейса как `s_axi`, что показано на рисунке 5.7

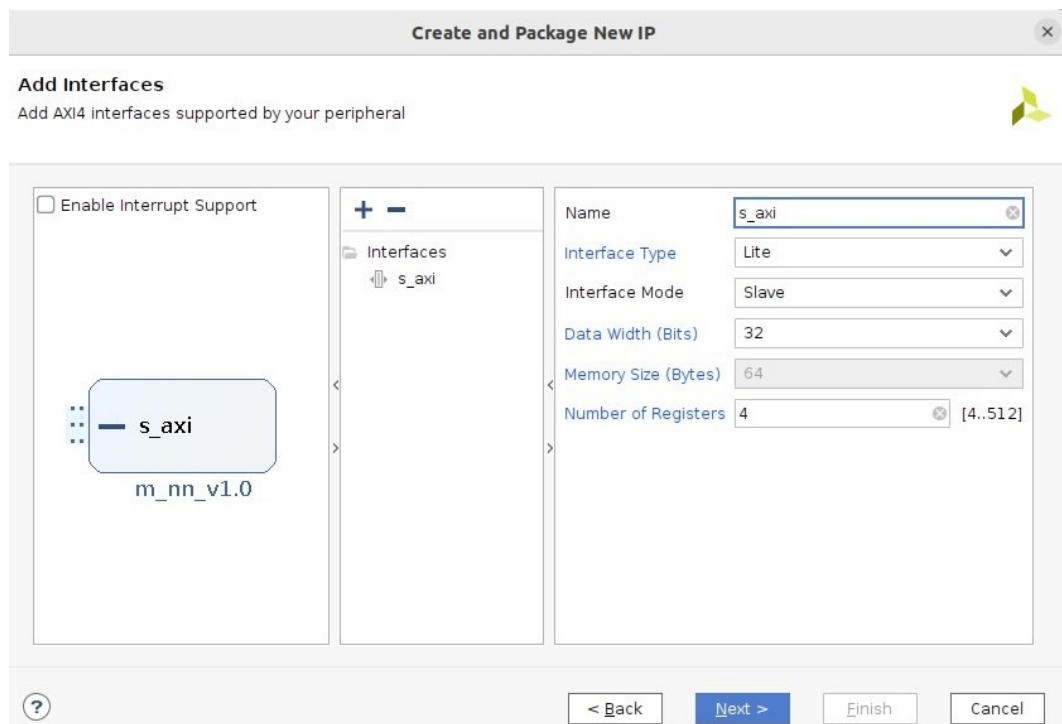


Рисунок 5.7 – Аппаратные интерфейсы IP-ядра

Финализируем создание IP-ядра и выберем его редактирование(рисунок 5.8).

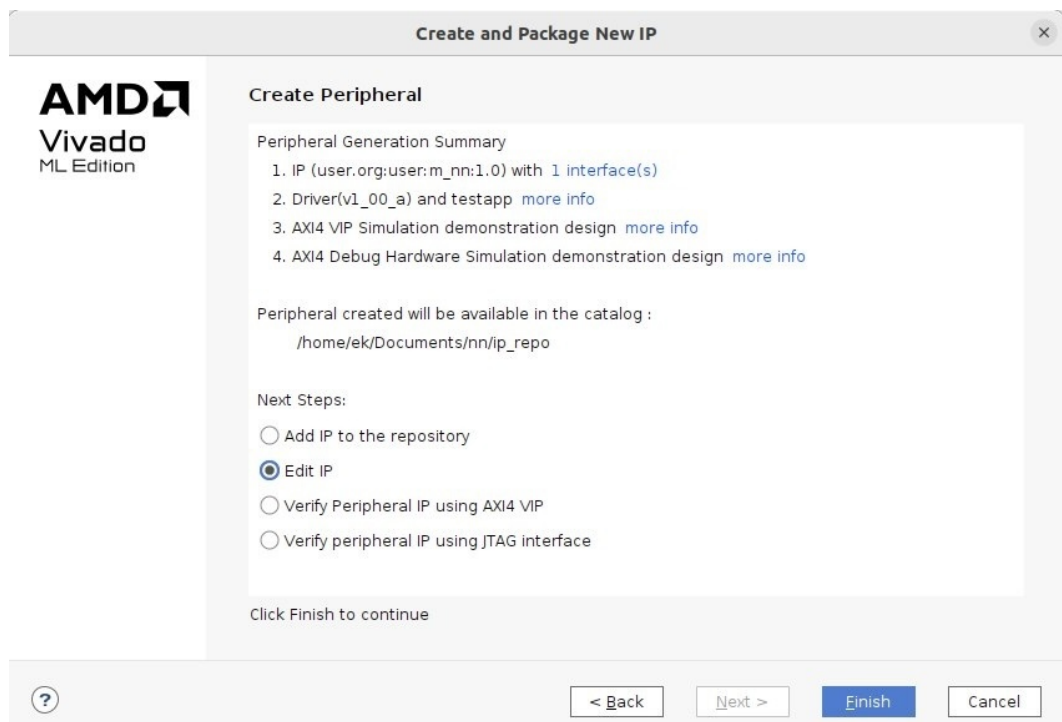


Рисунок 5.8 – Завершение создания IP-ядра

Далее выполним добавление всех файлов в IP-блок, что представлено на рисунке 5.9

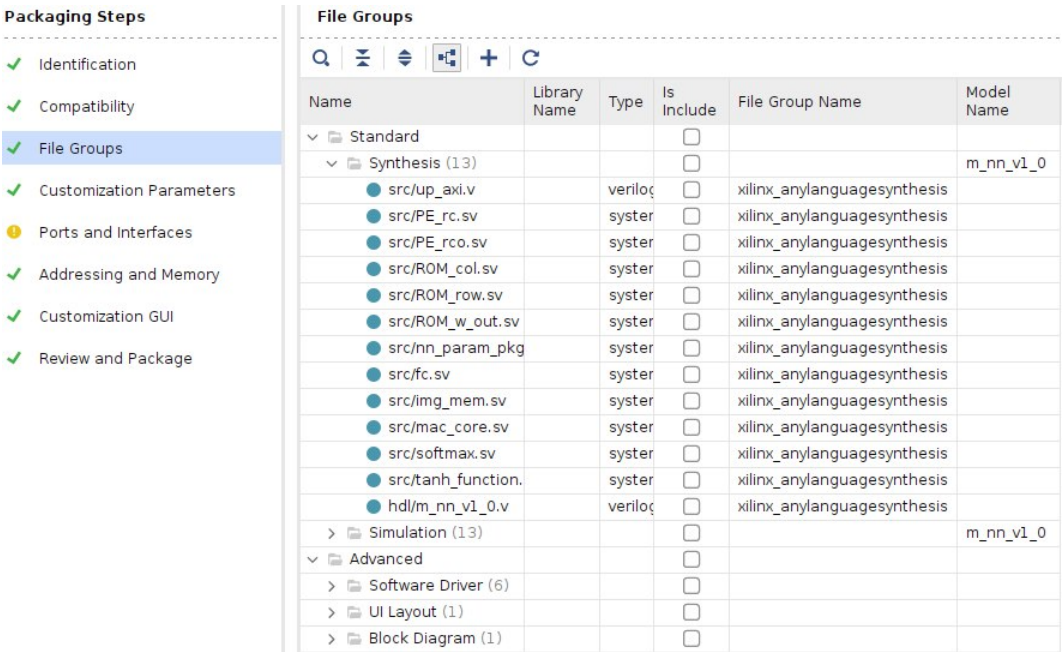


Рисунок 5.9 – Добавление файлов

Затем ядро можно перепаковать, что представлено на рисунке 5.9

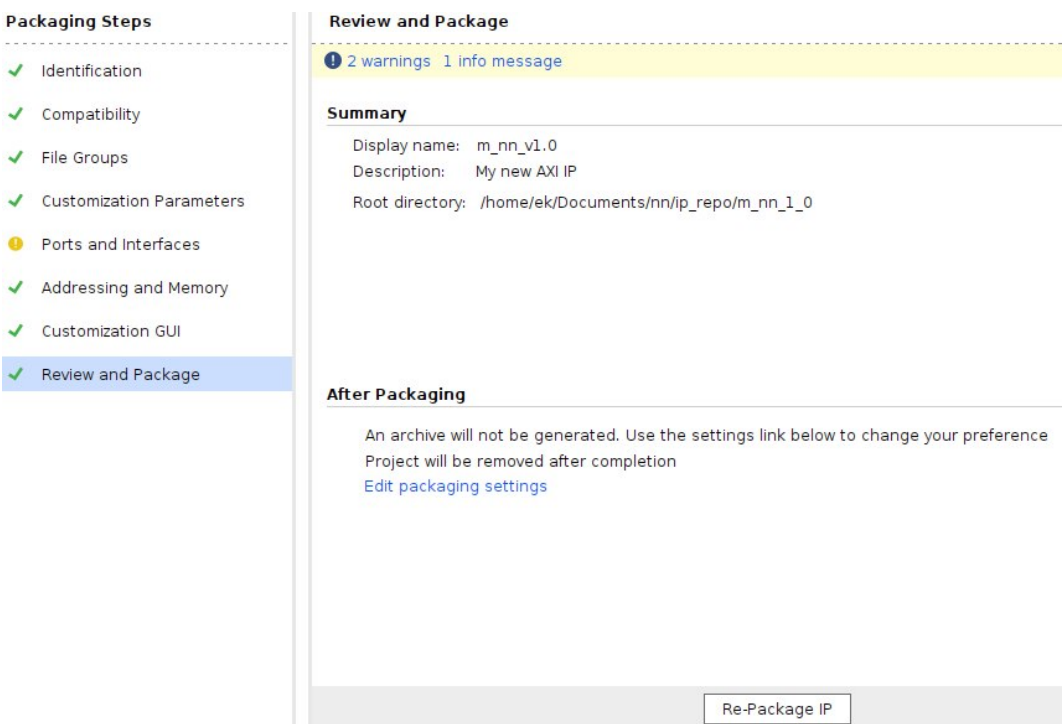


Рисунок 5.10 – Завершение создания IP-ядра

Далее необходимо перейти к построению блочной диаграммы аппаратной части проекта в среде Vivado. Блочная диаграмма представляет собой графиче-

ское описание аппаратных соединений между IP-ядрами, процессорной системой и периферией.

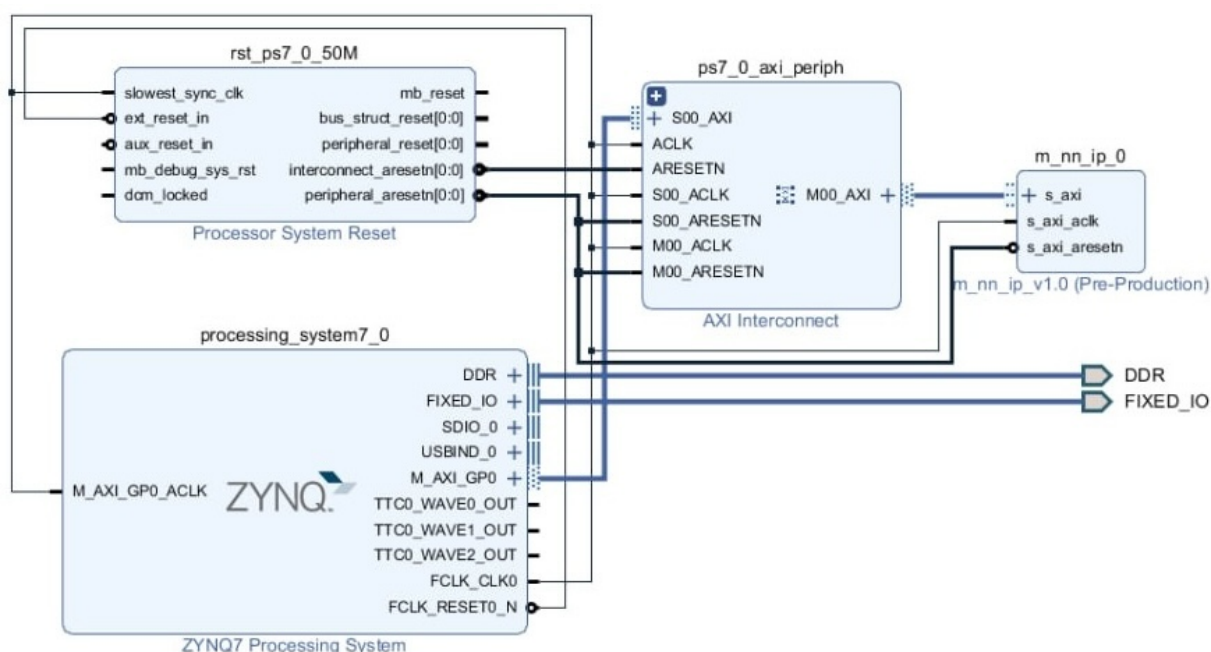


Рисунок 5.11 – Результирующий вид блочной диаграммы

Для корректной работы системы требуется обеспечить стабильный тактовый сигнал с частотой 80 МГц. По умолчанию, тактовый генератор в Zynq-процессорной системе может выдавать несколько частот, но для нужд пользовательского IP-блока необходима именно частота 80 МГц.

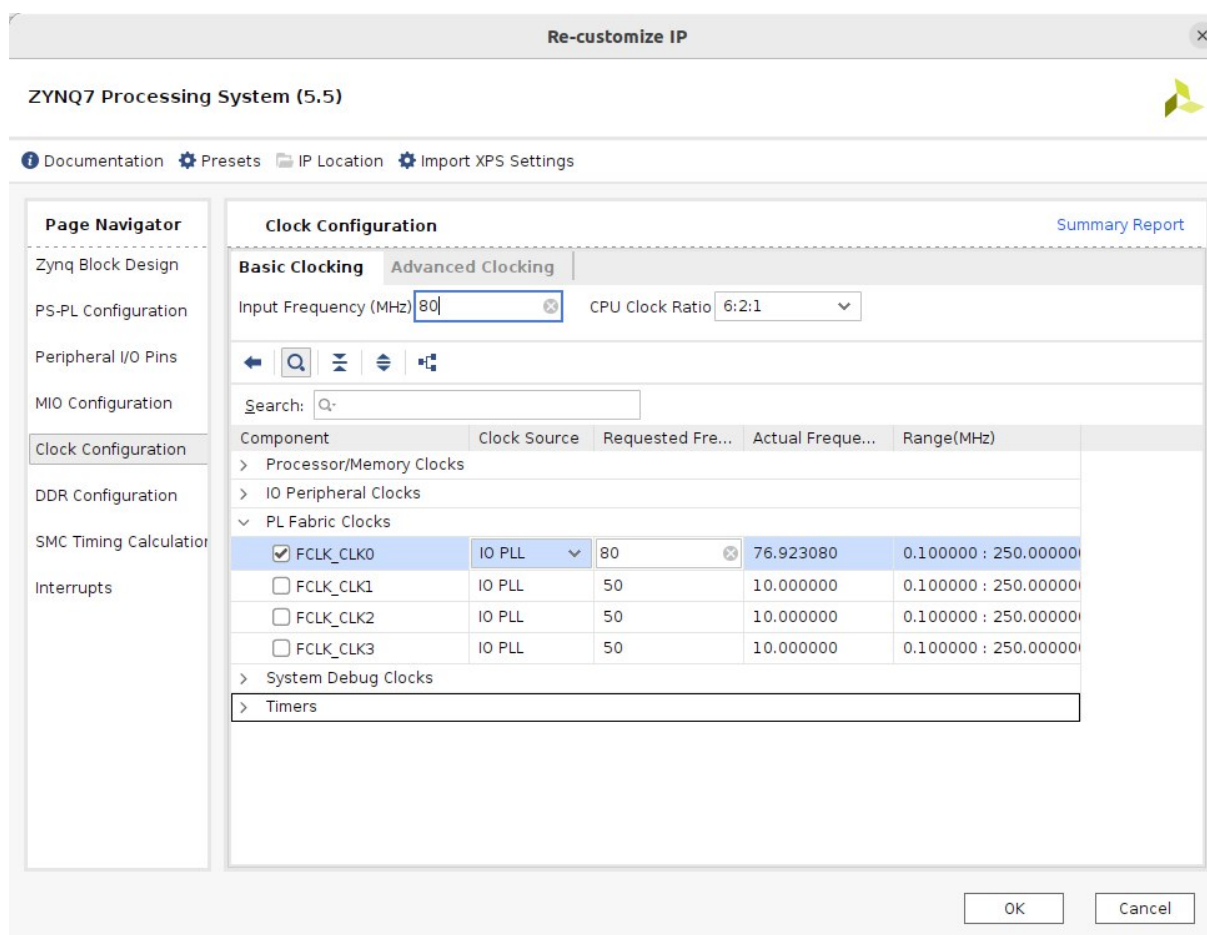


Рисунок 5.12 – Установка частоты 80 МГц в блоке Processing System

5.5 Результаты синтеза и симуляции

Для проверки работоспособности написанного устройства был составлен тест, в котором формируются управляющие воздействия и изображение. Работа устройства начинается по сигналу start_i.

Для тестирования было взято два изображения, что показано на рисунке 5.13.



Рисунок 5.13 – Тестируемые изображения

Тест представлен в приложении Г. Для проверки корректности работы нейронной сети необходимо сравнить выход результирующего полносвязного слоя и нейронной сети.

Name	Value
clk	1
rst_n	1
start_i	0
x[9:0]	30f
din[12:0]	1f80
num_o[5:0]	01
rdy_o	1
internal_rc[27:0][12:0]	1d29,1c5b,1e11,0055,1d6e,1e65,1f49,1d08,031
internal_tanh[27:0][12:0]	1ee3,1e4e,01d8,1ed5,1fc7,1f20,1fe9,0004,01a1,1fca,1ef6,1fc7,0130,1fd0,004b,1eda,020c,006
result[9:0][12:0]	-6.265625,-8.46875,-4.7734375,-10.828125,-6.265625,-8.46875,-4.7734375,-10.828125,-6.265625,-8.46875,-4.7734375,-10.828125,-6.265625,-8.46875,-4.7734375,-10.828125,-6.265625,-8.46875,-4.7734375,-10.828125
sel_data	0
init	0
en	1
mem_en	0
tg_en	0
fc_addr[4:0]	00
str_cn[4:0]	1c
rco_sel[1:0]	2

Softmax: [-17.140625 4.625 -7.2890625 -6.21875 -4.9453125 -6.1328125
 -10.828125 -4.7734375 -8.46875 -6.265625]

Num: 1

Timing diagram showing signals: Name, Value, clk, rst_n, start_i, x_[9:0], din_[12:0], num_o[5:0], rdy_o, internal_rcd[27:0][12:0], internal_tanh[27:0][12:0], result[9:0][12:0], sel_data, init, en, mem_en, tg_en, fc_addr[4:0], str_cnt[4:0], rco_sel[1:0]. The diagram shows a sequence of operations over time, with a yellow vertical line indicating a specific point in time.

В результате проведённого моделирования и сопоставления работы реализованной аппаратной модели с эталонной моделью на языке Python с использова-

нием библиотеки fixpoint было выявлено полное соответствие выходных данных. Это подтверждает корректность проектирования и реализации нейронной сети на уровне RTL-описания и её работоспособность при выполнении на целевой ПЛИС.

Для оценки ресурсоёмкости и эффективности реализации проекта был проведён анализ использования аппаратных ресурсов, таких как логические элементы (LUT), триггеры (FF), специализированные арифметические блоки (DSP) и память (BRAM), полученные в процессе синтеза HDL-описания.

Полученные результаты представлены на рисунке 5.16.

Utilization				Post-Synthesis	Post-Implementation
				Graph Table	
Resource	Estimation	Available	Utilization %		
LUT	1538	17600	8.74		
LUTRAM	169	6000	2.82		
FF	27	35200	0.08		
BRAM	33	60	55.00		
DSP	57	80	71.25		
IO	33	100	33.00		
BUFG	2	32	6.25		

Рисунок 5.16 – Аппаратные затраты

Для дополнительной верификации корректности функционирования разработанной системы проводится временная симуляция на основе результатов синтеза и имплементации.

На данном этапе осуществляется моделирование с учетом временных задержек, рассчитанных инструментом синтеза, однако ещё без учёта конкретных физических ресурсов устройства. Постсинтезная симуляция позволяет убедиться, что логическая структура проекта сохранена и корректно функционирует после преобразования исходного RTL-кода в структуру из примитивов целевой ПЛИС.

Результаты такой симуляции представлены на рисунке 5.17. Видно, что формируемые сигналы соответствуют ожидаемым значениям, соблюдаются временные зависимости и интерфейсы взаимодействуют корректно.

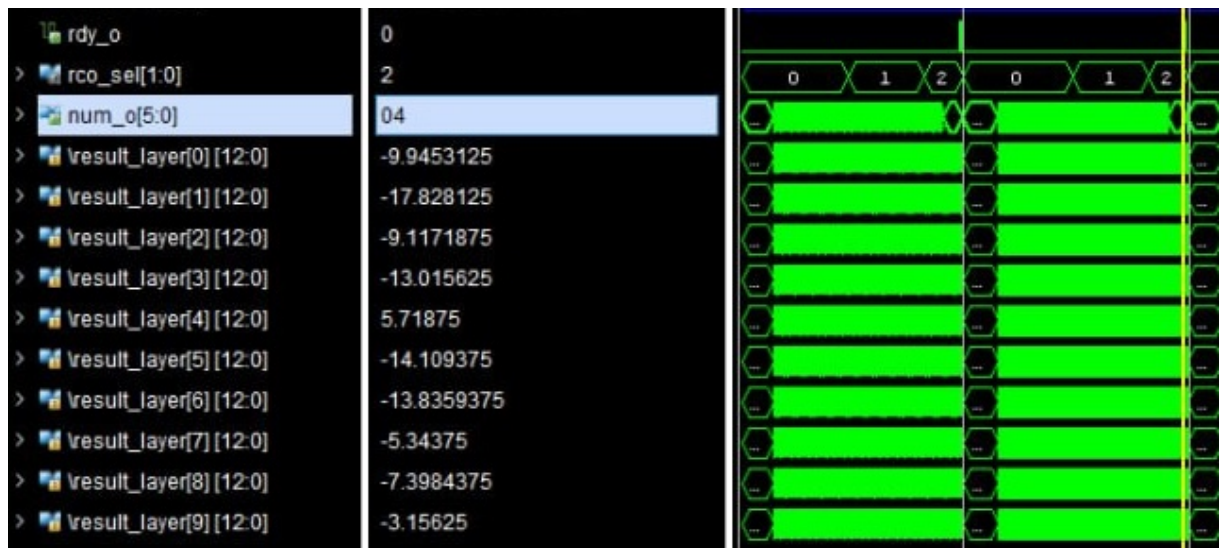


Рисунок 5.17 – Временная диаграмма нейронной сети на втором изображении при пост-синтез симмуляции

После завершения этапа имплементации, включающего размещение и трассировку (place & route), производится повторная временная симуляция, уже с учетом всех физических задержек, связанных с реальной топологией на кристалле. Это позволяет оценить окончательные временные характеристики и убедиться, что проект не нарушает временных ограничений (timing constraints).

На рисунке 5.18 показана временная диаграмма, соответствующая данному этапу. Анализ сигналов подтверждает корректную работу всех блоков системы, несмотря на добавление реальных задержек межсоединений и элементов логики.

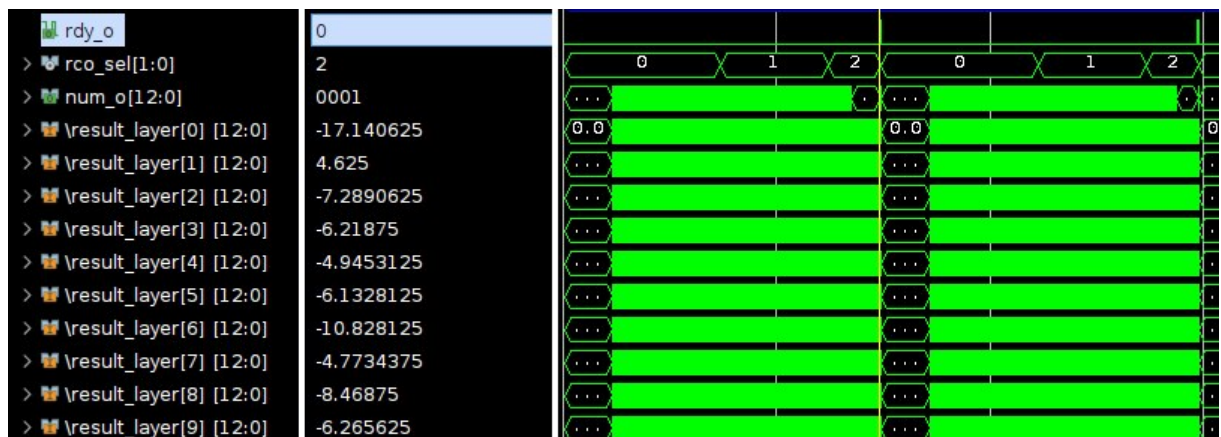


Рисунок 5.18 – Временная диаграмма нейронной сети на первом изображении при пост-имплементация симмуляции

В результате этапа имплементации (Placement and Routing) была получена финальная топология расположения логических элементов, специализированных блоков и соединений на кристалле ПЛИС. Полученная схема представлена на рисунке 5.19.

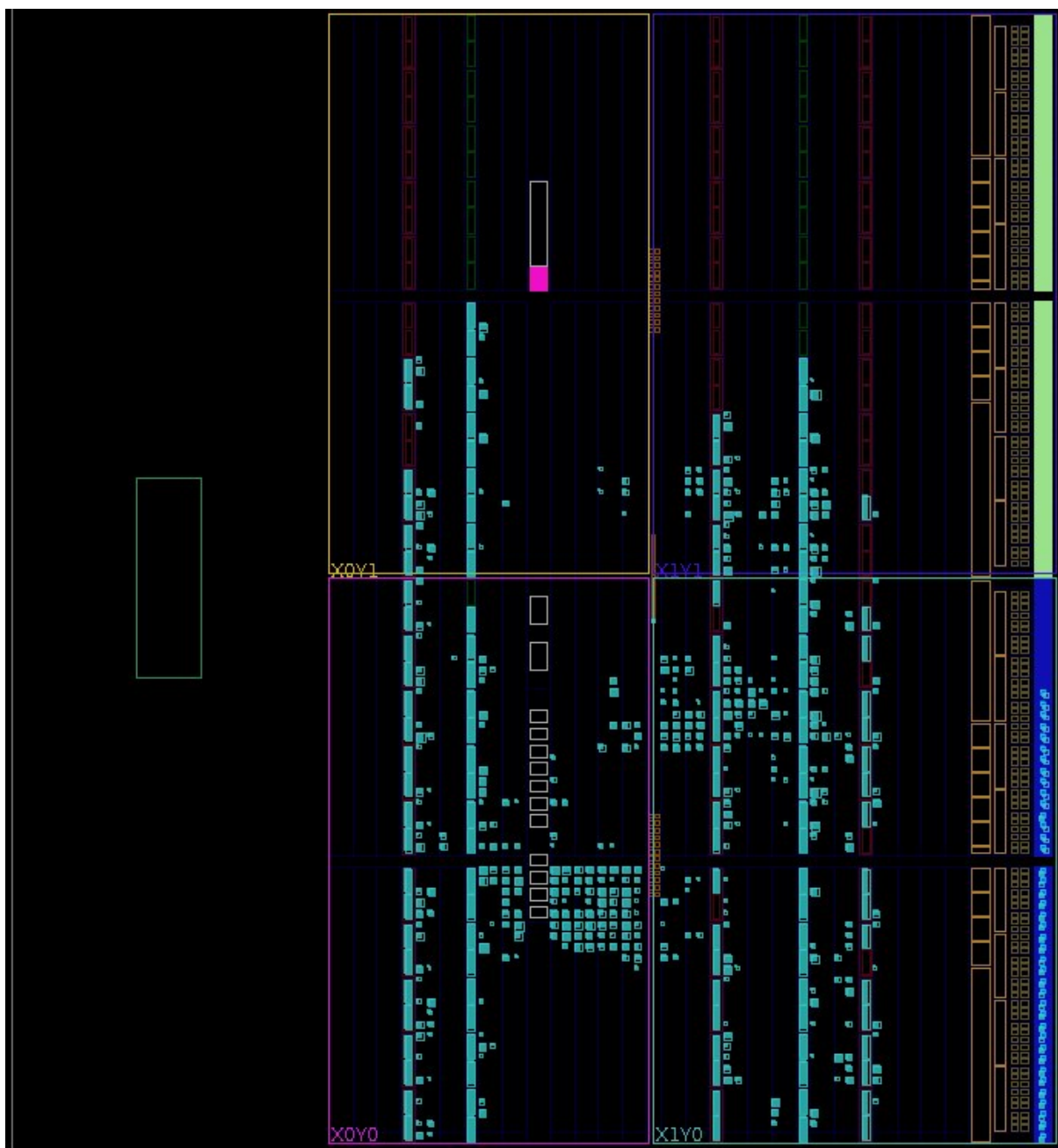


Рисунок 5.19 – Расположение на кристалле

На изображении показано физическое распределение ресурсов проекта по кристаллу. Яркие выраженные прямоугольные и квадратные блоки соответствуют различным логическим компонентам, таким как:

- блоки логики (LUT/FF);
- цифровые сигнальные процессоры (DSP48);
- блоки оперативной памяти (BRAM);
- входные/выходные буферы (IOB).

Размещение было выполнено автоматически средствами среды синтеза и имплементации, с учётом топологии кристалла и требований по задержкам сигнала. На изображении можно видеть, что распределение элементов достаточно компактное и сбалансированное, без чрезмерной фрагментации или перегруженных областей, что положительно сказывается на тактовой частоте и стабильности

работы системы.

Ещё одним важным параметром при анализе аппаратной реализации является потребляемая мощность, которая играет ключевую роль при выборе целевого устройства и оценке общей энергоэффективности системы.

На рисунке 5.20 представлена оценка энергопотребления реализованной нейронной сети на основе результатов имплементации.

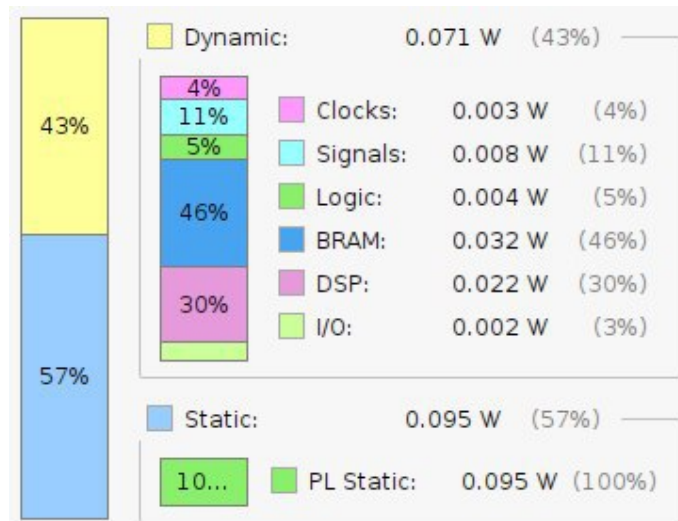


Рисунок 5.20 – Оценка потребляемой мощности

Общая мощность составляет 0.166 Вт, из которых:

1 Статическая мощность (Static) — 0.095 Вт, что составляет 57% от общего потребления. Это базовая мощность, потребляемая устройством при включении, вне зависимости от его активности.

2 Динамическая мощность (Dynamic) — 0.071 Вт (43%), обусловлена непосредственной работой схем, переключением сигналов, выполнением операций и передачей данных.

В структуре динамической мощности:

1 BRAM модули потребляют 0.032 Вт (46%), что обусловлено активной работой памяти при обработке входных данных и весов.

2 DSP блоки — 0.022 Вт (30%), благодаря интенсивному использованию для выполнения операций умножения.

3 Сигналы и тактовые линии составляют 0.008 Вт (11%) и 0.003 Вт (4%) соответственно.

4 Остальные компоненты (логика и I/O) потребляют сравнительно незначительное количество энергии.

Результаты проектирования сгруппированы и представлены в графическом материале ГУИР 431289.006 ПЛ.

5.6 Устройство основных функциональных блоков

Ключевым этапом в проектировании аппаратной реализации нейросети является синтез и анализ структуры функциональных блоков. Это позволяет оценить ресурсоемкость, функциональность и корректность разработанных аппаратных модулей, а также выявить узкие места при дальнейшей интеграции в систему на базе ПЛИС.

Одним из базовых компонентов системы является блок памяти изображений, необходимый для хранения входных данных – изображений рукописных цифр. Данный блок реализован с использованием двух логических таблиц, выполняющих формирование сигналов WE (Write Enable) и RSTRAM (Reset RAM) для управления чтением и записью. Благодаря такой реализации обеспечивается компактность и низкое потребление аппаратных ресурсов.

На рисунке 5.21 представлена структурная схема данного модуля. Видно, что блок организован эффективно.

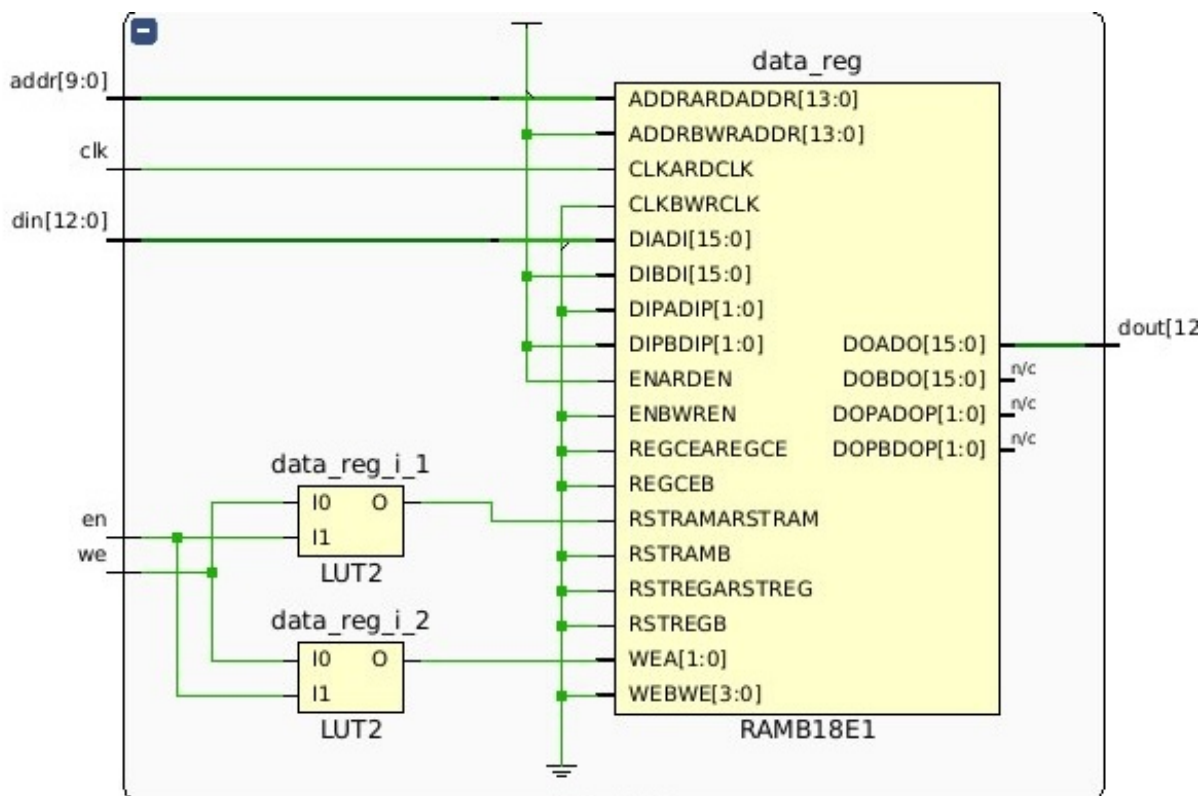


Рисунок 5.21 – Структура блока памяти изображения

Следующей критически важной частью являются процессорные блоки, отвечающие за выполнение вычислений, связанных с прохождением данных через нейронную сеть. Эти блоки реализуют операции умножения входных значений на веса, аккумуляции и применение функции активации. Процессорные элементы представлены в двух вариантах.

Первый тип процессорных блоков предназначен для обработки выходного слоя нейросети. Его особенностью является наличие встроенной памяти, хранящей весовые коэффициенты выходного полносвязного слоя.

На рисунке 5.22 показана структура такого блока. В отличие от предыдущего, он содержит дополнительные логические цепочки, для чтения значения из третьей памяти и требует больше логических элементов.

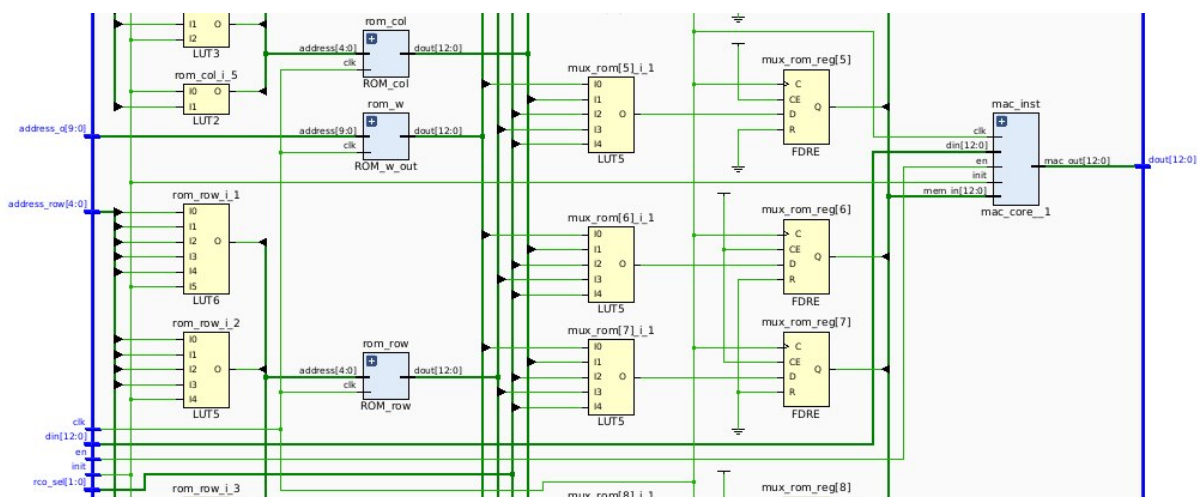


Рисунок 5.22 – Часть структуры первого блока обработки изображения

На рисунке 5.23 показана часть внутренней структуры второго процессорного блока. Видны цепочки логических элементов, осуществляющие чтение данных из двух различных блоков памяти весовых коэффициентов и выбор одного из коэффициентов для подачи на MAC ядро.

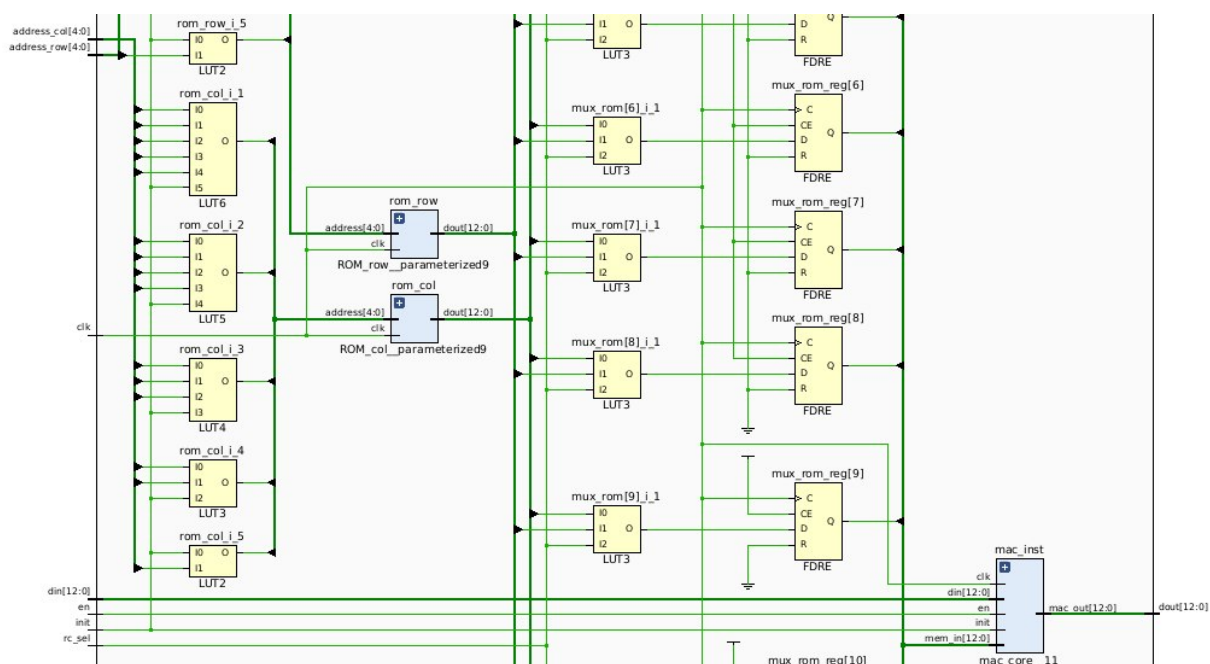


Рисунок 5.23 – Часть структуры второго блока обработки изображения

На рисунке 5.24 представлены результаты синтеза отдельных данных модулей. Каждый блок был синтезирован и проанализирован по количеству используемых логических элементов, блоков памяти, триггеров и цифровых сигнальных процессоров.

Resource	Estimation	Available	Utilization %	Resource	Estimation	Available	Utilization %
LUT	24	17600	0.14	LUT	18	17600	0.10
FF	12	35200	0.03	FF	12	35200	0.03
BRAM	1.50	60	2.50	BRAM	1	60	1.67
DSP	2	80	2.50	DSP	2	80	2.50
IO	49	100	49.00	IO	38	100	38.00
BUFG	1	32	3.13	BUFG	1	32	3.13

Рисунок 5.24 – Аппаратные затраты на различные типы блоков обработки пикселя

В частности, использование дополнительной встроенной памяти в блоке второго типа приводит к увеличению общего количества задействованных BRAM-блоков. Кроме того, возросло количество LUT-блоков, что обусловлено добавлением логики формирования мультиплексоров, обеспечивающих выбор между несколькими блоками хранения весов.

Структура MAC ядра основана на использовании блока DSP48, что показано на рисунке 5.25.

6 АНАЛИЗ РЕЗУЛЬТАТОВ ТЕСТИРОВАНИЯ IP-БЛОКА НЕЙРОННОЙ СЕТИ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

6.1 Описание среды тестирования

В качестве аппаратной платформы для реализации и тестирования разработанных решений использовалась плата на базе программируемой логической интегральной схемы ZYBO Z7 от компании Digilent. Плата ZYBO Z7 представляет собой одноплатный компьютер, основанный на СнК Xilinx Zynq-7000, которая сочетает в себе двухъядерный процессор ARM Cortex-A9 и среду программируемой логики (PL) на одном чипе[9].

ПЛИС ZYBO Z7 оснащена широким набором периферийных интерфейсов, таких как HDMI, USB, Ethernet, аудиоразъёмы и слот для карты памяти microSD. Такая конфигурация позволяет использовать плату как для высокопроизводительных вычислений, так и для задач обработки сигналов и изображений в реальном времени. Программируемая часть ПЛИС используется для создания специализированных аппаратных ускорителей обработки данных, а процессорная часть (Processing System, PS) обеспечивает управление и взаимодействие между различными компонентами системы[17].

Для упрощения разработки приложений и взаимодействия между процессорной системой и пользовательскими IP-блоками программируемой логики, в работе применялась среда PYNQ (Python Productivity for Zynq). PYNQ представляет собой специализированную программную платформу, основанную на операционной системе Linux, которая позволяет использовать язык программирования Python для управления аппаратными ресурсами Zynq[18]. PYNQ предоставляет удобный интерфейс для работы с IP-ядрами, созданными в среде Vivado, через использование библиотек и драйверов высокого уровня.

Программная часть проекта выполнялась на процессоре ARM, работающем под управлением операционной системы PYNQ Linux. Управление аппаратными IP-блоками осуществлялось через Python API, что позволило оперативно тестировать и модифицировать аппаратные модули без необходимости перекомпиляции всего проекта. Благодаря этому подходу существенно ускорился процесс отладки и тестирования системы.

6.2 Тестирование разработанного IP-блока нейронной сети

Тестирование осуществлялось посредством запуска специально разработанного скрипта на языке Python в среде PYNQ.

Основной задачей тестирования являлась проверка корректности выполнения операций нейронной сети, реализованных в аппаратной логике, и взаимодействия IP-блока с процессорной системой через интерфейсы AXI4-lite и uP. Для этого использовался скрипт на Python, который выполнял следующие функции:

- 1 Инициализация IP-блока нейронной сети.
- 2 Загрузка тестовых данных во входные регистры IP-блока.
- 3 Запуск аппаратной обработки данных.
- 4 Считывание результатов вычислений из выходных регистров.

Также отдельно осуществляется сравнение полученных результатов с эталонными значениями для верификации корректности работы.

На этапе инициализации скрипт с помощью предоставленных библиотек PYNQ, таких как Overlay, загружает конфигурацию программируемой логики и получает доступ к IP-блоку.

Для подачи входных данных использовался архив с тестовыми изображениями из базы MNIST. Осуществляется чтение изображения и значения. Затем изображение нормализуется в соответствии с описанными ранее требованиями. Скрипт производит запись данных во входные регистры через AXI-интерфейс и инициировал начало вычислений путём установки управляющих флагов.

По завершении обработки скрипт ожидает сигнал готовности от IP-блока. После этого результаты обработки считывались из выходных регистров. Полученные данные сравнивались со считанным значением из базы.

Скрипт по обработке изображения на ПЛИС представлен в приложении Д. На рисунке 6.1 представлен пример работы данного скрипта.

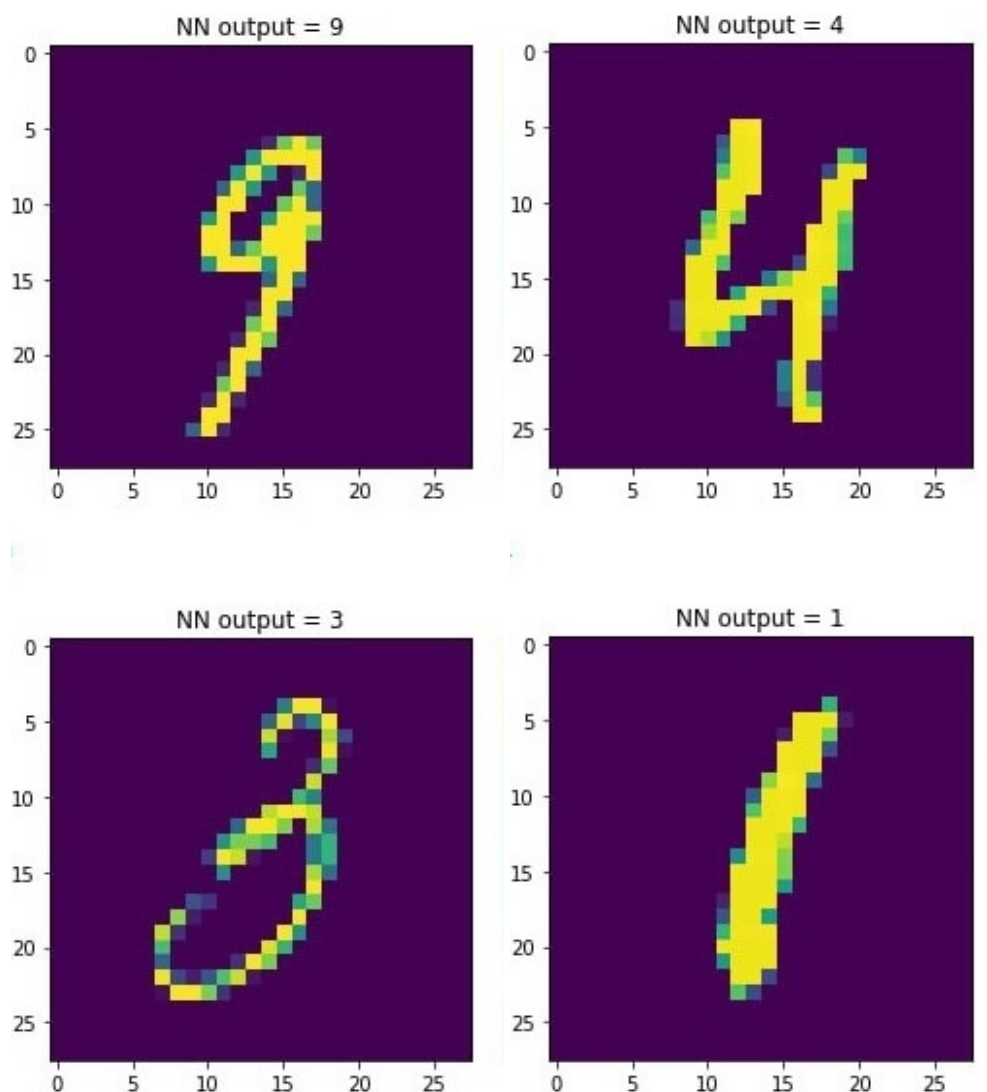


Рисунок 6.1 – Результаты распознавания рукописных цифр

6.3 Экспериментальное исследование IP-блока нейронной сети

Для оценки эффективности работы разработанного IP-блока нейронной сети было проведено три последовательных эксперимента. Эксперименты на-

правлены на проверку корректности работы системы, оценку точности классификации и исследование влияния разрядности весовых коэффициентов на качество работы сети. Произведено три эксперимента:

1 Тестирование на 10000 изображений из MNIST. Построение матрицы спутывания и сравнение с эталоном.

2 Использование консольного приложения по рисованию на холсте с возможностью сохранить изображение в формате 28 на 28 и дальнейшей обработке на ПЛИС.

3 Изменение разрядности представления весовых коэффициентов и проверка зависимости точности от разрядности.

Скрипт представлен в приложении Д.

6.3.1 Первым этапом тестирования стала проверка IP-блока на большом количестве стандартных данных. Для этого использовалась тестовая выборка из 10000 изображений базы данных MNIST, содержащей изображения рукописных цифр размером 28×28 пикселей.

Каждое изображение подавалось на вход разработанному IP-блоку, после чего считывался результат классификации. На основании полученных ответов была построена матрица спутывания, отражающая количество верных и ошибочных классификаций для каждого класса.

Матрица спутывания позволяет оценить, какие именно классы наиболее часто путаются между собой, а также вычислить стандартные метрики качества классификации, такие как точность (accuracy), полнота (recall) и специфичность (specificity).

Сравнение результатов работы IP-блока с эталонной программной моделью нейронной сети показало полное соответствие, что свидетельствует о правильности аппаратной реализации.

Если проанализировать результаты классификации при реализации модели в фиксированной точности и с плавающей запятой, можно отметить, что точность распознавания снизилась с 98.02% до 97.9%. Это соответствует потере базовой точности на 0.12%.

Данное снижение связано с квантованием весовых коэффициентов, которое приводит к небольшому искажению параметров модели. Однако потеря точности оказалась незначительной, что свидетельствует о высокой устойчивости предложенной архитектуры к операциям квантования.

Результаты представлены в графическом материале ГУИР 431289.007 ПЛ.

6.3.2 Вторым этапом исследования стало тестирование на изображениях цифр, созданных вручную в специально разработанном консольном приложении для рисования. Программа позволяла пользователю нарисовать цифру на виртуальном холсте, сохранить изображение в формате 28×28 пикселей.

Затем отдельно запускается скрипт по обработке изображения на ПЛИС.

Данный эксперимент позволил оценить устойчивость работы IP-блока к вариациям входных данных, отличающихся от стандартных изображений MNIST.

Отрисовано 100 различных изображений и проверена работа данного IP-блока с построением матрицы спутывания и вычислением точности.

Результаты представлены в графическом материале ГУИР 431289.007 ПЛ.

6.3.3 Заключительный эксперимент был направлен на изучение влияния разрядности представления весовых коэффициентов на качество классификации.

Для этого были проведены тестирования с различными значениями битности представления весов, такими как 16 бит, 12 бит, 8 бит и т.д.

На каждом этапе сети применялись квантизованные веса с соответствующей разрядностью. Далее выполнялось тестирование на стандартной выборке MNIST, с построением матриц спутывания и вычислением точности классификации.

Анализ результатов показал, что при снижении разрядности до 8 бит потери точности классификации остаются незначительными, что подтверждает эффективность использования фиксированной арифметики для оптимизации аппаратной реализации. Однако дальнейшее уменьшение разрядности до 6 бит приводило к заметному ухудшению качества распознавания.

Результаты представлены в графическом материале ГУИР 431289.007 ПЛ.

6.4 Сравнение с аналогичными архитектурами

Для оценки эффективности предложенного IP-блока нейронной сети на базе слоя LST было проведено сравнение с рядом существующих решений, предназначенных для задачи классификации изображений рукописных цифр из базы MNIST.

Таблица 4 – Сравнение архитектур нейронных сетей для задачи классификации рукописных цифр из базы MNIST

Автор	Архитектура нейронной сети	Число параметров	Точность
Liang, et al. (2018)[19]	784-2048-2048-10	10 100 000	98.32%
Umuroglu, et al. (2017)[20]	784-1024-1024-10	1 863 690	98.40%
Medus, et al. (2019)[21]	784-600-600-10	891 610	98.63%
Huynh[22]	784-126-126-10	115 920	98.16%
Huynh[22]	784-40-40-40-10	34 960	97.20%
Westby, et al.[23]	784-12-10	9 550	93.25%
LST-1	LST-784-10	9474	97.9%

Из таблицы видно, что большинство существующих архитектур нейронных сетей, ориентированных на высокую точность распознавания MNIST, характеризуются большим числом обучаемых параметров, что приводит к увеличению затрат ресурсов при аппаратной реализации. Например, нейронная сеть Liang et al.[19] имеет более 10 миллионов параметров, а сеть Umuroglu et al.[20] – порядка 1.8 миллиона параметров.

Предложенная архитектура LST-1 отличается принципиально меньшим числом параметров – всего 9474, что делает её крайне привлекательной для реализации в ресурсно-ограниченных средах, таких как ПЛИС или встроенные системы. При этом достигается высокая точность распознавания — 97.9%, что сравнимо с более крупными архитектурами.

Таким образом, аппаратная реализация нейронной сети на базе слоя LST демонстрирует компромисс между компактностью модели и точностью классификации. Это позволяет использовать её в задачах, где критичны малое энергопотребление, быстроедействие и ограниченный объем ресурсов.

Предложенный IP-блок LST может рассматриваться как альтернатива классическим полносвязным слоям при разработке нейронных сетей прямого распространения (многослойных перцептронов, MLP), особенно в случае, если требуется минимизация площади кристалла или энергопотребления в встраиваемых устройствах.

7 ТЕХНИКО–ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ IP–БЛОКА НЕЙРОННОЙ СЕТИ ДЛЯ РАСПОЗНАВАНИЯ РУКОПИСНЫХ ЦИФР

7.1 Характеристика программного средства, разрабатываемого для собственных нужд

Разрабатываемый IP-блок нейронной сети предназначен для аппаратного ускорения процесса распознавания рукописных цифр в системах обработки изображений. Основная цель разработки — создание эффективного и оптимизированного аппаратного решения, интегрируемого в FPGA-платформы, для высокоскоростного распознавания рукописных символов с целью использования в автоматизированных системах ввода рукописного текста или распознавание символов в банковских и почтовых системах. Основные задачи, решаемые IP-блоком заключаются в оптимизация вычислений за счёт аппаратной реализации и использовании специального LST преобразования.

Разработчиком IP-блока является сотрудник IT-отдела организации. Разработка ведётся в рамках внутренних потребностей компании для последующего внедрения в производственные и исследовательские процессы.

Необходимость в разработке IP-блока обусловлена высокой потребностью в аппаратных решениях для распознавания рукописных цифр. Встроенные программные решения не обеспечивают требуемую производительность. Разрабатываемое средство оптимизировать использование вычислительных ресурсов. Внедрение IP-блока позволит организации значительно повысить эффективность обработки изображений, что дает возможность адаптации IP-блока под специфические требования организации.

7.2 Расчет инвестиций в разработку программного средства для собственных нужд

Расчет затрат на основную заработную плату представлен в таблице 2.

Таблица 2 – Расчет затрат на основную заработную плату команды разработчиков

Категория исполнителя	Месячный оклад, р.	Часовой оклад, р.	Трудоемкость работ, ч.	Итого, р.
Бизнес-аналитик	1600	10	40	400
Системный архитектор	4000	25	40	1000
Программист	2400	15	40	600
Тестировщик	1600	10	40	400
Дизайнер	1600	10	40	400
Итого				2800
Премия и иные стимулирующие выплаты (50%)				1400
Всего затрат на основную заработную плату разработчиков				4200

Расчет инвестиций на разработку программного средства для собственных нужд представлен в таблице 3.

Таблица 3 – Расчет инвестиций на разработку программного средства для собственных нужд

Наименование статьи затрат	Формула/таблица для расчета	Сумма, р.
1. Основная заработная плата разработчиков	Таблица 3	4200
2. Дополнительная заработная плата разработчиков	$З_{д} = \frac{4200 \cdot 10\%}{100}$	420
3. Отчисления на социальные нужды	$P_{соц} = \frac{4200 + 420 \cdot 34,6\%}{100}$	1598,52
4. Прочие расходы	$P_{пр} = \frac{4200 \cdot 30\%}{100}$	1260
5. Общая сумма инвестиций на разработку	$З_p = 4200 + 420 + 1598,52 + 1260$	7478,52

7.3 Расчет экономического эффекта от использования программного средства для собственных нужд

Экономия на заработной плате и начислениях на заработную плату осуществляется в результате сокращения численности работников. В частности после внедрения программного средства сокращению подвергается 1 дизайнер

$$\begin{aligned} \Theta_{з.п.} = \sum_{i=1}^n \Delta Ч_i \cdot З_i \cdot \left(1 + \frac{10}{100}\right) \cdot \left(1 + \frac{34,6}{100}\right) &= \sum_{i=1}^1 \Delta Ч_i \cdot З_i \cdot \left(1 + \frac{H_{д}}{100}\right) \times \\ &\times \left(1 + \frac{H_{соц}}{100}\right) = 1 \cdot 19200 \cdot 1.48 = 28427.52 \text{ р.} \end{aligned} \quad (7.1)$$

где H – норматив дополнительной заработной платы;

$H_{соц}$ – ставка отчисления от заработной платы, включаемых в себестоимость.

Экономическим эффектом при использовании программного средства является прирост чистой прибыли

$$\begin{aligned} \Delta П_{ч} = (\Theta_{тек} - \Delta З_{тек}^{п.с}) \cdot \left(1 - \frac{H_{п}}{100}\right) &= (28427,52 - 15500) \cdot \left(1 - \frac{20\%}{100}\right) = \\ &= 10342,02 \text{ р.} \end{aligned} \quad (7.2)$$

где $\Delta П_{п}$ – ставка налога на прибыль согласно действующему законодательству.

7.4 Расчет показателей экономической эффективности разработки и использования программного средства в организации

Так как разработка программного средства ведется «с нуля», то расчет показателей экономической эффективности осуществляется следующим образом

$$ROI = \frac{\Delta П_{ч} - З_p}{З_p} \cdot 100\% = \frac{10342,02 - 7478,52}{7478,52} \cdot 100\% = 38\%, \quad (7.3)$$

где $\Delta\P_{\text{ч}}$ – прирост чистой прибыли, полученной от использования разработанного программного средства;

Z_p – затраты на разработку программного средства.

На основании полученных значений показателей эффективности инвестиций (затрат) следует сделать вывод, что разработка IP-блока нейронной сети для аппаратного ускорения распознавания рукописных цифр является экономически целесообразной. Проведенные расчеты показывают, что общие затраты на разработку составляют 7478,52 руб.

Экономический эффект от внедрения программного средства достигается за счет сокращения численности персонала (исключение 1 дизайнера), что позволяет сэкономить 28427,52 руб. на заработной плате и социальных начислениях. Прирост чистой прибыли в результате внедрения составляет 10342,02 руб.

Рассчитанный показатель ROI (рентабельность инвестиций) составляет 38%, что говорит о высокой эффективности вложений в разработку данного IP-блока. Это означает, что вложенные средства окупаются с существенной прибылью, а дальнейшее использование программного средства приведет к дополнительной экономии и росту прибыли организации.

ЗАКЛЮЧЕНИЕ

В рамках данной дипломной работы была спроектирована, разработана и протестирована аппаратная реализация нейронной сети на базе программируемой логической интегральной схемы ZYBO Z7 с использованием платформы PYNQ.

На первом этапе работы была произведена формализация структуры нейронной сети в виде управляющего и операционного автоматов, что позволило эффективно спроектировать архитектуру IP-блока. Для обеспечения взаимодействия между процессорной системой и аппаратной логикой применялась среда PYNQ, обеспечивающая удобную интеграцию аппаратных модулей и программного управления.

Был разработан специализированный IP-блок, реализующий основные функции нейронной сети, включая обработку входных данных, вычисление выходов нейронов и применение функций активации. Особое внимание было уделено оптимизации аппаратной реализации: использовалась фиксированная точность представления данных для уменьшения занимаемых ресурсов без существенного ухудшения качества классификации.

Разработанная система прошла всестороннее тестирование, включающее:

- тестирование на стандартном наборе данных MNIST с построением матрицы спутывания и сравнением с эталонной моделью.

- тестирование на изображениях, созданных вручную, с целью оценки устойчивости работы.

- исследование влияния разрядности весовых коэффициентов на точность классификации.

Результаты экспериментов показали высокую точность классификации, сравнимую с полносвязными архитектурами нейронной сети, при значительно меньших затратах вычислительных ресурсов благодаря LST преобразованию.

Таким образом, поставленные цели дипломной работы были достигнуты, а все задачи успешно решены. Разработанный IP-блок нейронной сети может быть использован в составе встроенных систем реального времени, требующих высокой скорости обработки данных при ограниченных аппаратных ресурсах.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] FPGA реализация нейронной сети прямого распространения для распознавания рукописных чисел / Е. А. Кривальцевич, М. И. Вашкевич // Информационные технологии и системы 2024 (ИТС 2024) = Information Technologies and Systems 2024 (ITS 2024) : материалы международной научной конференции, Минск, 20 ноября 2024 г. / Белорусский государственный университет информатики и радиоэлектроники; редкол. : Л. Ю. Шилин [и др.]. – Минск, 2024. – С. 85–86.
- [2] Кривальцевич Е.А., Вашкевич М.И. Аппаратная реализация нейронной сети прямого распространения для распознавания рукописных цифр на базе FPGA. Доклады БГУИР. 20**; **(*): ***-***.
- [3] Bouvrie J. Notes on convolutional neural networks. – 2006.
- [4] Giardino D. et al. FPGA implementation of hand-written number recognition based on CNN // International Journal on Advanced Science, Engineering and Information Technology. – 2019. – Т. 9. – No. 1. – P. 167-171.
- [5] Varsamopoulos, S. (2019). Neural Network based Decoders for the Surface Code. [Dissertation (TU Delft), Delft University of Technology]. <https://doi.org/10.4233/uuid:dc73e1ff-0496-459a-986f-de37f7f250c9>
- [6] Agrawal V., Jagtap J., Kantipudi M. V. V. P. Decoded-ViT: A Vision Transformer Framework for Handwritten Digit String Recognition //Revue d'Intelligence Artificielle. – 2024. – Т. 38. – №. 2. – С. 523.
- [7] Agrawal V. et al. Performance analysis of hybrid deep learning framework using a vision transformer and convolutional neural network for handwritten digit recognition //MethodsX. – 2024. – Т. 12. – С. 102554.
- [8] Maxim Vashkevich and Egor Krivalcevic Learned 2D separable transform: building block for designing compact and efficient neural network for image recognition.
- [9] Digilent Inc., "ZYBO Z7 Reference Manual,"[Электронный ресурс]. — Режим доступа: <https://digilent.com/reference/programmable-logic/zybo-z7/reference-manual>
- [10] MNIST Handwritten Numbers database [Electronic resource] – Electronic data – Access mode: <https://yann.lecun.com/exdb/mnist/Mittal>
- [11] Xilinx, AXI Reference Guide [Электронный ресурс] – Электронные данные – Режим доступа: <https://docs.amd.com/v/u/en-US/ug1037-vivado-axi-reference-guide>, 2017.
- [12] Pong P. Chu, FPGA Prototyping by Verilog Examples [Электронный ресурс] – Электронные данные – Режим доступа: <https://faculty.kfupm.edu.sa/COE/aimane/coe405/FPGA%20examples.pdf>, 2018.
- [13] Tokheim, R.L. Digital Electronics: Principles and Applications. – McGraw-Hill, 2003. – 480 p.
- [14] Katz, R.H. Contemporary Logic Design. – Benjamin-Cummings, 1994. – 700 p.
- [15] Xilinx Inc., *Vivado Design Suite User Guide*, UG892, 2021.
- [16] L. D. Medus, T. Iakymchuk, J. V. Frances-Villora, M. Bataller- Mompean, and A. Rosado-Munoz, "A novel systolic parallel hardware architecture for the FPGA acceleration of feedforward neural networks," IEEE Access, vol. 7, pp. 76 084–76 103, 2019.

- [17] Xilinx, "Zynq-7000 SoC Technical Reference Manual," [Электронный ресурс]. — Режим доступа: https://www.xilinx.com/support/documentation/user_guides/ug585-Zynq-7000-TRM.pdf
- [18] Xilinx, "PYNQ: Python Productivity for Zynq," [Электронный ресурс]. — Режим доступа: <http://www.pynq.io>
- [19] S. Liang, S. Yin, L. Liu, W. Luk, and S. Wei, "FP-BNN: Binarized neural network on FPGA," *Neurocomputing*, vol. 275, pp. 1072–1086, 2018
- [20] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, "FINN: A framework for fast, scalable binarized neural network inference," in *Proceedings of the 2017 ACM/SIGDA international symposium on field-programmable gate arrays*, 2017, pp. 65–74.
- [21] L. D. Medus, T. Iakymchuk, J. V. Frances-Villora, M. Bataller-Mompean, and A. Rosado-Munoz, "A novel systolic parallel hardware architecture for the FPGA acceleration of feedforward neural networks," *IEEE Access*, vol. 7, pp. 76 084–76 103, 2019.
- [22] T. V. Huynh, "Deep neural network accelerator based on FPGA," in *4th NAFOSTED Conference on Information and Computer Science*, 2017, pp. 254–257.
- [23] I. Westby, et al. "FPGA acceleration on a multilayer perceptron neural network for digit recognition," *Journal of Supercomputing*, 2021, vol. 77, no. 12, pp. 356–373.

ПРИЛОЖЕНИЕ А
(обязательное)
Отчет о проверке на заимствование

ПРИЛОЖЕНИЕ Б

(обязательное)

Python описание обучения сети

Python описание обучения сети:

```
import numpy as np
import torch
import torch.nn as nn
from torch.optim import lr_scheduler

import torchvision
import matplotlib.pyplot as plt
from time import time
from torchvision import datasets, transforms
from torch import nn, optim
from tqdm import tqdm

device = torch.device('cuda' if torch.cuda.is_available()
                       else 'cpu')
print(f"Working on device {device}")

# Transform image to 1x784 and normalize colors
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Download dataset
trainset_full = datasets.MNIST('.././../Datasets', download=
    False, train=True, transform=transform)
testset = datasets.MNIST('.././../Datasets', download=False,
    train=False, transform=transform)

val_size, train_size = 1000, 59000
val_dataset, train_dataset = torch.utils.data.random_split(
    trainset_full, [val_size, train_size], generator=torch.
    Generator().manual_seed(27))

# Create dataloaders
trainloader = torch.utils.data.DataLoader(train_dataset,
    batch_size=1000, shuffle=True)
val_loader = torch.utils.data.DataLoader(val_dataset,
    batch_size=1000, shuffle=False)
test_loader = torch.utils.data.DataLoader(testset, batch_size
    =1000, shuffle=False)

class L2DST(nn.Module):
    """The positionwise feed-forward network."""
    def __init__(self, ffn_num_hiddens, ffn_num_outputs):
        super().__init__()
        self.dense1 = nn.LazyLinear(ffn_num_hiddens)
        self.dense2 = nn.LazyLinear(ffn_num_outputs)

    def forward(self, X):
        out = torch.nn.functional.tanh(self.dense1(X))
```



```

        out1 = torch.transpose(out, -1, -2)
        out = torch.nn.functional.tanh(self.dense2(out1))
        out = torch.transpose(out, -2, -1)
        return out

    def get_embeddings(self,X):
        out = torch.nn.functional.tanh(self.dense1(X))
        e1 = out
        out = torch.transpose(e1, -1, -2)
        out = torch.nn.functional.tanh(self.dense2(out))
        e2 = out
        out = torch.transpose(e2, -2, -1)

        return e1,e2

class L2DST_1l(nn.Module):
    def __init__(self, input_size, num_classes, device='cpu')
    :
        super(L2DST_1l, self).__init__()

        self.L2DST = L2DST(input_size,input_size)

        dropout = 0.05
        self.dropout = nn.Dropout(dropout)

        # self.BN1 = nn.LayerNorm((1,input_size,input_size))

        self.W_o = nn.Linear(input_size*input_size,
                               num_classes, device=device)

        self.num_classes = num_classes

        self.log_softmax = nn.LogSoftmax(dim=1)

        nn.init.xavier_uniform_(self.W_o.weight)

    def forward(self, x):
        # Multiply input by weights and add biases

        out = self.L2DST(x)

        out = out.reshape(out.shape[0],-1)

        out = self.log_softmax(self.dropout(self.W_o(out)))

        return out

    def get_embeddings(self,x):
        e1,e2 = self.L2DST.get_embeddings(x)

        return e1,e2

# Build the Neural Network
input_size = 28 # 28x28 images flattened
output_size = 10 # 10 classes for digits 0-9

N_epoch = 300
model = L2DST_1l(input_size,output_size, device=device)

```

```

model.load_state_dict(torch.load(f'model_backup\
    L2DST_1l_epoch_{N_epoch}.pth', map_location=torch.device('
    cpu')))) #
# print(model)
model.to(device)

# criterion = nn.CrossEntropyLoss() # Use CrossEntropyLoss
# which includes softmax
criterion = nn.NLLLoss() # Use CrossEntropyLoss which
# includes softmax
optimizer = optim.Adam(model.parameters(), lr=15e-4,
    weight_decay=0e-5)
scheduler = lr_scheduler.StepLR(optimizer, step_size=10,
    gamma=0.97, verbose=False)

# Track loss
loss_list = []
val_loss_list = []

# Training the network
epochs = 70
time0 = time()

for epoch in range(epochs):
    running_loss = 0

    # for images, labels in tqdm(trainloader, leave=False):
    for images, labels in trainloader:
        images = images.to(device)
        labels = labels.to(device)

        images = images.squeeze(1)

        # Training pass
        optimizer.zero_grad()

        output = model(images)
        loss = criterion(output, labels)

        # This is where the model learns by back propagating
        loss.backward()

        # And optimizes its weights here
        optimizer.step()
        scheduler.step()

        running_loss += loss.item()

    # validation
    model.eval()
    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device)
            images = images.squeeze(1)

            output = model(images)
            val_loss = criterion(output, labels)
        val_loss = val_loss / len(val_loader)

```

```

    val_loss_list.append(val_loss)

    CE_curr = running_loss / len(trainloader)
    loss_list.append(CE_curr)
    # if (epoch%10)==0:
    print(f"Epoch_{epoch}-_Training_loss:_{CE_curr:.5f},_Val
          _loss:_{val_loss:.5f}")

print(f"\nTraining_Time_(in_minutes)_{(time()-time0)/60:.2f
      }")

# Convert lists to numpy arrays
loss_array = np.array(loss_list)

val_loss = [val.cpu().numpy() for val in val_loss_list]

val_loss_array = np.array(val_loss)

# Save the model
torch.save(model.state_dict(), f'model_backup\L2DST_1l_epoch_
    {N_epoch+epochs}.pth')

# Plot NLL_loss
plt.figure(figsize=(5, 2))
plt.plot(range(len(loss_array))[1:], loss_array[1:], label='
    Train')
plt.plot(range(len(val_loss_array)), val_loss_array, label='
    Validation')

plt.xlabel('Epochs')
plt.ylabel('NNLoss')
plt.title('NNLoss')
plt.legend()
plt.show()

```

ПРИЛОЖЕНИЕ В

(обязательное)

Python описание эталонной сети

Python описание сети в fixpoint:

```
import numpy as np
import torch
import os
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
from fixedpoint import FixedPoint
import math

from LST_lib import LST_2d_one

input_size = 28
output_size = 10
model = LST_2d_one(input_size, output_size)

N_epoch = 300
model.load_state_dict(torch.load(f'model_backup/
    L2DST_1l_epoch_{N_epoch}.pth')) #, map_location=device

weights = model.state_dict()

def min_max_show(A, key):
    print(f'{key}:_size_= ', A[key].shape)
    print(f'{key}:_max_value_= ', torch.max(A[key]))
    print(f'{key}:_min_value_= ', torch.min(A[key]))

# Weights and biases analysis
min_max_show(weights, 'L2DST.dense1.weight')
min_max_show(weights, 'L2DST.dense1.bias')
min_max_show(weights, 'L2DST.dense2.weight')
min_max_show(weights, 'L2DST.dense2.bias')
min_max_show(weights, 'W_o.weight')
min_max_show(weights, 'W_o.bias')

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

valset = datasets.MNIST('c:/Docs/GitHub/Datasets/', download=
    False, train=False, transform=transform)

img, label = valset[4]
img_ = img.view(img.size(0), -1).squeeze().numpy()

fig = plt.figure(figsize=(1,1))
plt.imshow(img_.reshape(28,-1))
# plt.colorbar()

e1, e2 = model.get_embeddings(img)

e1 = e1.squeeze(0)
e2 = e2.squeeze(0)
```

```

class MAC():
    def __init__(self, int_bits, frac_bits):
        self.int_bits = int_bits
        self.frac_bits = frac_bits
        self.acc = FixedPoint(0, signed=True, m=self.int_bits
                               , n=self.frac_bits, rounding="down")

    def initialization(self, value):
        self.acc = FixedPoint(value, signed=True, m=self.
                               int_bits, n=self.frac_bits, rounding="down")

    def run(self, a, b, q_bit_to_remove):
        p = a*b
        p = FixedPoint(float(p), signed=True, m=self.int_bits
                        , n= (p.n - q_bit_to_remove), rounding="down")
        self.acc = self.acc + p
        self.acc = FixedPoint(float(self.acc), signed=True, m
                               =self.int_bits, n = self.acc.n, rounding="down")

class MEM():
    def __init__(self, size, int_bits, frac_bits):
        self.int_bits = int_bits
        self.frac_bits = frac_bits
        self.size = size
        self.mem = [] # FixedPoint(np.zeros((size)), signed=
                        True, m=self.int_bits, n=self.frac_bits)

    def initialization(self, array):
        for addr in range(len(array)):
            self.mem.append(FixedPoint(array[addr], signed=
                                        True, m=self.int_bits, n = self.frac_bits))

    def write_value(self, value, addr):
        self.mem[addr] = FixedPoint(float(value), signed=True
                                     , m=self.int_bits, n = self.frac_bits)

    def numpy(self):
        np_arr = np.zeros((self.size))
        for addr in range(self.size):
            np_arr[addr] = float(self.mem[addr])

        return np_arr

class tanh():
    def __init__(self, int_bits, frac_bits):
        self.int_bits = int_bits
        self.frac_bits = frac_bits
        self.const_q_plus_one = FixedPoint(1-2**(-frac_bits)
                                             , signed=True, m=int_bits, n=frac_bits, str_base
                                             =16, rounding="down")
        self.const_q_minus_one = FixedPoint(-1, signed=True,
                                              m=int_bits, n=frac_bits, str_base=16, rounding="
                                              down")

    def calc(self, x):
        x_fp = FixedPoint(float(x), signed=True, m=self.
                           int_bits, n = self.frac_bits, rounding="down")
        x_quater = x_fp >> 2

```

```

        if (x_fp>=2):
            mul_out = FixedPoint((1), signed=True, m=self.
                int_bits, n = self.frac_bits, str_base=16)
        elif (x<=-2):
            mul_out = FixedPoint((-1), signed=True, m=self.
                int_bits, n = self.frac_bits, str_base=16)
        else:
            if (x>0):
                add_sub_out = self.const_q_plus_one -
                    x_quater
            else:
                add_sub_out = self.const_q_plus_one +
                    x_quater
            mul_out = (add_sub_out*x_fp)
            mul_out.m = self.int_bits
            mul_out.n = self.frac_bits
            mul_out = FixedPoint(float(mul_out), signed=True,
                m=self.int_bits, n = self.frac_bits, rounding=
                "down")
        return mul_out

def addr_width(x):
    return math.ceil(math.log(x, 2))

# number of MAC-cores
N = 28
L = 10 # output layer

# Bit representation
int_bits, frac_bits = 6,7

# Creating MAC cores
MAC_array = []
for i in range(N):
    MAC_array.append(MAC(int_bits, frac_bits))

# Creating row-weights memory blocks
ROW_memory = []
for i in range(N):
    array_i = weights['L2DST.dense1.weight'][i].numpy()
    ROW_memory.append(MEM(len(array_i), int_bits, frac_bits))
    ROW_memory[i].initialization(array_i)

# Creating memory blocks for ROW biases
row_biases_array = weights['L2DST.dense1.bias']
ROW_biases = MEM(len(row_biases_array), int_bits, frac_bits)
ROW_biases.initialization(row_biases_array)

COL_memory = []
for i in range(N):
    array_i = weights['L2DST.dense2.weight'][i].numpy()
    COL_memory.append(MEM(len(array_i), int_bits, frac_bits))
    COL_memory[i].initialization(array_i)

# Creating memory blocks for COL biases
col_biases_array = weights['L2DST.dense2.bias']
COL_biases = MEM(len(col_biases_array), int_bits, frac_bits)
COL_biases.initialization(col_biases_array)

```

```

memory = []
for i in range(10):
    array_i = weights['W_o.weight'][i].numpy()
    memory.append(MEM(len(array_i), int_bits, frac_bits))
    memory[i].initialization(array_i)

# Creating memory blocks for COL biases
biases_array = weights['W_o.bias']
biases = MEM(len(biases_array), int_bits, frac_bits)
biases.initialization(biases_array)

# Creating internal RAM block for image storage
RAM_bock = MEM(N*N, int_bits, frac_bits)
RAM_bock.initialization(np.zeros(N*N))

# Createing tanh function module
Tanh = tanh(int_bits, frac_bits)

# Loading image to RAM block (stage 1)
for addr in range(N*N):
    RAM_bock.write_value(float(img_[addr]), addr)

# Row processing (stage 2)
for row in range(N):
    # MAC-core initialization
    for i in range(N):
        MAC_array[i].initialization(float(ROW_biases.mem[i]))

    for col in range(N):
        # Read data from memory
        in_data = RAM_bock.mem[row*N + col]

        for i in range(N):
            MAC_array[i].run(in_data, ROW_memory[i].mem[col],
                             q_bit_to_remove=frac_bits)

        # Write result to memory
        for col in range(N):
            tmp = Tanh.calc(MAC_array[col].acc)
            RAM_bock.write_value(float(tmp), row*N + col)

# Copy for visualization
FC1_out = RAM_bock.numpy()

# Column processing (stage 3)
for col in range(N):
    # MAC-core initialization
    for i in range(N):
        MAC_array[i].initialization(float(COL_biases.mem[i]))

    for row in range(N):
        # Read data from memory
        in_data = RAM_bock.mem[row*N + col]

        for i in range(N):
            MAC_array[i].run(in_data, COL_memory[i].mem[row],
                             q_bit_to_remove=frac_bits)

    # Write result to memory

```

```

        for row in range(N):
            tmp = Tanh.calc(MAC_array[row].acc)
            RAM_bock.write_value(float(tmp), row*N + col)

# Copy for visualization
FC2_out = RAM_bock.numpy()

# Classification (stage 4)
final_result = [] # Softmax output

# Run MAC-cores
for i in range(L):
    # MAC-core initialization
    MAC_array[i].initialization(float(biases.mem[i]))

    # 784 MAC iteration
    for j in range(N * N):
        MAC_array[i].run(RAM_bock.mem[j] , memory[i].mem[j],
            q_bit_to_remove=frac_bits)

    # save FC output
    final_result.append(float(MAC_array[i].acc))

# Convert to Numpy
final_result = np.array(final_result)

# Output FC layer
print("Softmax_input:", final_result)

# Result image
predicted_label = np.argmax(final_result)
print("Predicted_label:", predicted_label)

FC2_out_ = FC2_out.reshape(N,-1)
FC1_out_ = FC1_out.reshape(N,-1)

fig = plt.figure(figsize=(8,4))
plt.subplot(2,3,1)
plt.imshow(e1.detach().numpy())
plt.title('Float_precision')
plt.colorbar()
plt.subplot(2,3,2)
plt.imshow(FC1_out_)
plt.title('Fixed_point')
plt.colorbar()
plt.subplot(2,3,3)
plt.imshow(e1.detach().numpy() - FC1_out_) # , vmax=1, vmin
    =-1
plt.title('difference')
plt.colorbar()

plt.subplot(2,3,4)
plt.imshow(e2.detach().numpy())
plt.title('Float_precision')
plt.colorbar()
plt.subplot(2,3,5)
plt.imshow(FC2_out_)
plt.title('Fixed_point')
plt.colorbar()

```



```
plt.subplot(2,3,6)
plt.imshow(e2.detach().numpy() - FC2_out_) # , vmax=1, vmin
      =-1
plt.title('difference')
plt.colorbar()

plt.subplots_adjust(wspace=0.4, hspace=0.4)
```

ПРИЛОЖЕНИЕ Г
(обязательное)
Verilog описание устройства

Модуль с параметрами:

```
package nn_param_pkg;

    localparam INT_BITS    = 6;
    localparam FRC_BITS    = 7;
    localparam BITS        = 13;

    localparam PICT_SIZE   = 28;
    localparam WIDTH       = 784;

    localparam ADDR_RC     = 5;
    localparam ADDR_OUT    = 10;

    localparam RCO_TYPE    = 2;

    localparam NN_FSM_BUS_WIDTH = 5;

endpackage
```

Модуль верхнего уровня:

```
(* dont_touch = "yes" *) (* keep_hierarchy = "yes" *)

module fc
    import nn_param_pkg::*;
    (
        input  logic          clk,
        input  logic          rst_n,
        input  logic          start_i,
        input  logic [ADDR_OUT-1:0] x_i,
        input  logic [INT_BITS+FRC_BITS-1:0] din_i,
        output logic [INT_BITS-1:0] num_o,
        output logic          rdy_o
    );

    logic [INT_BITS + FRC_BITS-1:0] memory          [PICT_SIZE-1:0][PICT_SIZE-1:0];
    logic [INT_BITS + FRC_BITS-1:0] internal_rc      [PICT_SIZE-1:0];
    logic [INT_BITS + FRC_BITS-1:0] internal_tanh    [PICT_SIZE-1:0];
    logic [INT_BITS + FRC_BITS-1:0] result          [ ADDR_OUT-1:0];

    logic sel_data;
    logic init;
    logic en;
    logic mem_en;
    logic tg_en;

    (* dont_touch = "yes" *) logic [ ADDR_RC-1:0] fc_addr;
    (* dont_touch = "yes" *) logic [ ADDR_RC-1:0] str_cnt;
```

```

(* dont_touch = "yes" *) logic [RCO_TYPE-1:0] rco_sel;
// selects row, column or w_out memory bloc
logic [INT_BITS + FRC_BITS-1:0] tang;

logic [INT_BITS + FRC_BITS-1:0] din;
logic [INT_BITS + FRC_BITS-1:0] dout;
logic we;
logic m_en;
(* dont_touch = "yes" *) logic [ADDR_OUT-1:0] addr;

img_mem img_mem_inst(
    .clk(clk),
    .we(we),
    .en(m_en),
    .addr(addr),
    .din(din),
    .dout(dout)
);

assign we = (sel_data)? 1 :
            | (tg_en & rco_sel == 0)? 1 :
            | (tg_en & rco_sel == 1)? 1 : 0
            ;
assign addr = ( sel_data)? x_i :
              | (!sel_data & tg_en & rco_sel == 0)? PICT_SIZE*
                str_cnt + fc_addr :
              | (!sel_data & !tg_en & rco_sel == 0)? PICT_SIZE*
                str_cnt + fc_addr-1 :
              | (!sel_data & tg_en & rco_sel == 1)? PICT_SIZE*
                fc_addr + str_cnt :
              | (!sel_data & !tg_en & rco_sel == 1)? PICT_SIZE
                *(fc_addr-1) + str_cnt :
              | (!sel_data & !tg_en & rco_sel == 2)? PICT_SIZE*
                str_cnt + fc_addr : 0
              ;

assign din = (sel_data)? din_i :
            | (tg_en & rco_sel== 0)? tang :
            | (tg_en & rco_sel== 1)? tang : 0
            ;

(* dont_touch = "yes" *) logic [ADDR_OUT-1:0] out_addr;
logic [INT_BITS + FRC_BITS-1:0] data2PE;
assign data2PE = dout;

genvar i, j;
generate
    for (i = 0; i < 10; i = i + 1) begin : PE_rco_inst
        PE_rco#(.INT_BITS(INT_BITS), .FRC_BITS(FRC_BITS),
            .N(i)) PE_rco_0(
            .clk(clk),
            .init(init),
            .en(en),
            .address_row(fc_addr[ADDR_RC-1:0]),
            .address_col(fc_addr[ADDR_RC-1:0]),
            .address_o(out_addr),
            .rco_sel(rco_sel),
            .din(data2PE),
            .dout(internal_rc[i]));
    end

```

```

        end
    endgenerate

    generate
        for (j = 0; j < 18; j = j + 1) begin : PE_rc_inst
            PE_rc#(.INT_BITS(INT_BITS), .FRC_BITS(FRC_BITS), .N(j
                +10)) PE_rc_0(
                .clk(clk),
                .init(init),
                .en(en),
                .address_row(fc_addr[ADDR_RC-1:0]),
                .address_col(fc_addr[ADDR_RC-1:0]),
                .din(data2PE),
                .dout(internal_rc[j+10]),
                .rc_sel(rco_sel[0]));
        end
    endgenerate

    logic [INT_BITS + FRC_BITS-1:0] data_tg;

    always_comb begin
        data_tg = internal_tanh[fc_addr];
    end

    tanh_function #(.INT_SIZE(INT_BITS), .FRC_SIZE(FRC_BITS))
        tanh_inst(.X(data_tg), .Y(tang));

    always_comb begin
        if(mem_en) begin
            internal_tanh = internal_rc;
        end
    end

    assign result = internal_rc[ADDR_OUT-1:0];

    softmax #(.BITS(BITS), .HEIGHT(10)) softmax_inst(.
        result_layer(result), .predict_num(num_o));

    // -----
    // CNT
    // -----
    logic out_en;
    logic [ADDR_OUT-1:0] out_addr_next;

    always_ff @(posedge clk) begin
        if(~rst_n) begin
            out_addr = ADDR_OUT'(0);
        end else if(out_en) begin
            out_addr <= out_addr_next;
        end else begin
            out_addr <= ADDR_OUT'(0);
        end
    end

    assign out_addr_next = out_addr + ADDR_OUT'(1);

    // -----
    // CNT
    // -----

```

```

logic fc_en;
logic [ADDR_RC-1:0] fc_addr_next;

always_ff @( posedge clk ) begin
    if(~rst_n) begin
        fc_addr = ADDR_RC'(0);
    end else if(fc_en) begin
        fc_addr <= fc_addr_next;
    end else begin
        fc_addr <= ADDR_RC'(0);
    end
end

assign fc_addr_next = fc_addr + ADDR_RC'(1);

// -----
// CNT
// -----
logic str_en;
logic rst_str_cnt;
logic [ADDR_RC-1:0] str_cnt_next;

always_ff @( posedge clk ) begin
    if(~rst_n) begin
        str_cnt = ADDR_RC'(0);
    end else if(str_en) begin
        str_cnt <= str_cnt_next;
    end else begin
        if(~rst_str_cnt) begin
            str_cnt <= str_cnt;
        end else begin
            str_cnt <= ADDR_RC'(0);
        end
    end
end

assign str_cnt_next = str_cnt + ADDR_RC'(1);

// -----
// FSM
// -----

typedef enum logic [NN_FSM_BUS_WIDTH-1:0] {
    NN_INIT          = NN_FSM_BUS_WIDTH'('b00000),
    NN_LOAD_IMG      = NN_FSM_BUS_WIDTH'('b10000),
    NN_LD_MEM        = NN_FSM_BUS_WIDTH'('b10001),
    NN_INIT_CALC     = NN_FSM_BUS_WIDTH'('b10011),
    NN_STRING_CALC   = NN_FSM_BUS_WIDTH'('b10010),
    NN_STRING_RDY    = NN_FSM_BUS_WIDTH'('b10110),
    NN_TANG_STRING_CALC = NN_FSM_BUS_WIDTH'('b10111),
    NN_UPD_RCO_TYPE  = NN_FSM_BUS_WIDTH'('b10101),
    NN_INIT_OUT      = NN_FSM_BUS_WIDTH'('b11101),
    NN_OUT_CALC      = NN_FSM_BUS_WIDTH'('b11111),
    NN_OUT_RDY       = NN_FSM_BUS_WIDTH'('b11110)
} nn_fc_state_t;

nn_fc_state_t fsm_state_ff, fsm_state_next;

always_ff @(posedge clk) begin

```

```

    if(~rst_n) begin
        fsm_state_ff <= NN_INIT;
    end else begin
        fsm_state_ff <= fsm_state_next;
    end
end

logic fsm_init_tr;
logic fsm_load_img_tr;
logic fsm_ld_mem_tr;
logic fsm_init_calc_tr;
logic fsm_str_calc_tr;
logic fsm_str_rdy_tr;
logic fsm_tang_str_calc_tr;
logic fsm_upd_rco_type_tr;
logic fsm_init_out_tr;
logic fsm_out_calc_tr;
logic fsm_out_rdy_tr;

assign fsm_state_next = (fsm_init_tr          ) ? NN_INIT
                        :
                        (fsm_load_img_tr      ) ?
                          NN_LOAD_IMG         :
                        (fsm_ld_mem_tr        ) ? NN_LD_MEM
                          :
                        (fsm_init_calc_tr      ) ?
                          NN_INIT_CALC         :
                        (fsm_str_calc_tr       ) ?
                          NN_STRING_CALC       :
                        (fsm_str_rdy_tr        ) ?
                          NN_STRING_RDY        :
                        (fsm_tang_str_calc_tr  ) ?
                          NN_TANG_STRING_CALC  :
                        (fsm_upd_rco_type_tr   ) ?
                          NN_UPD_RCO_TYPE      :
                        (fsm_init_out_tr       ) ?
                          NN_INIT_OUT          :
                        (fsm_out_calc_tr       ) ?
                          NN_OUT_CALC          :
                        (fsm_out_rdy_tr        ) ? NN_OUT_RDY
                          : fsm_state_ff;

assign rdy_o = (fsm_state_ff == NN_OUT_RDY      );

// -----
logic col_en;
logic rdy_en;
logic str_tg_en;

logic rc_done;

assign fsm_init_tr      = (fsm_state_ff == NN_INIT
                          ) & ~start_i
                          | (fsm_state_ff == NN_OUT_RDY
                          );

assign fsm_load_img_tr  = (fsm_state_ff == NN_INIT
                          ) & start_i;
assign fsm_ld_mem_tr    = (fsm_state_ff == NN_LOAD_IMG

```

```

        ) & (x_i == WIDTH-1)
            | (fsm_state_ff ==
              NN_TANG_STRING_CALC ) & (
              fc_addr == PICT_SIZE-1)
            | (fsm_state_ff ==
              NN_UPD_RCO_TYPE      ) & (
              rco_sel != 2)
        ;
assign fsm_init_calc_tr      = (fsm_state_ff == NN_LD_MEM
    );
assign fsm_str_calc_tr       = (fsm_state_ff == NN_INIT_CALC
    ) & (str_cnt != PICT_SIZE);
assign fsm_str_rdy_tr        = (fsm_state_ff == NN_STRING_CALC
    ) & (fc_addr == PICT_SIZE);
assign fsm_tang_str_calc_tr  = (fsm_state_ff == NN_STRING_RDY
    );
assign fsm_upd_rco_type_tr   = (fsm_state_ff == NN_INIT_CALC
    ) & col_en;
assign fsm_init_out_tr       = (fsm_state_ff ==
    NN_UPD_RCO_TYPE      ) & (rco_sel == 2);
assign fsm_out_calc_tr       = (fsm_state_ff == NN_INIT_OUT
    );
assign fsm_out_rdy_tr        = (fsm_state_ff == NN_OUT_CALC
    ) & rdy_en;

// -----

assign m_en = (fsm_state_ff == NN_LOAD_IMG      )
    | (fsm_state_ff == NN_STRING_CALC      )
    | (fsm_state_ff == NN_TANG_STRING_CALC )
    | (fsm_state_ff == NN_OUT_CALC      )
    | (fsm_state_ff == NN_INIT) & start_i;

// -----

assign rdy_en      = (out_addr == WIDTH+1);
assign rst_str_cnt = fsm_upd_rco_type_tr;
assign tg_en       = (fsm_state_ff == NN_TANG_STRING_CALC );
assign rc_done     = (fc_addr == PICT_SIZE);
assign mem_en      = (fsm_state_ff == NN_STRING_RDY      );
assign str_en      = (fsm_state_ff == NN_TANG_STRING_CALC ) & (
    fc_addr == PICT_SIZE-1)
    | (fsm_state_ff == NN_OUT_CALC      ) & (
    fc_addr == PICT_SIZE-1);
assign fc_en       = (fsm_state_ff == NN_INIT_CALC      )
    | (fsm_state_ff == NN_STRING_CALC      ) & (
    fc_addr != PICT_SIZE)
    | (fsm_state_ff == NN_TANG_STRING_CALC )
    | (fsm_state_ff == NN_OUT_CALC      ) & (
    fc_addr != PICT_SIZE-1);

assign str_tg_en = (str_cnt == PICT_SIZE-1);

assign col_en     = (fc_addr == 0) & (str_cnt == PICT_SIZE)
    & (rco_sel != 1) | (fc_addr == PICT_SIZE) & (str_cnt ==
    PICT_SIZE) & (rco_sel == 1);

assign out_en      = (fsm_state_ff == NN_OUT_CALC      )
    | (fsm_state_ff == NN_INIT_OUT      )
    | (fsm_state_ff == NN_UPD_RCO_TYPE);

```

```

// -----
assign sel_data          = (fsm_state_ff == NN_LOAD_IMG) |
    (fsm_state_ff == NN_INIT) & start_i;

// -----
logic [RCO_TYPE-1:0] rco_sel_next;

always_ff @(posedge clk) begin
    if(~rst_n) begin
        rco_sel = RCO_TYPE'(0);
    end else if(start_i) begin
        rco_sel <= RCO_TYPE'(0);
    end else if(fsm_upd_rco_type_tr) begin
        rco_sel <= rco_sel_next;
    end else begin
        rco_sel <= rco_sel;
    end
end

assign rco_sel_next = rco_sel + RCO_TYPE'(1);

assign init = (fsm_state_ff == NN_LOAD_IMG          )
    | (fsm_state_ff == NN_LD_MEM                    )
    | (fsm_state_ff == NN_STRING_RDY                 )
    | (fsm_state_ff == NN_TANG_STRING_CALC           )
    | (fsm_state_ff == NN_INIT_OUT                   )
    | (fsm_state_ff == NN_UPD_RCO_TYPE               )
    ;

assign en    = (fsm_state_ff == NN_STRING_CALC)
    | (fsm_state_ff == NN_OUT_CALC    )
    | (fsm_state_ff == NN_OUT_RDY    )
    ;

endmodule

```

Модуль памяти изображения:

```

`timescale 1ns / 1ps

module img_mem
    import nn_param_pkg::*;
    (
        input  logic          clk, // clock
        input  logic          we, // clock
        input  logic          en, // clock
        input  logic [ADDR_OUT-1:0] addr,
        input  logic [INT_BITS + FRC_BITS-1:0] din,
        output logic [INT_BITS + FRC_BITS-1:0] dout
    );

    (* rom_style = "block" *) reg [INT_BITS + FRC_BITS-1:0]
        data [WIDTH-1:0];

    always @(posedge clk) begin
        if(we & en) begin
            data[addr] <= din;
        end
    end

    assign dout = (!we & en)? data[addr] : 0;

```



```
endmodule
```

Модуль обработки пикселя:

```
'timescale 1ns / 1ps
//
/////////////////////////////////////////////////////////////////

// Module Name: PE_rc0
// Project Name: LST-1 model
//
/////////////////////////////////////////////////////////////////

(* dont_touch = "yes" *) (* keep_hierarchy = "yes" *)

module PE_rc#(
    parameter int INT_BITS = 5, // integer part
    parameter int FRC_BITS = 7, // fractional part
    parameter int N = 0 // block number
)(
    input logic clk, // clock
    input logic init, // write initial value
    input logic en, // execute MAC-operation
    input logic [4:0] address_row,
    input logic [4:0] address_col,
    input logic rc_sel, // selects row, column or w_out
    memory block
    input [INT_BITS + FRC_BITS-1:0] din,
    output [INT_BITS + FRC_BITS-1:0] dout
);

// Internal signals
logic [INT_BITS + FRC_BITS-1:0] rom_row_out, rom_col_out,
    rom_w_out, mux_rom;
logic [4:0] addr_row, addr_col;
assign addr_row = (init)? 0 : address_row+1;
assign addr_col = (init)? 0 : address_col+1;

ROM_row #(.INT_BITS(INT_BITS), .FRC_BITS(FRC_BITS), .N(N))
    rom_row(.address(addr_row), .dout(rom_row_out), .clk(clk));
ROM_col #(.INT_BITS(INT_BITS), .FRC_BITS(FRC_BITS), .N(N))
    rom_col(.address(addr_col), .dout(rom_col_out), .clk(clk));

always_comb begin : MUX_ROM
    case (rc_sel)
        1'b0: mux_rom = rom_row_out;
        1'b1: mux_rom = rom_col_out;
    endcase
end

mac_core #(.INT_BITS(INT_BITS), .FRC_BITS(FRC_BITS))
    mac_inst(.din(din), .mem_in(mux_rom), .mac_out(dout
        ), .clk(clk), .init(init), .en(en));

endmodule

'timescale 1ns / 1ps
//
/////////////////////////////////////////////////////////////////
```

```

// Module Name: PE_rco
// Project Name: LST-1 model
//
/////////////////////////////////////////////////////////////////

(* dont_touch = "yes" *) (* keep_hierarchy = "yes" *)

module PE_rco#(
    parameter int INT_BITS = 5, // integer part
    parameter int FRC_BITS = 7, // fractional part
    parameter int N = 0 // block number
)(
    input logic clk, // clock
    input logic init, // write initial value
    input logic en, // execute MAC-operation
    input logic [4:0] address_row,
    input logic [4:0] address_col,
    input logic [9:0] address_o,
    input logic [1:0] rco_sel, // selects row, column or
    w_out memory block
    input [INT_BITS + FRC_BITS-1:0] din,
    output [INT_BITS + FRC_BITS-1:0] dout
);

// Internal signals
logic [INT_BITS + FRC_BITS-1:0] rom_row_out, rom_col_out,
    rom_w_out, mux_rom;
logic [4:0] addr_row, addr_col;
assign addr_row = (init)? 0 : address_row+1;
assign addr_col = (init)? 0 : address_col+1;

ROM_row #(.INT_BITS(INT_BITS), .FRC_BITS(FRC_BITS), .N(N))
    rom_row(.address(addr_row), .dout(rom_row_out), .clk(clk));
ROM_col #(.INT_BITS(INT_BITS), .FRC_BITS(FRC_BITS), .N(N))
    rom_col(.address(addr_col), .dout(rom_col_out), .clk(clk));
ROM_w_out #(.INT_BITS(INT_BITS), .FRC_BITS(FRC_BITS), .N(N))
    rom_w(.address(address_o), .dout(rom_w_out), .clk(clk));

always_comb begin : MUX_ROM
    unique case (rco_sel)
        2'b00: mux_rom = rom_row_out;
        2'b01: mux_rom = rom_col_out;
        2'b10: mux_rom = rom_w_out;
    endcase
end

mac_core #(.INT_BITS(INT_BITS), .FRC_BITS(FRC_BITS))
    mac_inst(.din(din), .mem_in(mux_rom), .mac_out(dout
    ), .clk(clk), .init(init), .en(en));

endmodule

    Модуль памяти строк:
timescale 1ns / 1ps

//
/////////////////////////////////////////////////////////////////

```

```

// WEIGHT MEMORY (ROM)
//
///////////////////////////////////////////////////////////////////

(* dont_touch = "yes" *) (* keep_hierarchy = "yes" *)

module ROM_row #(
    parameter int INT_BITS = 6, // integer part
    parameter int FRC_BITS = 7, // fractional part
    parameter int N = 0 // block number
)(
    input logic clk, // clock
    input logic [4:0] address,
    output [INT_BITS + FRC_BITS-1:0] dout
);

(* rom_style = "block" *) reg [INT_BITS + FRC_BITS-1:0]
    data;

generate
    if (N == 0) begin
        always @(posedge clk) begin
            case(address)
                5'b00000: data <= 13'h0000;
                ...
                default: data <= 0;
            endcase
        end
    end
endgenerate

generate
    if (N == 1) begin
        always @(posedge clk) begin
            case(address)
                5'b00000: data <= 13'h000d;
                ...
                default: data <= 0;
            endcase
        end
    end
endgenerate

...

generate
    if (N == 27) begin
        always @(posedge clk) begin
            case(address)
                5'b00000: data <= 13'h0027;
                ...
                default: data <= 0;
            endcase
        end
    end
endgenerate

```

```

    assign dout = data;
endmodule

```

Модуль памяти столбцов:

```

`timescale 1ns / 1ps

//
// //////////////////////////////////////
// WEIGHT MEMORY (ROM)
//
// //////////////////////////////////////

(* dont_touch = "yes" *) (* keep_hierarchy = "yes" *)

module ROM_col #(
    parameter int INT_BITS = 6, // integer part
    parameter int FRC_BITS = 7, // fractional part
    parameter int N = 0 // block number
)(
    input logic clk, // clock
    input logic [4:0] address,
    output [INT_BITS + FRC_BITS-1:0] dout
);

    (* rom_style = "block" *) reg [INT_BITS + FRC_BITS-1:0]
        data;

generate
    if (N == 0) begin
        always @(posedge clk) begin
            case(address)
                5'b00000: data <= 13'h0030;
                ...
                default: data <= 0;
            endcase
        end
    end
endgenerate

generate
    if (N == 1) begin
        always @(posedge clk) begin
            case(address)
                5'b00000: data <= 13'h0052;
                ...
                default: data <= 0;
            endcase
        end
    end
endgenerate

...

generate
    if (N == 27) begin
        always @(posedge clk) begin
            case(address)

```

```

        5'b00000: data <= 13'h1fec;
        ...
        default: data <= 0;
    endcase
end
end
endgenerate

    assign dout = data;
endmodule

```

Модуль памяти выходных весов:

```

`timescale 1ns / 1ps

```

```

//
//
// WEIGHT MEMORY (ROM)
//
//

```

```

(* dont_touch = "yes" *) (* keep_hierarchy = "yes" *)

```

```

module ROM_w_out #(
    parameter int INT_BITS = 5, // integer part
    parameter int FRC_BITS = 7, // fractional part
    parameter int N = 0 // block number
)(
    input logic clk, // clock
    input logic [9:0] address,
    output [INT_BITS + FRC_BITS-1:0] dout
);

    (* rom_style = "block" *) reg [INT_BITS + FRC_BITS-1:0]
        data;

generate
    if (N == 0) begin
        always @(posedge clk) begin
            case(address)
                10'b0000000000: data <= 13'h1ff2;
                ...
                default: data <= 0;
            endcase
        end
    end
endgenerate

generate
    if (N == 1) begin
        always @(posedge clk) begin
            case(address)
                10'b0000000000: data <= 13'h0006;
                ...
                default: data <= 0;
            endcase
        end
    end
endgenerate

```

```

        endgenerate

...

generate
    if (N == 9) begin
        always @(posedge clk) begin
            case(address)
                10'b0000000000: data <= 13'h1ff2;
                ...
                default: data <= 0;
            endcase
        end
    end
endgenerate

    assign dout = data;
endmodule

```

Модуль MAC:

```

`timescale 1ns / 1ps

(* use_dsp = "yes" *) (* dont_touch = "yes" *) (*
    keep_hierarchy = "yes" *)

module mac_core #(
    parameter int INT_BITS = 5, // integer part
    parameter int FRC_BITS = 7 // fractional part
)(
    input logic clk,           // clock
    input logic init,          // write initial value
    input logic en,            // execute MAC-operation
    input logic [INT_BITS + FRC_BITS-1:0] din,           // input
        data
    input logic [INT_BITS + FRC_BITS-1:0] mem_in,         // weight
        of NN
    output logic [INT_BITS + FRC_BITS-1:0] mac_out        //
        accumulator output
);
logic [INT_BITS + FRC_BITS-1:0] acc;                      // register-
    accumulator
logic [INT_BITS + FRC_BITS-1:0] m_o;                     // multiplier output
logic [2*(INT_BITS + FRC_BITS)-1:0] mul_out;
logic [2*(INT_BITS + FRC_BITS)-1:0] mul_out_1;
logic [2*(INT_BITS + FRC_BITS)-1:0] correct = 24'
    b0000000000000_000001000000;
//assign mul_out = signed'(din) * signed'(mem_in);
MUL_N #(.N(INT_BITS + FRC_BITS), .qA(FRC_BITS)) mul_inst(.A(
    din), .B(mem_in), .P(mul_out));

always_ff @(posedge clk) begin
    if (init) begin
        acc <= mem_in;
    end else begin
        if (en) begin
            acc <= m_o + acc; // MAC: A = A + B * C
        end
    end
end
end

```

```

assign mul_out_1 = mul_out; // + correct;
assign m_o = mul_out_1[INT_BITS + 2*FRC_BITS-1 : FRC_BITS];
assign mac_out = acc;
endmodule

```

Модуль умножителя:

```

--
-- -----

-- Company:
-- Engineer:
--
-- Create Date:      11:11:10 01/12/2008
-- Design Name:
-- Module Name:      ADD1 - Behavioral
-- Project Name:
-- Target Devices:
--
-- -----

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

---- Uncomment the following library declaration if
---- instantiating
---- any Xilinx primitives in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity ADD1 is
    Port ( A : in  STD_LOGIC;
           B : in  STD_LOGIC;
           Ci : in  STD_LOGIC;
           S : out STD_LOGIC;
           Co : out STD_LOGIC);
end ADD1;

architecture Behavioral of ADD1 is
begin
S<= (A xor B xor Ci);
Co<=(A and B)or(A and Ci)or (B and Ci);
end Behavioral;

--
-- -----

-- Company:
-- Engineer:
--
-- Create Date:      11:03:14 01/17/2008
-- Design Name:
-- Module Name:      HA - Behavioral
--
-- -----

```

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

---- Uncomment the following library declaration if
      instantiating
---- any Xilinx primitives in this code.
--library UNISIM;
--use UNISIM.VComponents.all;

entity HA is
    Port ( A : in  STD_LOGIC;
           B : in  STD_LOGIC;
           S : out  STD_LOGIC;
           Co : out  STD_LOGIC);
end HA;

architecture Behavioral of HA is

begin
S<=A xor B;
Co<=A and B;
end Behavioral;

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity MUL_N is
    generic (N:natural:=16;
             qA:natural:=8); -- fractional part of A (
                               to remove from product)
    port ( A : in  STD_LOGIC_VECTOR (N-1 downto 0);
           B : in  STD_LOGIC_VECTOR (N-1 downto 0);
           P : out  STD_LOGIC_VECTOR (2*N-1 downto 0) --
               full multiplier
--           P : out STD_LOGIC_VECTOR(N-1 downto
--               0) -- half of output bits is needed
    );
end MUL_N;

architecture Behavioral of MUL_N is
type abij is array(0 to N-1) of std_logic_vector(N-1 downto
0);
component ADD1
    Port ( A : in  STD_LOGIC;
           B : in  STD_LOGIC;
           Ci : in  STD_LOGIC;
           S : out  STD_LOGIC;
           Co : out  STD_LOGIC);
end component;
component HA
    Port ( A : in  STD_LOGIC;
           B : in  STD_LOGIC;
           S : out  STD_LOGIC;
           Co : out  STD_LOGIC);
end component;

```



```

signal ab,s,c: abij;
signal c_out: std_logic_vector(N-1 downto 0);
signal p_out: std_logic_vector(2*N-1 downto 0);
begin

row: for i in 0 to N-1 generate
begin
col: for j in 0 to N-1 generate
begin
n8: if ((j=N-1)and(i/=N-1))OR
((i=N-1)and(j/=N-1))
generate
begin
ab(i)(j)<=
not(A(j)
and B(i));
end generate;
a8: if ((j/=N-1)and(i/=N-1))
OR((i=N-1)and(j=N-1))
generate
begin
ab(i)(j)<=(A(
j) and B(i)
);
end generate;
end generate;
end generate;
s(0)<=ab(0);
c(0)<=(others=>'0');
mul: for i in 0 to N-2 generate
begin
sm: for j in 0 to N-1 generate
begin
one: if (j=0) generate
ad: HA port map (A=>s
(i)(j),B=>c(i)(j),S
=>p_out(i),Co=>c(i
+1)(j));
end generate;
oth: if (j/=0) generate
ad: ADD1 port map (A
=>s(i)(j),B=>ab(i
+1)(j-1),Ci=>c(i)(j
),S=>s(i+1)(j-1),Co
=>c(i+1)(j));
end generate;
end generate;
s(i+1)(N-1)<=ab(i+1)(N-1);
end generate;
mul_end:
for i in 0 to N-1 generate
begin
fst: if (i=0) generate
ad: HA port map (A=>s(N-1)(i),B=>c(N
-1)(i),S=>p_out(N-1+i),Co=>c_out(i
));
end generate;
one: if (i=1) generate
ad: ADD1 port map (A=>s(N-1)(i),B=>'1',Ci

```

```

=>c(N-1)(i),S=>p_out(N-1+i),Co=>c_out(i
));
    end generate;
oth: if ((i/=0)and (i/=1)) generate
    ad: ADD1 port map (A=>s(N-1)(i),B=>c_out(
    i-1),Ci=>c(N-1)(i),S=>p_out(N-1+i),Co=>
    c_out(i));
    end generate;
end generate;
p_out(2*N-1) <= c_out(N-1);
P<=p_out;
    full integer multiplier
--P<=p_out(2*(N-1) downto N-1);-- half of product fractional
multiplier
--P<=p_out(N-1 downto 0);-- half of product fractional
multiplier
--P<=p_out(N+(N/2)-1 downto N/2);-- half of product
--P<=p_out(N+qA-1 downto qA);-- half of product
end Behavioral;

```

Модуль тангенса:

```

'timescale 1ns / 1ps
//
//
// Create Date: 29.12.2024 18:44:00
// Design Name:
// Module Name: tanh_function
// Project Name: L2DST-one-block-NN
//
//
module tanh_function #(
    parameter int INT_SIZE = 3, // integer part
    parameter int FRC_SIZE = 8 // fractional part
) (
    input [(INT_SIZE+FRC_SIZE - 1):0] X,
    output logic [(INT_SIZE+FRC_SIZE - 1):0] Y
);
    localparam logic [FRC_SIZE-1:0] zero_string = '0;
    localparam logic [FRC_SIZE-1:0] ones_string = '1;

    localparam const_plus_two = {3'b010, zero_string};
    localparam const_minus_two = {3'b110, zero_string};
    localparam const_q_plus_one = {1'b0, ones_string};
    localparam const_q_minus_one = {1'b1, zero_string};

    logic [FRC_SIZE:0] add_sub_out;
    logic [INT_SIZE+FRC_SIZE-1:0] add_sub_out_ext;
    // logic [2*(INT_SIZE+FRC_SIZE)-1:0] mul_out;
    logic [(INT_SIZE+FRC_SIZE-1):0] x_quater;
    logic x_sign;

    logic [INT_SIZE + FRC_SIZE-1:0] m_o; // multiplier output
    logic [2*(INT_SIZE + FRC_SIZE)-1:0] mul_out;

```

```

logic [2*(INT_SIZE + FRC_SIZE)-1:0] mul_out_1;
logic [2*(INT_SIZE + FRC_SIZE)-1:0] correct = 26'
    b000000_0000_0000_0000_0100_0000;

    assign x_sign = X[INT_SIZE+FRC_SIZE-1];

    always_comb begin : Tanh_approximation
//      if (x_sign == 0) begin // (x>0) branch
        if ($signed(X) >= $signed(const_plus_two)) Y = 13'
            b00000_1000_0000;
        else if ($signed(X) <= $signed(const_minus_two)) Y =
            13'b11111_1000_0000;
        else Y = m_o;
//      end
    end

    assign x_quater = {x_sign, x_sign, X[INT_SIZE+FRC_SIZE
        -1:2]};

    always_comb begin : Add_Sub_block
        if (x_sign == 0) add_sub_out = const_q_plus_one -
            x_quater[FRC_SIZE:0];
        else add_sub_out = const_q_plus_one + x_quater[FRC_SIZE
            :0];
    end

    assign add_sub_out_ext = {add_sub_out[FRC_SIZE],
        add_sub_out[FRC_SIZE], add_sub_out[FRC_SIZE], add_sub_out
        [FRC_SIZE], add_sub_out};

    MUL_N #(
        .N (INT_SIZE + FRC_SIZE),
        .qA(FRC_SIZE)
    ) mult_inst (
        .A(add_sub_out_ext),
        .B(X),
        .P(mul_out)
    );

    assign mul_out_1 = mul_out; // + correct;
    assign m_o = mul_out_1[INT_SIZE + 2*FRC_SIZE-1 : FRC_SIZE];

endmodule

```

Модуль softmax:

```

`timescale 1ns / 1ps

```

```

//
// //////////////////////////////////////
//
// SOFTMAX activation function
// This code find max element of 10 input elements,
// but don't use exp() and as a result don't use probability.
// This module just finde max elements whithout any
// transformations.
//
// //////////////////////////////////////

```

```

//(*use_dsp = "yes"*)
module softmax#(
    parameter int BITS = 24, // bit depth
    parameter int HEIGHT = 10 // size of array_weight
)(
    //    input logic clk, //clock
    //    input logic reset, // reset
    input logic [BITS - 1 : 0] result_layer [HEIGHT-1:0], //
        input 10 elements from full connected layer
    output logic [BITS - 1 : 0] predict_num // predict number
);

// Internal signal
logic [BITS - 1 : 0] max;

always_comb begin
    max = result_layer[0];
    predict_num = 0;
    for (int i = 1; i < HEIGHT; i++) begin
        if (signed'(result_layer[i]) > signed'(max)) begin
            max = result_layer[i];
            predict_num = i;
        end
    end
end

end
endmodule

```

ПРИЛОЖЕНИЕ Д (обязательное) Python скрипты тестирования

Python описание сети для консольного приложения:

```
import tkinter as tk
from PIL import Image, ImageDraw
import numpy as np
import cv2

screen_size = (512, 512)

image = Image.new("L", screen_size, 0)
draw = ImageDraw.Draw(image)

last_x, last_y = None, None

def draw_line(event):
    global last_x, last_y
    canvas.create_line(last_x, last_y, event.x, event.y, fill
        ="white", width=15)

    draw.line([last_x, last_y, event.x, event.y], fill=255,
        width=15)

    last_x, last_y = event.x, event.y

def set_last_xy(event):
    global last_x, last_y
    last_x, last_y = event.x, event.y

def clear_canvas():
    global image, draw
    canvas.delete("all")
    canvas.create_rectangle(0, 0, screen_size[0], screen_size
        [1], fill="black")

    image = Image.new("L", screen_size, 0)
    draw = ImageDraw.Draw(image)

def save_image():
    np_img = np.array(image)
    img = process_single_digit(np_img)

    img = Image.fromarray(img)
    img.save("drawing.png")
    print("Image saved as 'drawing.png'")

def process_single_digit(image):
    resized_digit = cv2.resize(image, (28, 28))
    return resized_digit

root = tk.Tk()
root.title("Drawing Numbers")

canvas = tk.Canvas(root, width=screen_size[0], height=
    screen_size[1], bg="black")
```

```

canvas.pack()

canvas.bind("<Button-1>", set_last_xy)
canvas.bind("<B1-Motion>", draw_line)

btn_clear = tk.Button(root, text="Clear", command=
    clear_canvas)
btn_clear.pack(side=tk.LEFT, padx=10, pady=10)

btn_save = tk.Button(root, text="Save", command=save_image)
btn_save.pack(side=tk.RIGHT, padx=10, pady=10)

clear_canvas()

root.mainloop()

```

Python описание сети для сбора матрицы спутывания:

```

from pynq import Overlay
from PIL import Image
from time import sleep
import matplotlib.pyplot as plt

# connect network
platform = Overlay("design_1.bit")
neural_net = platform.m_nn_0

# Address for in and out ports
NEURAL_NET_IN_ADDR = 3 * 4
NEURAL_NET_IN_COUNT = 4 * 4
NEURAL_NET_OUT_ADDR = 5 * 4
NEURAL_NET_IN_ADDR_RESET = 6 * 4

image_list = ["pic5.png", "pic9.png", "pic4.png", "pic1.png",
    "pic8.png"]
for file_name in image_list:
    neural_net.write(NEURAL_NET_IN_ADDR_RESET, int(0))
    image = Image.open(file_name)
    pixel_values = list(image.getdata())
    width, height = image.size
    pixel_val = [(pixel - 127.5) / 127.5 for pixel in
        pixel_values]
    pixel_values = [pixel_val[i * width: (i + 1) * width]
        for i in range(height)]

    pixel_count = 1
    output = neural_net.read(NEURAL_NET_OUT_ADDR)
    neural_net.write(NEURAL_NET_IN_ADDR_RESET, 1)
    output = neural_net.read(NEURAL_NET_OUT_ADDR)
    for i in range(height):
        for j in range(width):
            neural_net.write(NEURAL_NET_IN_COUNT, pixel_count
                )
            # send pixel in Q12.12
            neural_net.write(NEURAL_NET_IN_ADDR, int(
                pixel_values[i][j] * 2**12))
            pixel_count = pixel_count + 1

    output = neural_net.read(NEURAL_NET_OUT_ADDR)

```

```
plt.imshow(image)
plt.title(f'NN_output={output}')
plt.colorbar()
plt.show()
print (output)
neural_net.write(NEURAL_NET_IN_ADDR_RESET, int(0))
```

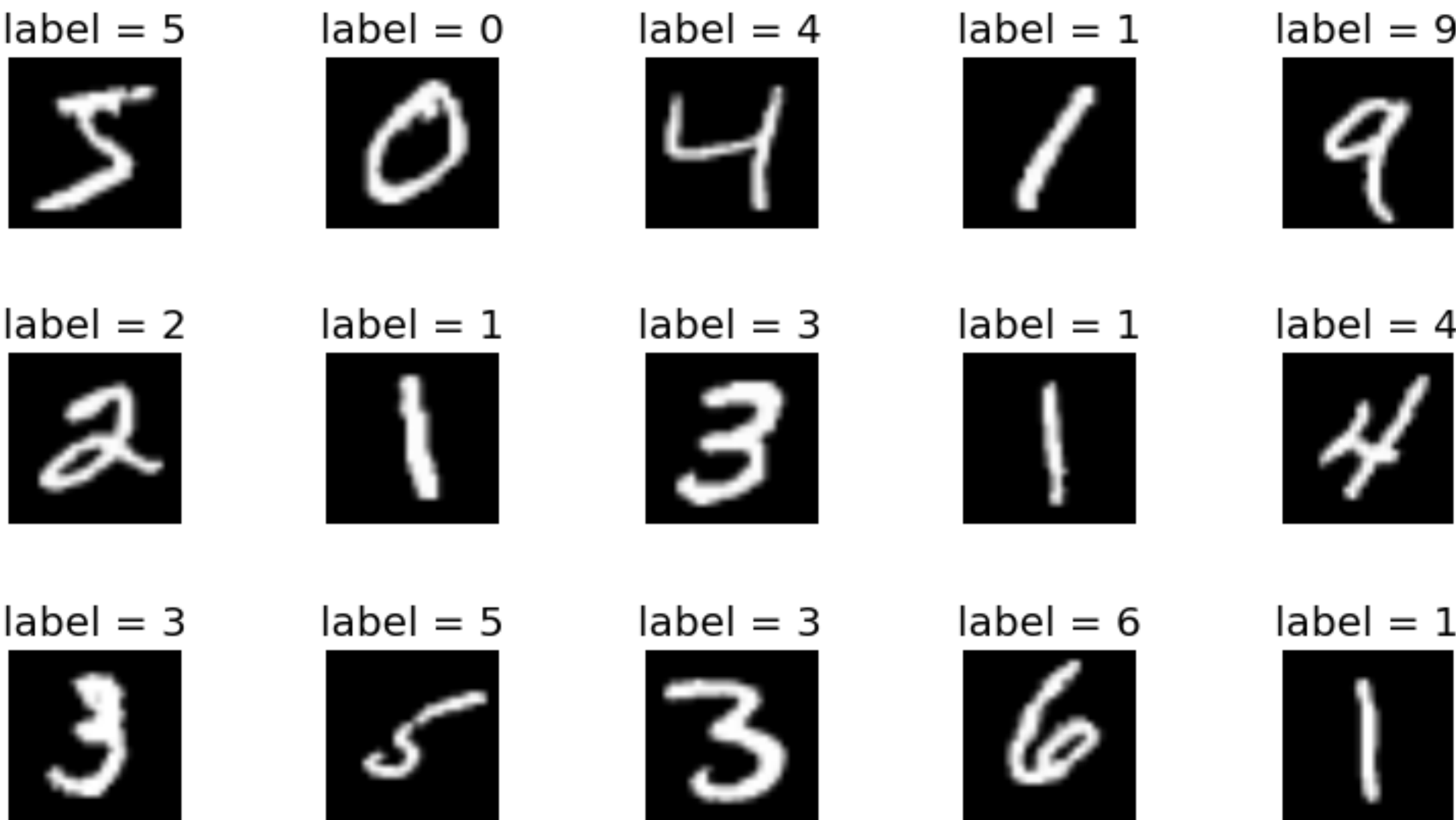
Перв. примен.		Зона	Обозначение				Наименование				Дополнительные сведения		
							Текстовые документы						
			БГУИР ДП 1-40 02 02 01 012 ПЗ				Пояснительная записка				86 с.		
Справочный №							Отзыв руководителя				1 с.		
							Рецензия				1 с.		
							Графические документы						
			ГУИР.431289.001 ПЛ				Постановка задачи построения				Формат А1		
							IP-блока нейронной сети						
			ГУИР.431289.002 Э1				Схема электрическая структурная				Формат А2		
							IP-блока нейронной сети						
			ГУИР.431289.003 Э1				Схема электрическая функциональная				Формат А1		
							IP-блока нейронной сети						
			ГУИР.431289.004 Э1				Схема электрическая функциональная				Формат А2		
							процессорных элементов						
			ГУИР.431289.005 ПД				Схема алгоритма работы IP-блока				Формат А1		
							Нейронной сети						
			ГУИР.431289.006 ПЛ				Результаты проектирования				Формат А1		
							IP-блока нейронной сети						
Подпись и дата							ГУИР.431289.007 ПЛ				Формат А1		
							IP-блока нейронной сети						
Инв. № дубл.													
Взам. Инв. №													
Подпись и дата													
Инв. № подл.							БГУИР ДП 1-40 02 02 01 012 Д1						
		Изм.	Лист	№ докум.	Подп.	Дата							
		Разраб.	Кривальцевич				IP-ядро нейронной сети прямого распространения для распознавания рукописных цифр				Лит.	Лист	Листов
Пров.	Вашкевич				Т	86					86		
Т.Контр	Вашкевич				ЭВС, гр. 150701								
Н.контр.	Лихачёв												
Утв.	Азаров												

Постановка задачи

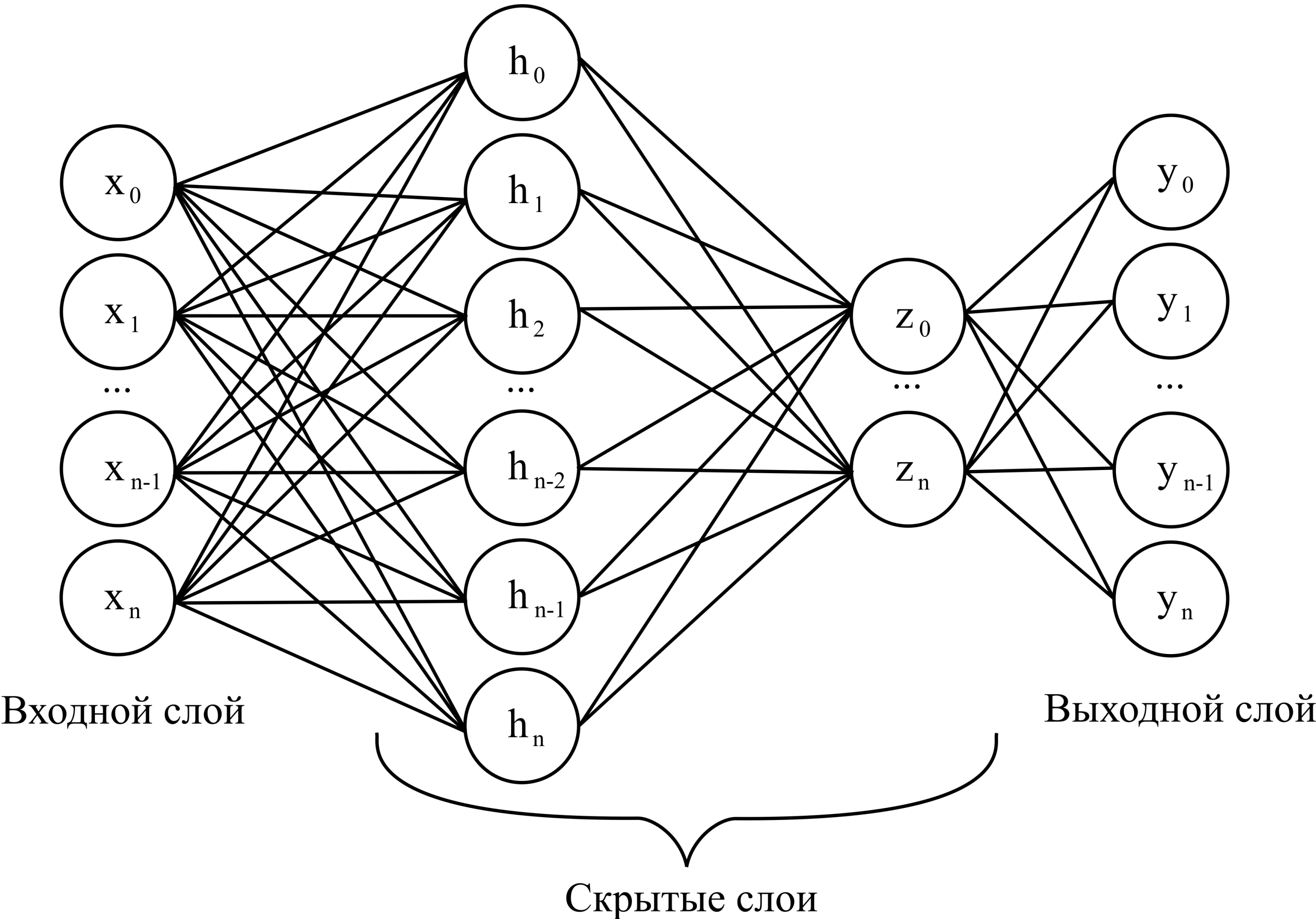
Пример полносвязных архитектур

Автор	Архитектура НС	# Число параметров	Точность
Liang, et al. (2018)	784-2048-2048-2048-10	10 100 000	98.32%
Umuroglu, et al. (2017)	784-1024-1024-10	1 863 690	98.40%
Medus, et al. (2019)	784-600-600-10	891 610	98.63%
Huynh	784-126-126-10	115 920	98.16%
Huynh	784-40-40-40-10	34 960	97.20%
Westby, et al.	784-12-10	9 550	93.25%

Пример изображений



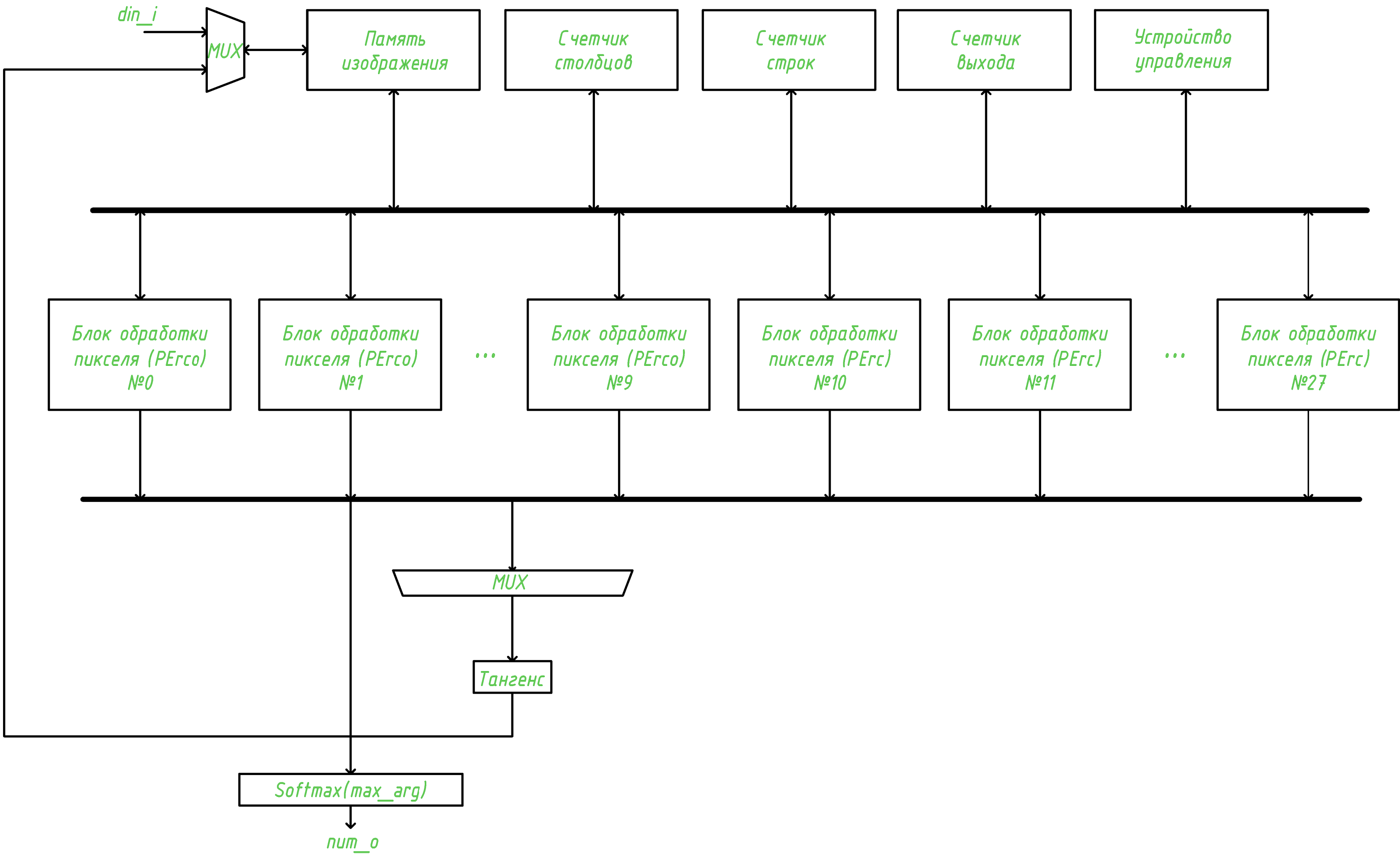
Пример полносвязной архитектуры

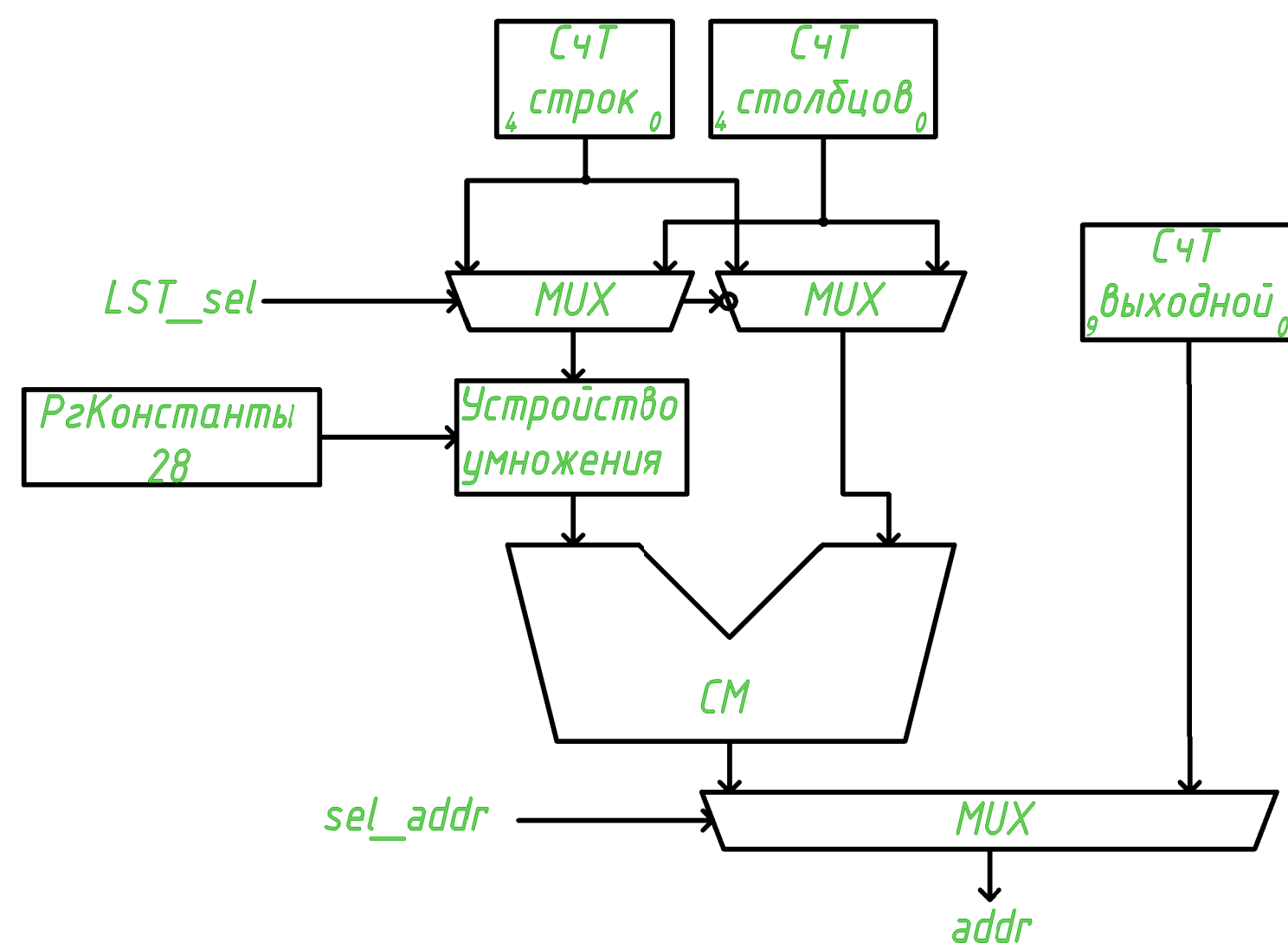


Сверточная нейронная сеть, точность 88% и 23520 параметров

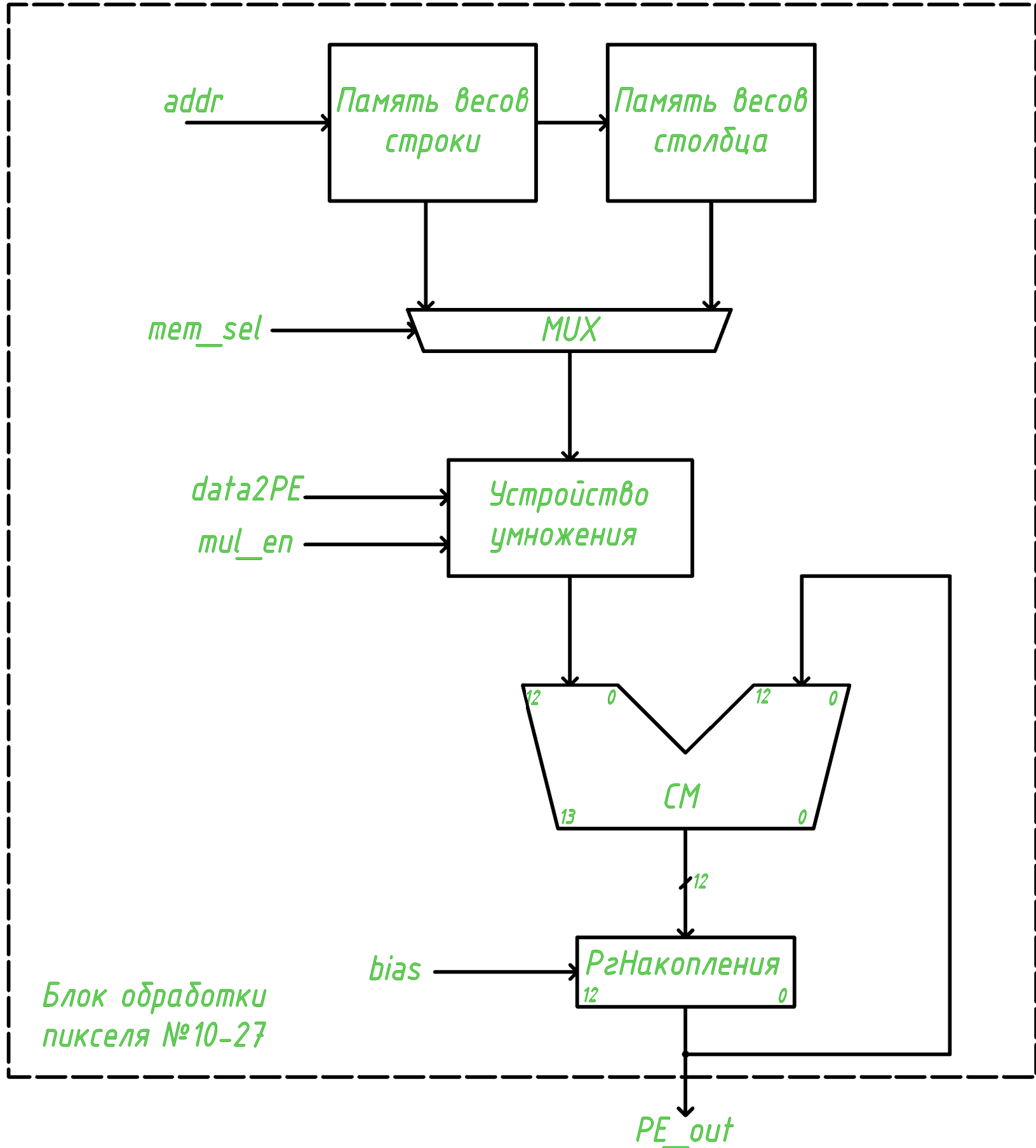
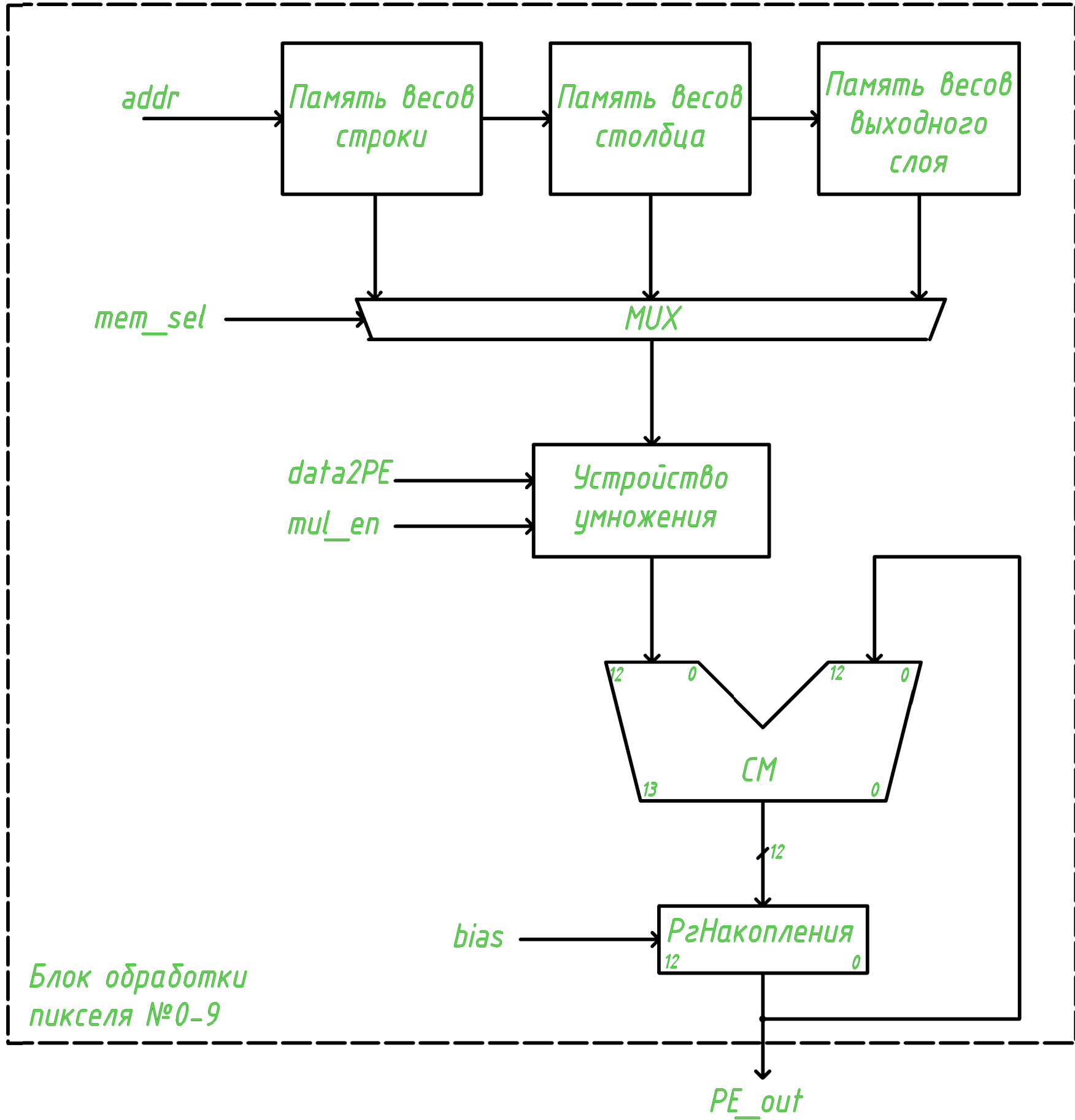
		Detected class									
		0	1	2	3	4	5	6	7	8	9
Target Class	0	99.5%	0.0%	0.2%	0.0%	0.0%	0.0%	0.0%	0.0%	0.1%	0.2%
	1	3.6%	89.4%	0.5%	1.0%	2.0%	0.0%	0.0%	3.1%	0.2%	0.2%
	2	4.1%	1.6%	90.0%	2.0%	0.3%	0.2%	0.0%	0.4%	0.8%	0.6%
	3	1.1%	0.3%	2.4%	90.8%	0.1%	0.2%	0.1%	0.4%	0.3%	4.3%
	4	1.3%	0.3%	0.0%	0.1%	97.6%	0.0%	0.4%	0.2%	0.1%	0.0%
	5	1.2%	0.0%	0.3%	22.0%	0.4%	68.0%	2.4%	0.2%	0.7%	4.8%
	6	6.4%	0.4%	0.4%	1.1%	6.2%	0.6%	82.4%	0.7%	1.0%	0.8%
	7	0.5%	0.4%	7.4%	0.2%	0.4%	0.0%	0.5%	90.1%	0.0%	0.5%
	8	1.5%	0.1%	0.2%	6.7%	1.8%	0.5%	0.9%	0.9%	87.3%	0.1%
	9	8.7%	0.2%	0.0%	0.8%	0.7%	0.3%	0.0%	3.9%	0.5%	84.9%

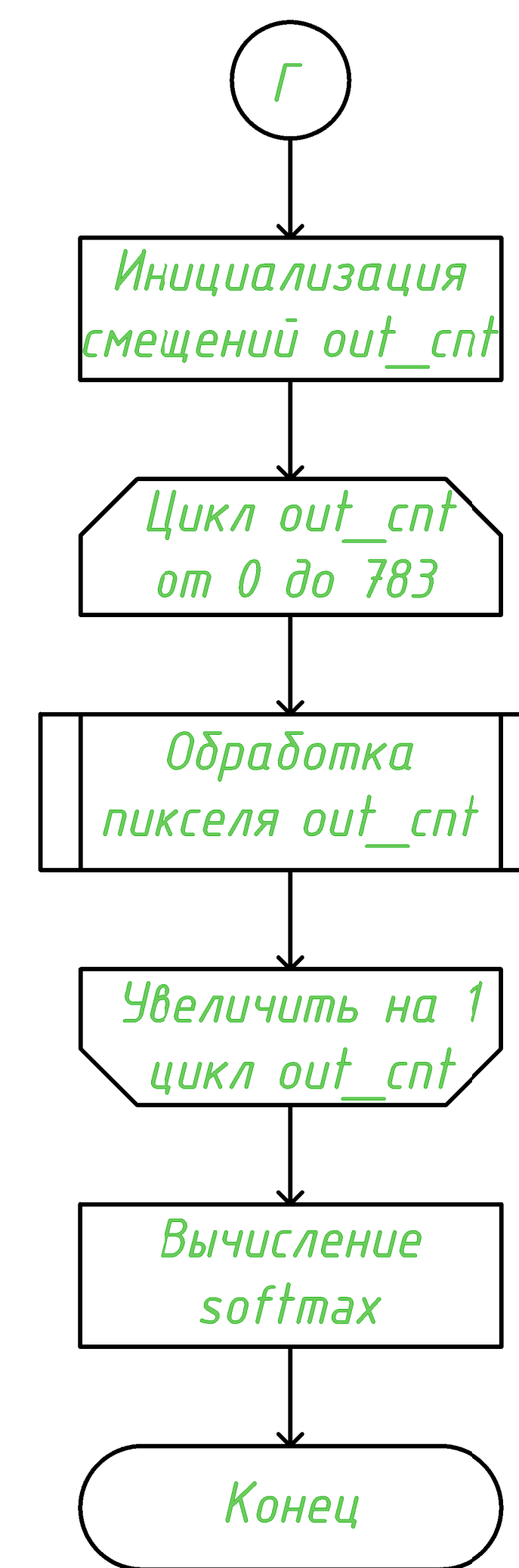
ГЧИР.431289.002 Э1





ГЧИР.431289.004 Э1



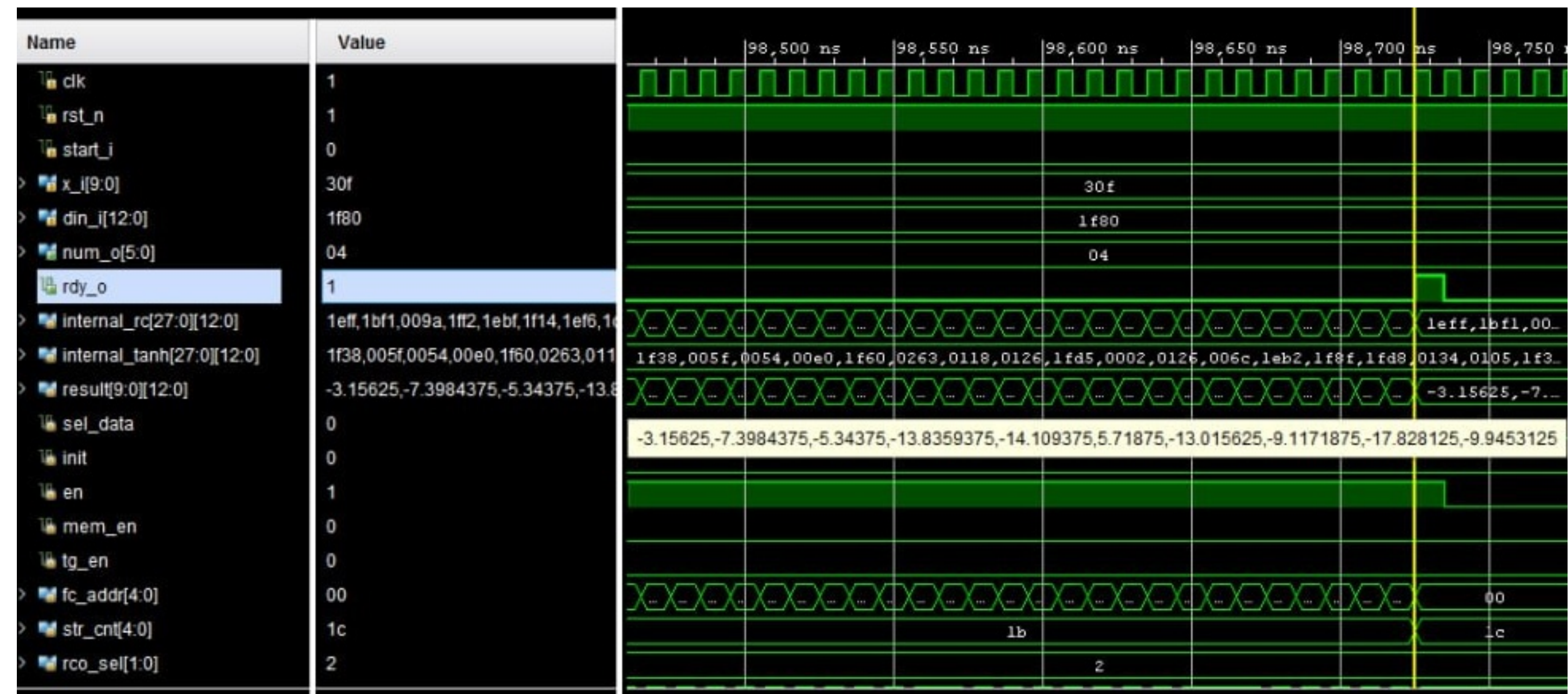


Результаты проектирования

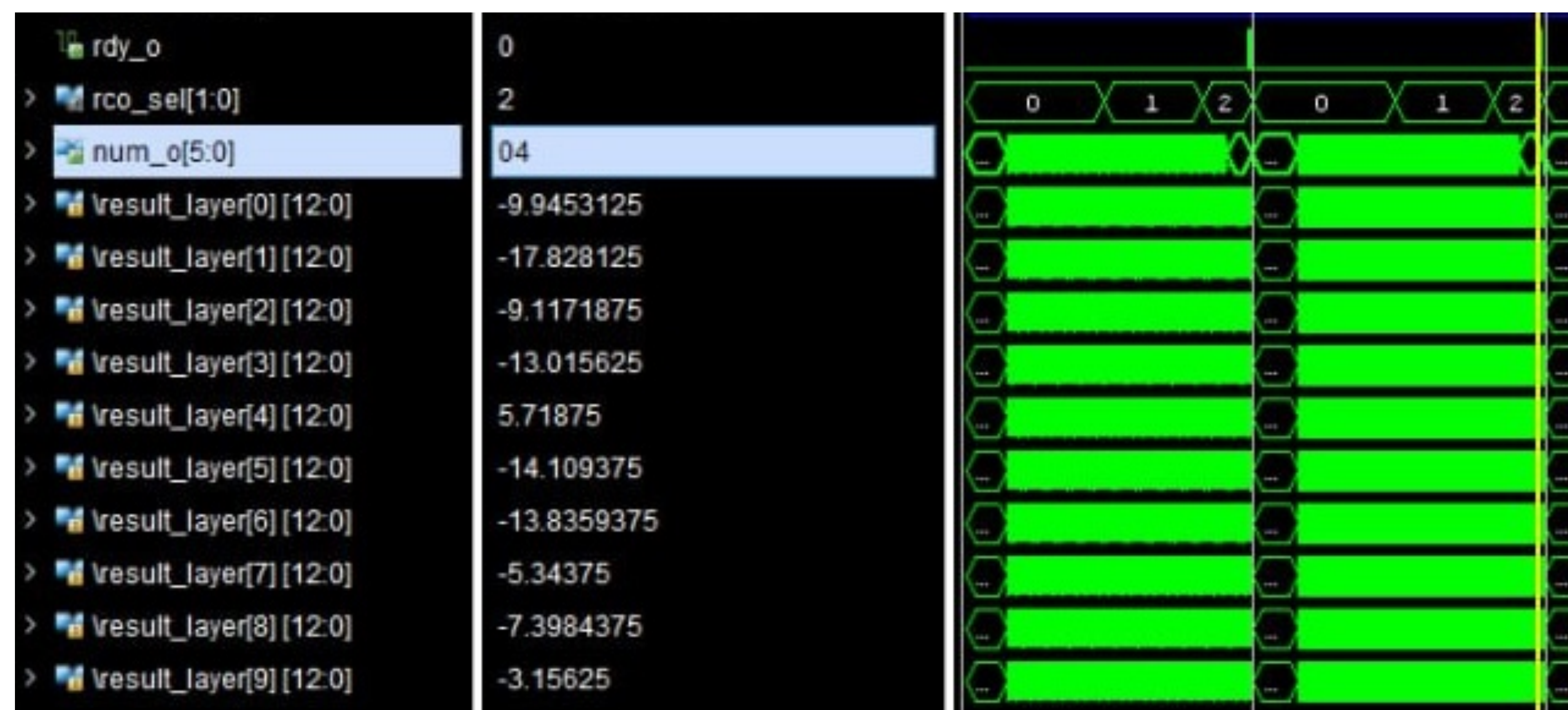
Результаты симмуляции

Аппаратные затраты

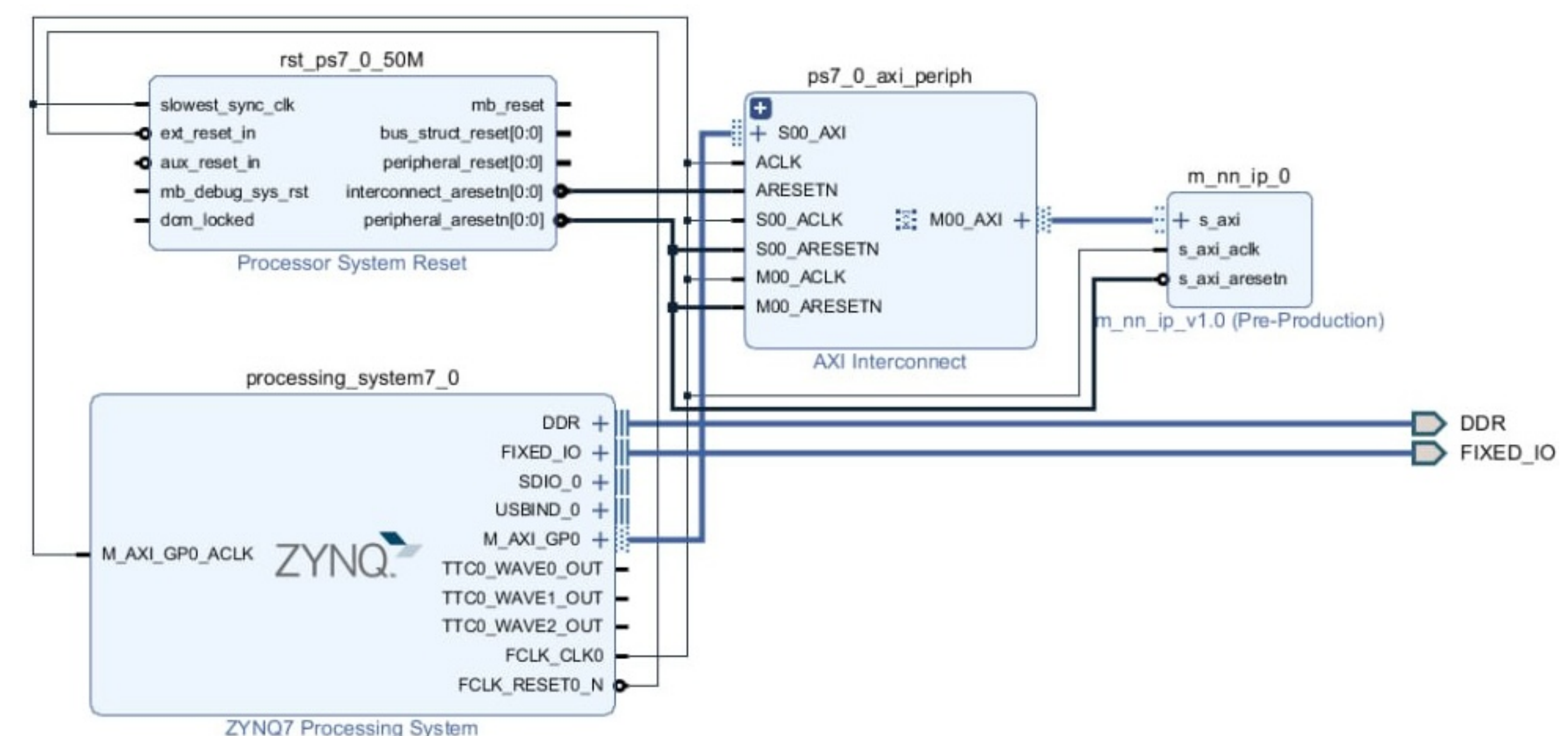
Utilization			
Post-Synthesis Post-Implementation			
Graph Table			
Resource	Estimation	Available	Utilization %
LUT	1538	17600	8.74
LUTRAM	169	6000	2.82
FF	27	35200	0.08
BRAM	33	60	55.00
DSP	57	80	71.25
IO	33	100	33.00
BUFG	2	32	6.25



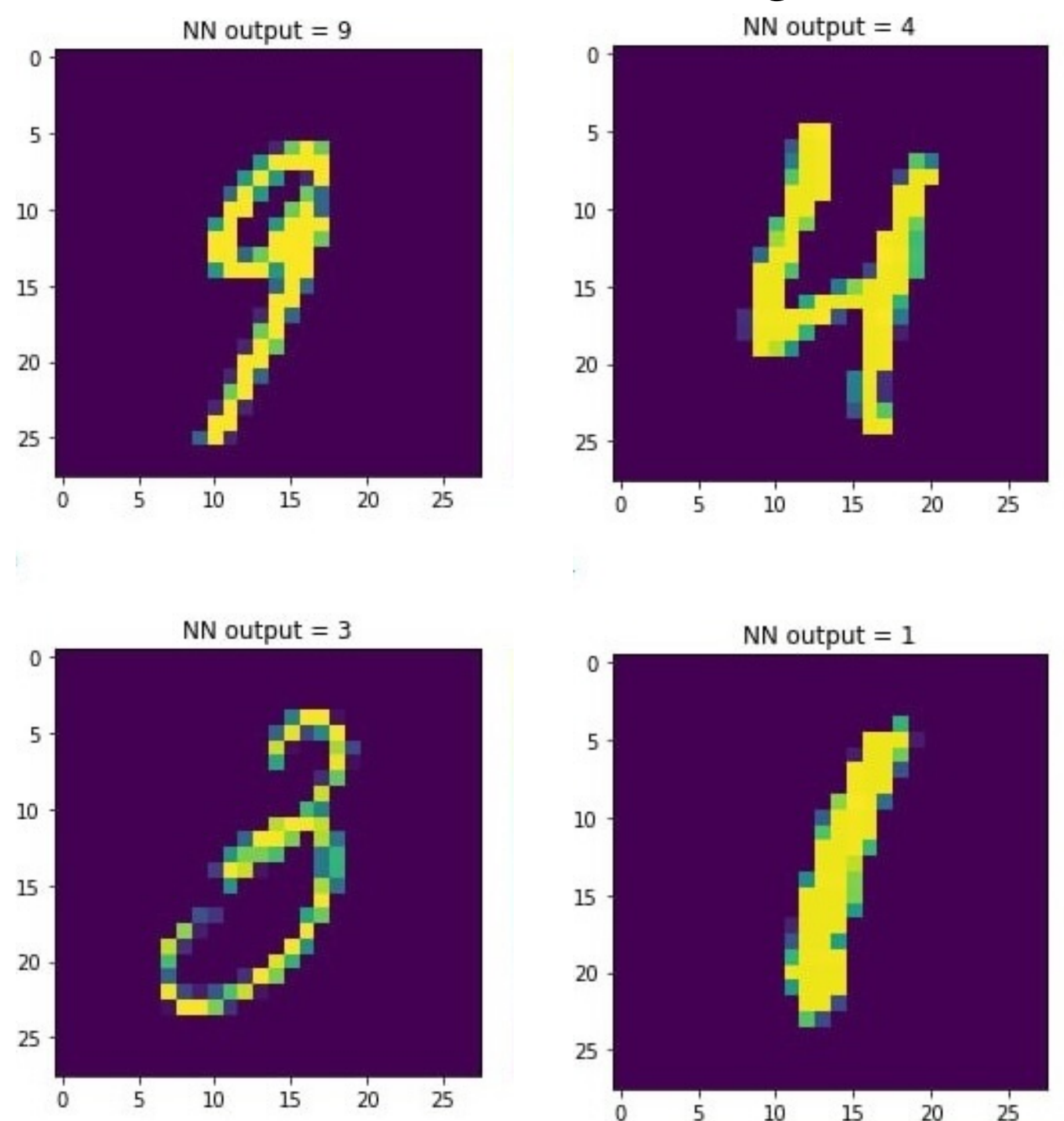
Результаты пост-синтез симмуляции



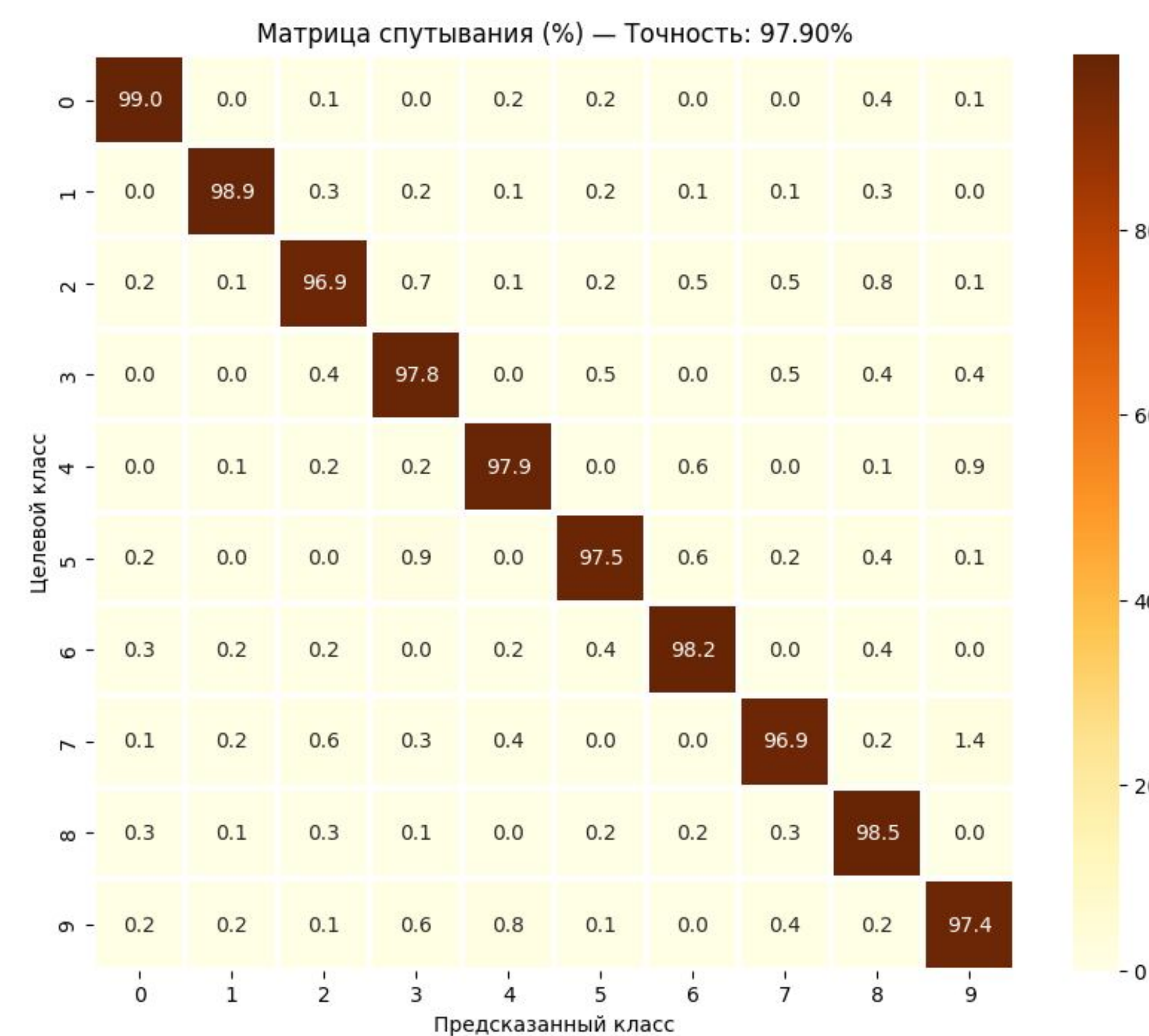
Блочная диаграмма



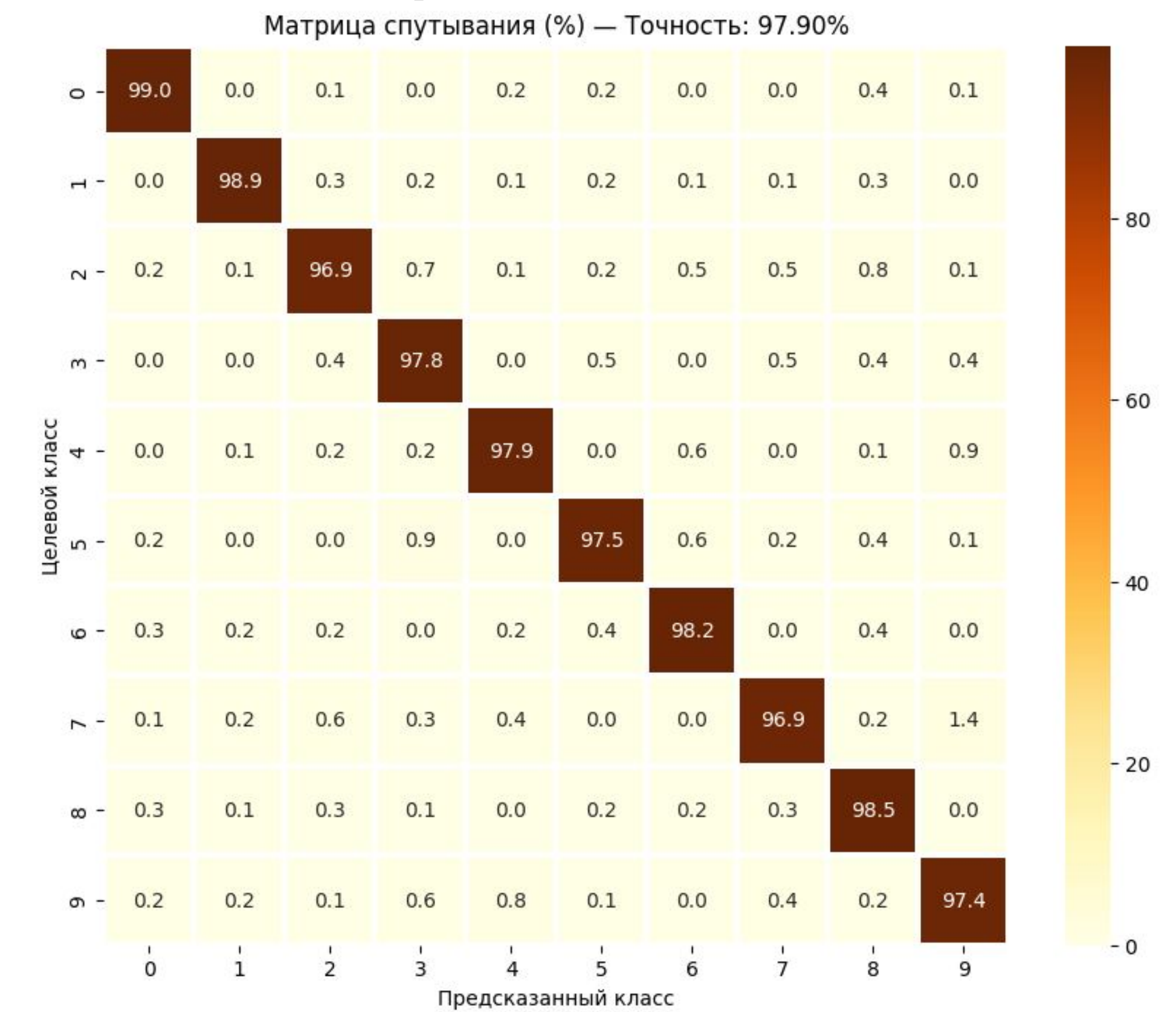
Результаты экспериментов



Матрица fixpoint



Матрица FPGA



Самописные цифры

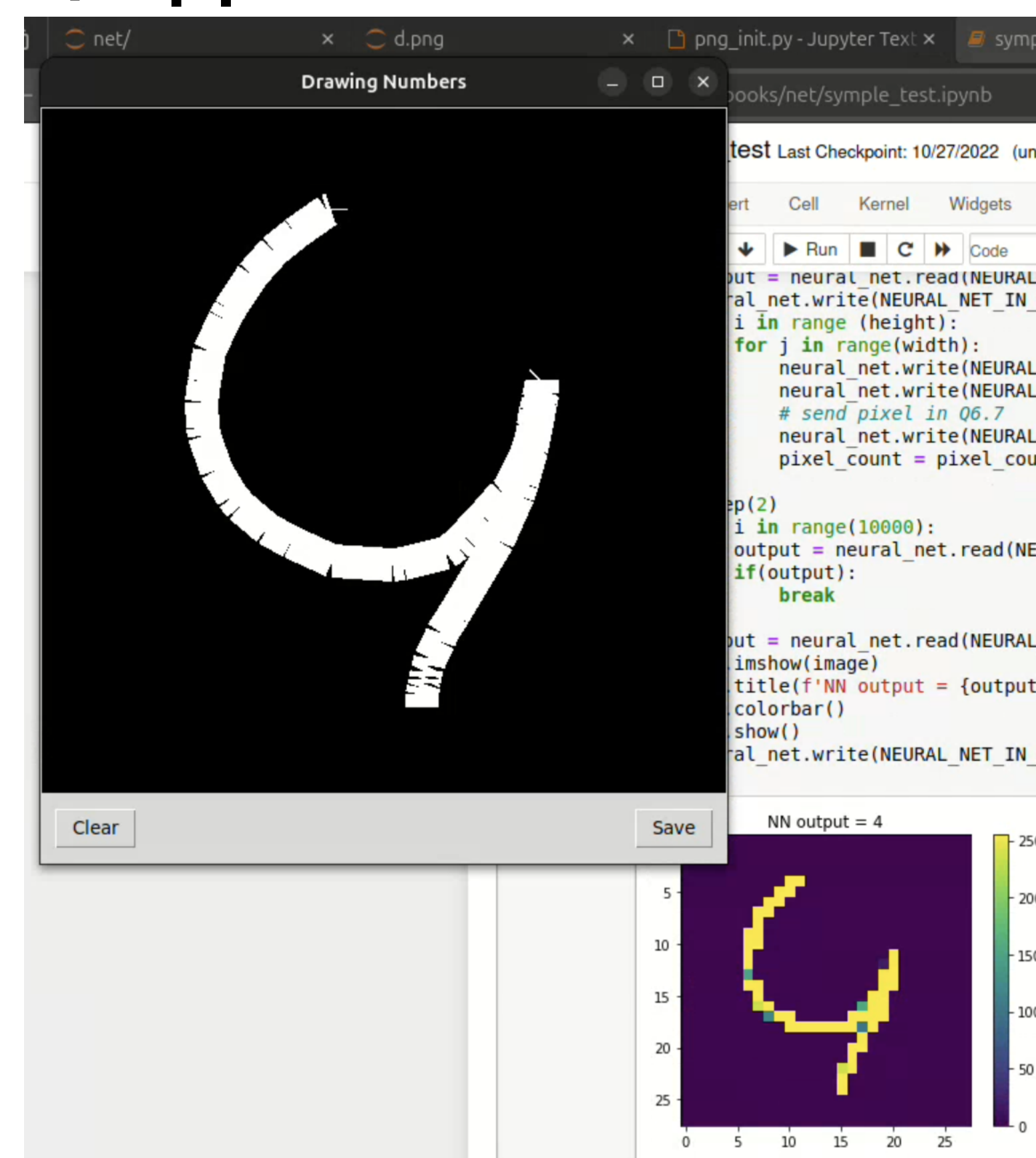
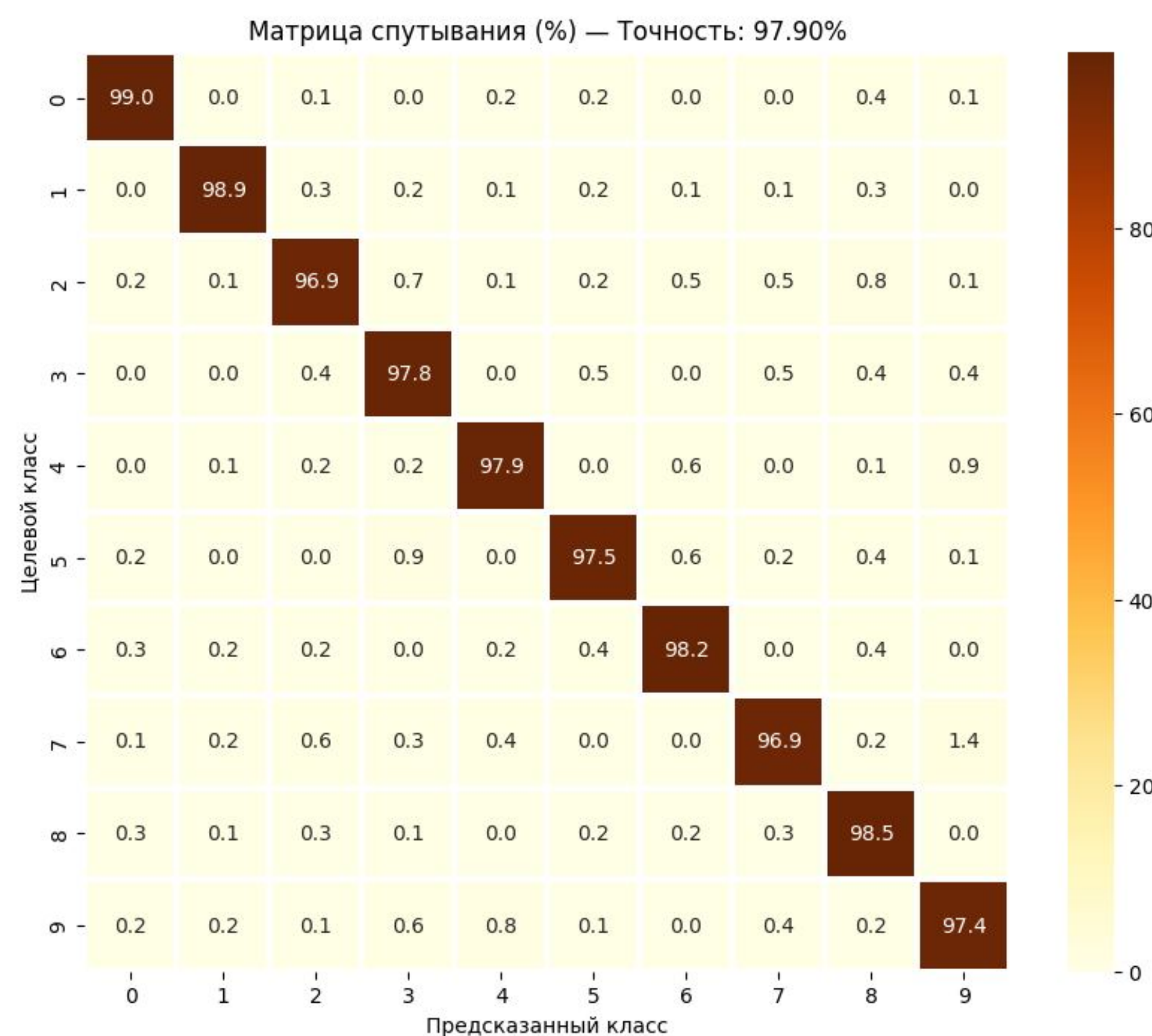


График точности

